Beginning C++ Programming - From Beginner to Beyond

David Corzo

2020 May 18

Índice general

Capítulo 1

Introduction

1.1. Why learn C++?

- Popular:
  - Lots of code is still written in C++.
  - Programming language popularity indexes ranks C++ high.
  - Active community, GitHub, Stack overflow.
- Relevant:
  - Windows, Linux, MaxOSX, Photoshop, Illustrator, MySQL, MongoDB.
  - Amazon, Apple, Microsoft, PayPal, Google, Facebook, MySQL, Oracle, HP, IBM, more...
  - VR, Unreal Engine, Machine learning, networking & telecom, more...
- Powerful:
  - Super-fast, scalable, portable.
  - Supports both procedural and object-oriented programming.
- Good career opportunities:
  - C++ skills always in demand.
  - C++ = Salary++.

1.2. Modern C++ and the C++ standard

- Early 1970s: C programming language; Dennis Ritchie.
- 1979: Bjarne Stroustrup; 'C with classes'.
- 1983: Name changed to C++.
- 1989:First commercial release.
- 1998: C++98 Standard.
- 2003: C++03 Standard.
- 2011: C++11 Standard.
- 2014: C++14 Standard.
- 2017: C++17 Standard.

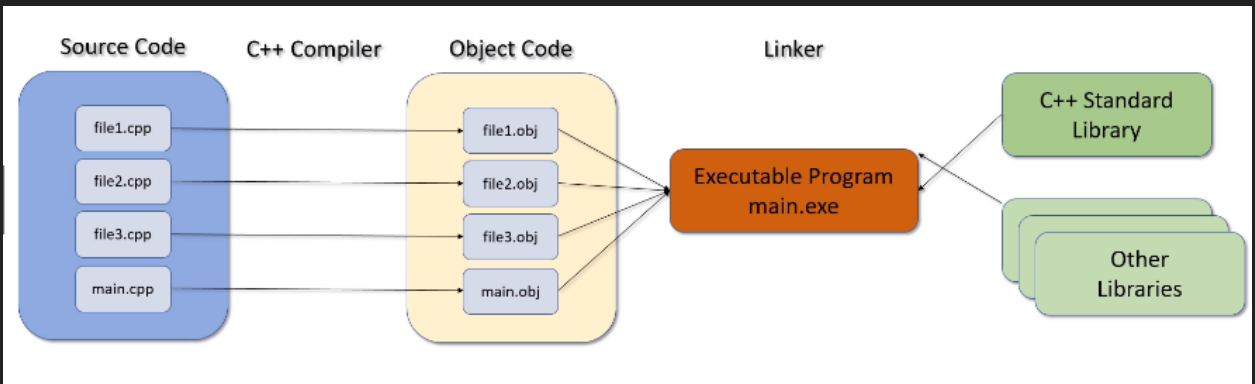
1.2.1. Modern C++ and C++ Standard

- Classical C++: Pre C++11 Standard.
- Modern C++:
  - C++11: Lots of new features.
  - C++14: Smaller changes.
  - C++17: Simplification.

1.3. How does it all work?

- Use non-ambiguous instructions.
- Programming language: source code, high level, for humans.
- Editor: text editor. *.cpp* and *.h* files.
- Binary or other low level representation: object code for computers.
- Compiler: Translates from high-level to low-level.
- Linker: links together our code with other libraries, creates *.exe*.
- Testing and debugging: finding and fixing program errors.

1.3.1. The C++ build process



1.3.2. Integrated Development Environments (IDEs)

- Editor.
- Compiler.
- Linker.
- Debugger.
- Keep everything in sync.

IDEs

- CodeLite.
- Code::Blocks.
- NetBeans.
- Eclipse.
- CLion.
- Dev-C++.
- KDevelop.
- Visual Studio.
- Xcode.

Capítulo 2

Installation and setup

2.1. Installing C++ Compiler for windows

- Go to: `http://mingw-w64.org/doku.php/download/mingw-builds`
- Go to: Downloads, find the build, download and run executable.
- Set the environment variable:
  - Control panel → Edit system environment variables.
  - Environment variables → System → Path → Edit.
  - New → Browse → < go to your instaltion dir > → OK.
- Go to CMD: type `c++ --version` → Should print version.

2.2. VSCode Project Setting Up

Inside the .vscode directory add:

- `c_cpp_properties.json`:

```
{
    "configurations": [
        {
            "name": "Win32",
            "includePath": [
                "${workspaceFolder}/**"
            ],
            "defines": [
                "_DEBUG",
                "UNICODE",
                "_UNICODE"
            ],
            "browse": {
                "path": [
                    "${workspaceRoot}",
                    "C:\\Program Files\\mingw-w64\\mingw64\\bin\\gcc.exe"
                ]
            },
            "compilerPath": "C:\\Program Files\\mingw-w64\\mingw64\\bin\\gcc.exe",
            "cStandard": "gnu18",
```

```
            "cppStandard": "gnu++14"
        }
    ],
    "version": 4
}
```

- `launch.json`:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "g++.exe - Compilar y depurar el archivo activo",
            "type": "cppdbg",
            "request": "launch",
            "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": false,
            "MIMode": "gdb",
            "miDebuggerPath": "C:\\Program
↪  Files\\mingw-w64\\mingw64\\bin\\gdb.exe",
            "setupCommands": [
                {
                    "description": "Habilitar la impresin con sangra para gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ],
            "preLaunchTask": "C/C++: g++.exe build active file"
        }
    ]
}
```

- In order to establish the formatting style put the following in `settings.json`:

```
{
    "C_Cpp.clang_format_fallbackStyle": "{ BasedOnStyle: LLVM, UseTab: Never,
↪  IndentWidth: 4, TabWidth: 4, BreakBeforeBraces: Attach,
↪  AllowShortIfStatementsOnASingleLine: false, IndentCaseLabels: false,
↪  ColumnLimit: 0, AccessModifierOffset: -4 }",
    "emmet.showSuggestionsAsSnippets": true,
    "files.associations": {
        "*.rmd": "markdown",
        "iostream": "cpp"
    }
}
```

- `task.json`:

```
{
    "version": "2.0.0",
```

```
    "tasks": [
        {
            "label": "C/C++: g++.exe build active file",
            "type": "shell",
            "command": "C:\\Program Files\\mingw-w64\\mingw64\\bin\\g++.exe",
            "args": [
                "-g", "main.cpp", "-o", "a.exe" // , "&&", "main"
            ],
            "problemMatcher": [
                "$gcc"
            ],
            "presentation": {
                "echo": false,
                "reveal": "always",
                "focus": false,
                "panel": "shared",
                "showReuseMessage": true,
                "clear": false
            },
            "group": {
                "kind": "build",
                "isDefault": true
            }
        }
    ]
}
```

Capítulo 3

Curriculum Overview

3.1. Curriculum overview

Get started quickly programming in C++.

- Getting started.
- Structure of a C++ program.
- Variables and constants.
- Arrays and vectors.
- Strings in C++.
- Expressions, Statements and Operators.
- Statements and operators.
- Determining control flow.
- Functions.
- Pointers and references.
- OPP: Classes and objects.
- Operator overloading.
- Inheritance.
- Polymorphism.
- Smart pointers.
- The Standard Template Library (STL).
- I/O Stream.
- Exception Handling.

Capítulo 4

Getting Started

4.1. Writting our first program

- Create a project.
- Create a file and type the following code.
- This program is going to take in a number and then display "Wow that is my favorite number".

```cpp
#include <iostream>
int main() {
    int favorite_number; // stores what the user will enter.
    std::cout << "Enter your favorite number between 1 and 100"; // prints.
    std::cin >> favorite_number;
    std::cout << "Amazing!! That's my favorite number too!" << std::endl; // prints
    ↪ that line. endl adds "\n" and flushes the buffer.
}
```

4.2. Building our first program

- Building involves compiling and linking.
- In vscode you can run the compile task by pressing ctrl+b.
- Linking means grabbing all the dependencies the main function needs, making .o or object files and adding them to the executable or the .exe.
- Tipically modern compilers have the option to not produce the object files and go ahead and just produce a single executable. IDEs tipically also hide the object files if they are produced.
- By *cleaning* a project what we mean is the object files are deleted and an executable will be produced.

4.3. What are compiler errors?

- Programming languages have rules.
- Syntax errors: something wrong with the structure.

```cpp
std::cout << "Errors << std::endl; // the string is never terminated.
```

- Semantic errors: something wrong with the meaning:

```cpp
a + b; // to sum a and b when it doesn't make sense to add them, maybe they are not numbers for example.
```

4.3.1. Examples of errors

Not enclosing a string with the çharacters.

```cpp
int main() {
    std::cout << "Hello world << std::endl; // string is not terminated with the other ".
    return 0;
}
```

Typos in your program:

```cpp
int main() {
    std::cout << "Hello world" << std::endll; // endll doesn't exist, this is syntaxis errors.
    return 0;
}
```

Missing semi-colons:

```cpp
int main() {
    std::cout << "Hello world" << std::endl // missing semicolon.
    return 0;
}
```

Function doesn't return the type specified, in this case the function doesn't return an integer.

```cpp
int main() {
    std::cout << "Hello" << std::endl;
    return; // main needs to return an integer and it is returning a void.
}
```

Not returning the specified type.

```cpp
int main() {
    std::cout << "Hello World" << std::endl;
    return "Hello"; // "Hello" is not an integer. Error.
}
```

Missing Curly brace:

```cpp
int main() // opening curly brace missing.
    std::cout << "Hello World" << std::endl;
}
```

Semantic error (example: adding something when it doesn't make sense).

```cpp
int main() {
    std::cout << ("Hello world" / 125) << std::endl; // dividing a string by a number, this doesn't make sense.
    return 0;
}
```

4.4. What are compiler warnings?

- It is good practice to never ignore compiler warnings.
- The compiler will recognize a potential issue but is still able to produce object code from the source code, things such as uninitialized variables.
- It's only a warning because the compiler is still able to generate correct machine code.
- Example:

```
int miles_driven; // never initialized, this value could be anything.
std::cout << miles_driven << std::endl;
/* Warning: 'miles_driven' is used uninitialized in this function. */
```

- Another example is when you declare variables and never use them.

```
int miles_driven = 100;
std::cout << "Hello world" << std::endl;
/* Warning: unused variable 'miles_driven'. */
```

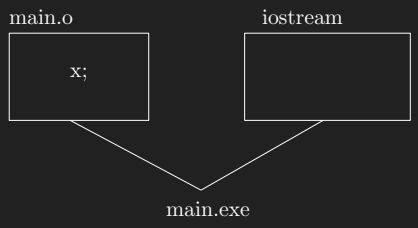
- As a rule you want to produce warning free source code.

4.5. Linked errors

- The linker is having trouble linking all the object files together to create an executable.
- Usually there is a library or object file that is missing.

4.5.1. Example

```
#include <iostream>
extern int x; // this means the variable is defined outside this file.
int main() {
    std::cout << "Hello world" << std::endl;
    std::cout << x;
    return 0;
}
/* This program will compile, but in runtime you will get a linker error. */
```



According to the linker x is undefined, thus an error is produced.

4.6. Runtime Errors

- Errors that occur when the program is executing.
- Some typical runtime errors include:
  - Divide by zero.
  - File not found.
  - Out of memory.
- Can cause your program to crash.
- Exception handling can help deal with runtime errors.

4.7. What are logic errors?

- Errors or bugs in your code that cause your program to run incorrectly.
- Logic errors are mistakes made by the programmer.

Suppose we have a program that determines if a person can vote in an election and you must be 18 years or older to vote.

```
if (age > 18) { // This means that age cannot be 18 thus 18 yearolds would not be able to vote. 18 is not included.
    std::cout << "Yes you can vote" << endl;
}
```

4.8. Section challenge solution

```
#include <iostream>
int main() {
    int favorite_number;
    std::cout << "Enter your favorite number between 1 and 100: ";
    std::cin >> favorite_number;
    std::cout << "Amazing!! Thats my favorite number too!" << std::endl;
    std::cout << "No really!!, " << favorite_number << " is my favorite number!" << std::endl;
}
/* Output:
Enter your favorite number between 1 and 100: 67
Amazing!! Thats my favorite number too!
No really!!, 67 is my favorite number!
*/
```

Capítulo 5

Structure of a C++ program

5.1. Overview of structure of a C++ program

- C++ has lots of keywords.
- Compared to other languages:
  - Java has about 50.
  - C has 32.
  - Python has 33.
  - C++ has 90.
- You can view these words in the following link: c++ reference.
- Keywords cannot be redefined nor used in a way they are not intentionally made for. This is why variables have naming conventions for example.
- Difference between keywords and identifiers, a keyword is something that is built in the programming language, a keyword has specific purpose and meaning that cannot be changed; identifiers are defined and user for the programmers purposes, a variable name is an example of an identifier, another example are function names, there are rules for identifiers, also conventions.
- C++ also has punctiation, and you must adhere to the punctuation rules, things such as , $\ll$, $\gg$, ;, etcetera, are examples of punctuation.
- When you assemble rules of punctuation, keywords, identifiers, rules, you end up with *syntax* this refers to rules and conventions that you must follow.

5.2. #include Preprocessor directive

- What is a preprocessor?
  - A preprocessor is a program that processes your source code before your compiler sees it. C++ preprocessor strips all the comments from the source code and then sends it to the compiler, it replaces each comment with a single space.
- Directives in C++ start with a # sign. Examples include:

```
#include <iostream>
#include "myfile.h"
#if
#elif
```

```
#else
#endif
#ifdef
#ifndef
#define
#undef
#line
#error
#pragma
```

- In simple terms the compiler replaces the include directive with the file that its refering to, then it recursively processes that file as well.
- By the time the compiler sees the source code it has been stripped of all comments and all preprocessor directives have been processed and removed.
- Preprocessor directives are routinely used to conditionally compile code, for example say I want to execute some piece of code only if the program is running in the windows operating system and execute some other if the program is being run by MacOS, preprocessor directives can check to see if you are on windows and run your code, if you're not on windows then the preprocesor directive will run the MacOS code.
- The C++ preprocessor does not understand C++. It simply processes the preprocessor directives and gets the code ready for the compiler, the compiler is the program that does understand C++.

5.3. Comments in C++

- Comments are programmer readable explanations in the source code; explanations, notes, annotations, or anything that adds meaning to what the program is doing.
- The comments never makes it to the compiler, the preprocessor strips them, the comments are just to enhance the readibility of your source code, it's intended for humans and the compiler never sees it.
- There are basically two ways of writing comments, single line comments and multiline comments.
- Single line comments start with `//`, the remaining characters in that line are considered a comment by the preprocessor.

```
int var = 100; // This is a comment.
```

- Multiline comments start with `/*` anything after these characters up until `*/` are considered a comment by the preprocessor. Everything between the `/* */` will be ignored.

```
/* This is a line.
This is another commented line.
This is yet another.
...
*/ int var = 100;
```

- The idea behind comments are to make self documenting code. Self documenting code is the practice of wrinting down what your code does and how it does it so that other people may understand it.

```
int var = 100; // I'm initializing a variable to 100 in order to use <where I will
↪  use it> for <state the purpose>.
```

- Keep in mind not every single line needs commenting, comments must be used only when they are needed, sometimes code can be very hard to read and that's were you want to use comments. You don't want to write a comment for every line of code saying what you do because what you are doing is easily and clearly dedusable, other code however can become very unreadable and complicated and that's were you might want to use comments so that other people know what you are doing. Don't comment the obvious.
- As a rule: don't comment the obvious, good commenting doesn't justify bad code, and if you made changes to code make sure you also change the comments to save time and confusion, keep comments in sync.

5.4. The main() function

- Every C++ program must have exactly one `main()` function.
- main() is the starting point of program execution.
- the `return 0;` statement in the `main()` function indicates successful program execution.
- When a C++ program executes the operating system calls the `main()` and the code between the curly braces executes. When execution hits the return statement, the program returns the integer 0 to the operating system, if the return value is 0 then the program terminated sucesfully, if the value returned is not 0 then the operating system can check the value returned and determine what when wrong.
- There are two posible ways of writing the main function. They work almost the same and the differences are suttle. All the program actions are contained in the main function.

```
// First way:
int main(/* No parameters. */) {
    // <code>
    return 0;
}
// Second way:
int main(int argc, char *argv[]){
    // <code>
    return 0;
}
```

- The first version expects no information to be passed in from the operating system in order to run. This is the most common version.
- The second version allows for parameters to be passed in to the main function when its called, so for example you can pass in an integer or a string that you can then use in your program when it's called from the command line. You would call your program like this in the command line: `program.exe n arg1 arg2 argn` the first argument or `int argc` must be the number of arguments that you will pass in, the next arguments are delimited by spaces and collected and stored inside your program in the `char *argv[]` array which you can use in your program respectively.
- It's important to see the distinction in order not to get confused when you are looking at code out on the internet.

5.5. Namespaces

- As C++ programs become more complex the combination of our own code, the C++ standard library, and other third party libraries, sooner or later C++ encounters two names repeated, and at that point C++ doesn't know which one to use. This is called a naming conflict and its described when company X named something the same as company Y.

- This is where namespaces come in, you can specify which library you are refering to by using the name of the library and the scope resolution operatior (`::`)
- C++ allows developers to use namespaces as container to group their code entities in to a namespace scope.
- An example is the `std::cout` statement, tecnically it is saying to the C++ compiler to search in "std" or the standard library the function "cout" and use that one.
- Namespaces are introduced to reduce the posibility of a naming conflict.
- However, it can get tedious to write the library name, then the scope resolution operator and finally the function name; thus you can use the `using namespace <insert library name>;` statement to specify that any function from there on will come from the library spefied, now C++ knows which namespace you are using.

```cpp
#include <iostream>
using namespace std;
int main() {
    // <code>.
    return 0;
}
```

  - C++ in the code above knows the moment we said `using namespace std;` that any function referenced from that point will be refering to the std library function.

- However there is still a problem, `using namespace std;` doesn't just state to use cout and cin, it brings a lot of other functions we might not know about, for this we can explicitly say we just want to use a certain function of a certain library.

```cpp
#include <iostream>
using std::cout;
using std::cin;
int main() {
    // <code>.
    return 0;
}
```

  - You can still use `cout` and `cin` without having to write `std::cout` and `std::cin` and we are not getting any other names from the standard library of which we won't need. In larger programs its best practice to declare the functions you will use manually and not the namespace.

5.6. Basic input and output

- `cout, cin, cerr, clog` are objects representing streams. They are defined in the iostream library.
- `cout`:
  - It is a standard output stream.
  - Defaults to the console.
- `cin`:
  - It is a standard input stream.
  - Defaults to the keyboard.

- `cerr`:
  - It is a standard error stream.
  - Defaults to the console with the error stream.
- `clog`:
  - It is a standard log stream.
  - Defaults to the console with the log stream.
- `<<` is the insertion operator:
  - Used on output streams.
- `>>` is the extraction operator:
  - Used on input streams.

5.6.1. `<<` with `cout`

- The insertion operator (`<<`) inserts data into the cout stream. It inserts the value of the operad(s) to its right, this operand can chain various variables or objects.

```
cout << data;
```

- Chaining multiple insertion in the same statement:

```
cout << "data 1 is " << data1 << endl;
cout << "data 1 is " << data1 << "\n";
```

- It is important to understand that the insertion operator does not automatically add line breaks to what is printed, that can be added by `endl` or by explicitly adding it using the newline escape character `"\n"`. The `endl` manipulator it will also flush the stream, this is important because if the stream is buffered the program might not print anything until its flushed.

5.6.2. `>>` with `cin`

- Used to extract data from the cin (which defaults to the keyboard) stream based on the data's type and then stores the information into the variable to the right of the extraction operator.

```
cin >> data;
```

- It can be chained.

```
cin >> data1 >> data2;
```

- Can fail if the entered data cannot be interpreted. For example if the variable being read is expecting an integer and what is read is a string of characters the cin will fail and nothing will be assigned to the variable. The cin function will ignore white space and tabs.
- The cin function will only be executed when the enter key is pressed.
- We can use the extraction and insertion operator to work with data from different streams, for example file streams.

Capítulo 6

Variables and constants

6.1. What is a variable?

- A variable allows us to use a name for a memory location in RAM in which our variable is stored.
- A variable is an abstraction for a memory location.
- Variables allow programmers to use meaninful names and not memory addresses.
- Variables have:
  - Type: their category (integer, real number, string, objects).
  - Value: the content (10, 3.14, "String").
- Variables must be declared before they are used.
- A variable value may change.
- Example:

```
1 age = 21; // Compiler error.
```

In the above code we never declared age so the compiler does not know how much memory to allocate for the variable, thus a compiler error is produced, age was never declared. This is called static typing because all the rules are enforced when the program is compiled rather than when the program is running.

```
1 int age;
2 age = 21;
```

The above code is correct.

6.2. Declaring and initializing variables

- The syntax is: `<variable_type> <variable_name>;`.
- Variable types can be anything, such as: `int`, `double`, `float`, `string` you can also declare user defined types (this is object oriented programming) such as `Account`, `Person` with the same syntax you would use with any other type.

6.2.1. Naming rules and conventions

- Naming variables:
  - Can contain letters, numbers and underscores.
  - Must begin with a letter to underscore.
    - Cannot begin with a number digit.
  - Cannot use C++ reserved keywords.
  - Cannot redeclare a name in the same scope.
    - Remember that C++ is case sensative.
  - Example:

Legal	Illegal
Age	int
age	$age
_age	2014_age
My_age	My age
your_age_in_2014	Age+1
INT	cout
Int	return

- The best thing to do is to be consistent with your naming conventions.
  - Stick to the convention you chose from the beggining.
  - Avoid beginning names with underscores.
- Use meaningful names:
  - Not too long and not too short.
- Never use variables before initializing them.
  - Using variables before initializing can cause undefined behaviour.
- Declare variables close to when you need them in your code.

6.2.2. Declaring and initializing

- There are many ways of initializing variables. And all reflect the way C++ has advanced as the years have passed.

```
int age; // uninitialized.
int age = 21; // C-Like initialization.
int age (21); // Constructor initialization.
int age {21}; // C++11 list initialization syntax.
```

- Using the `{}` list intialization will also check if the assigned values cause an overflow in the program.

6.2.3. Example

```
#include <iostream>
using namespace std;

int main() {
    int room_width {0};
    cout << "Enter the width of the room: ";
```

```
7       cin >> room_width;
8
9       cout << "Enter the lenght of the room: ";
10      int room_lenght {0};
11      cin >> room_lenght;
12
13      cout << "The area is " << room_width*room_lenght << endl;
14      return 0;
15  }
```

6.3. Global variables

- Variables declared outside of any function are called global variables and they can be accessed in any point of your program.
- Since they can be accessed by any part of the program this also means they can be changed by any part of the program which could mean the likelyhood for bugs is higher.
- Local variables are declared and used within a part of the code, as your code progresses out of scope these variables are automatically destoyed.
- Local variables have higher precedence than global variables, if you call a local and global variable the same and decide to use one of both the local is going to be prioritized and the global variable will no be used.

6.4. C++ Built-in Primitive Types

- These are sometimes called fundamental data types because they are implemented directly by the C++ language.
- They include:
  - Character type.
  - Integer type.
    - signed and unsigned.
  - Floating-point types.
  - Boolean types.
- Size and precision is often compiler dependent, this means you must be aware how much storage is allocated for each type. The C++ library `#include <climits>` conatins information about your specific compiler.

6.4.1. Type sizes

- A computer works by storing bits on memory, data types are stored in bits.
- The more bits the more values that can be represented.
- The more bits the more storage requiered.
- The computer however is not conserned with bits but with bytes, a byte is 8 bits stored continously in memory. To find out how many values you can store with n number of bits the formula is 2^n, below are some examples:

Size (in bits)	Representable values	
8	256	2^8
16	65,536	2^{16}
32	4,294,927,296	2^{32}
64	18,446,744,073,551,615	2^{64}

6.4.2. Character types

- Used to represent single characters: 'A', 'X', '@'.
- Wider types are used to represent wide character sets.

Type name	Size / Precision
char	Exactly one byte. At least 8 bits.
char16_t	At least 16 bits.
char32_t	At least 32 bits.
wchar_t	Can represent the largest available character set.

- Example:

```
char m {'j'};
cout << m << endl;
```

6.4.3. Integer data types

- Used to represent whole numbers.
- Signed and unsigned versions.

Type	Size / Precision	Type	Size / Precision
signed short int	At least 16 bits.	unsigned short int	At least 16 bits.
signed int	At least 16 bits.	unsigned int	At least 16 bits.
signed long int	At least 32 bits.	unsigned long int	At least 32 bits.
signed long long int	At least 64 bits.	unsigned long long int	At least 64 bits.

  - Ints can be overflowed, when an overflow happens you can have undefined behaviour.

- Example:

```
unsigned short int exam_score {55};
cout << exam_score << endl;
int countries_represented {65};
cout << countries_represented << endl;
long people_in_florida {2061100000};
cout << people_in_florida << endl;
long long people_on_earth {7'600'000'000};
cout << people_on_earth << endl;
long long distance_to_alpha_century {9'461'000'000'000};
cout << distance_to_alpha_century << endl;
```

6.4.4. Floating point type

- Used to represent non-integer numbers.
- Represented by mantissa and exponent (scientific notation).
- Precision is the number of digits in the mantissa.
- Precission and size are compiler dependent.

Type name	Size / Typical Precision	Typical Range
float	/7 decimal digits	$1{,}2 \times 10^{-38}$ to $3{,}4 \times 10^{38}$
double	No less than float / 15 decimal digits	$2{,}2 \times 10^{-308}$ to $1{,}8 \times 10^{308}$
float	No less than double / 19 decimal digits	$3{,}3 \times 10^{-4932}$ to $1{,}2 \times 10^{4932}$

  - Remember there is no such thing yet as storing an infinitely precise decimal such as π computers store approximations using scientific notation.

- Example:

```
float car_payment {401.23};
cout << car_payment << endl;
double pi {3.14159};
cout << pi << endl;
long double large_amount {2.7e120};
cout << large_amount << endl;
```

6.4.5. Boolean type

- Used to represnt true and false.
- Zero is false.
- Non-zero is true.

Type name	Size / Precision
bool	Usually 8 bits true or false (C++ Keywords)

- Example:

```
bool game_over {false};
cout << game_over << endl;
```

6.4.6. Buffer overflows

```
short val1 {30'000};
short val2 {1'000};
short product {val1 * val2}; // overflow, maximum is about 32,000.
```

6.5. What is the size of a variable?

- The `sizeof` operator determines the size in bytes of a type or variable.
- Examples:

```
sizeof(int);
sizeof(double);
sizeof(some_variable);
sizeof some_variable; // parenthesis are optional for variables.
```

- The sizeof operator gets its information from two C++ include files. <climits> <cfloat>, the climits and cfloat influde files contain size and precision information about your implemntation of C++. These libraries also provide information constants such as INT_MAX, INT_MIN, LONG_MAX, LONG_MIN, FLT_MIN, FLT_MAX, .

6.5.1. Examples

*Take into account that the values returned by the sizeof operator will be different depending on your machine.

```
#include <iostream>
#include <climits>
using namespace std;
int main() {
    cout << "sizeof: " << endl;
    cout << "char: " << sizeof(char) << endl;
    cout << "int: " << sizeof(int) << endl;
    cout << "unsigned int: " << sizeof(unsigned int) << endl;
    cout << "short: " << sizeof(short) << endl;
    cout << "long: " << sizeof(long) << endl;
    cout << "long long: " << sizeof(long long) << endl;
    cout << "float: " << sizeof(float) << endl;
    cout << "double: " << sizeof(double) << endl;
    cout << "long double: " << sizeof(long double) << endl;
    cout << "==================<climits>==================" << endl;
    cout << "climits minimum vals" << endl;
    cout << "char: " << CHAR_MIN << endl;
    cout << "int: " << INT_MIN << endl;
    cout << "short: " << SHRT_MIN << endl;
    cout << "long: " << LONG_MIN << endl;
    cout << "long long: " << LLONG_MIN << endl;
    cout << "climits maximum vals" << endl;
    cout << "char: " << CHAR_MAX << endl;
    cout << "int: " << INT_MAX << endl;
    cout << "short: " << SHRT_MAX << endl;
    cout << "long: " << LONG_MAX << endl;
    cout << "long long: " << LLONG_MAX << endl;
    cout << "==================sizeof variable names==================" << endl;
    int age {21};
    cout << "age is " << sizeof(age) << endl;
    double wage {22.24};
    cout << "wage is " << sizeof(wage) << endl;
    return 0;
}
/* OUTPUT:
sizeof:
char: 1
int: 4
unsigned int: 4
```

```
short: 2
long: 4
long long: 8
float: 4
double: 8
long double: 16
=================<climits>=================
climits minimum vals
char: -128
int: -2147483648
short: -32768
long: -2147483648
long long: -9223372036854775808
climits maximum vals
char: 127
int: 2147483647
short: 32767
long: 2147483647
long long: 9223372036854775807
=================sizeof variable names==================
age is 4
wage is 8
*/
```

6.6. What is a constant?

- Constants are very much like variables in C++.
- The follow the same naming convensions of variables.
- They occupy storage.
- And they are usually typed.
- However, their value cannot change once declared.

6.6.1. Literal constants

- The most obvious kind of constant.
- Integer literal constants:
  - `12` an integer.
  - `12U` an unsigned integer.
  - `12L` a long integer.
  - `12LL` a long long integer.
- Floating-point literal constants:
  - `12.1` a double.
  - `12.1F` a float.
  - `12.1L` a long double.
- Character literal constants (escape codes).

- `\n` newline.
- `\r` return.
- `\t` tab.
- `\b` backspace.
- `\'` single quote.
- `\"` double quote.
- `\\` backslash.

6.6.2. Defined constants

- Constants that are declared using the `const` keyword.

```
const double pi {3.1415926};
const int months_in_year {12};
pi = 2.5; // compiler error.
```

Defined constants

- Constants defined as preprocessor directive.

```
#define pi 3.1415926
```

- Notice you don't declare a type, this is sort of a blind find and replace done before compilation of the program, whenever the program encounters `pi` it will replace it with the value of the define constant, since the preprocessor does not know C++ it can't type check and this can make it difficult to find errors. The best is to never define constants and to always use the `const` keyword in your code.

Capítulo 7

Arrays and vectors

7.1. Arrays and vectors

- Arrays and vectors are compound data types. Which means they are data types made out of other data types.

7.2. What is an array?

- An array is a compound data type or a data structure.
  - Which means they are data types composed of other data types. They allow us to have collections of elements.
- All elements are of the same type.
- Each element can be accessed directly.
- Why do we need arrays?
  - Suppose we would need to store the scores of a test.
  - We could declare n variables for n students:

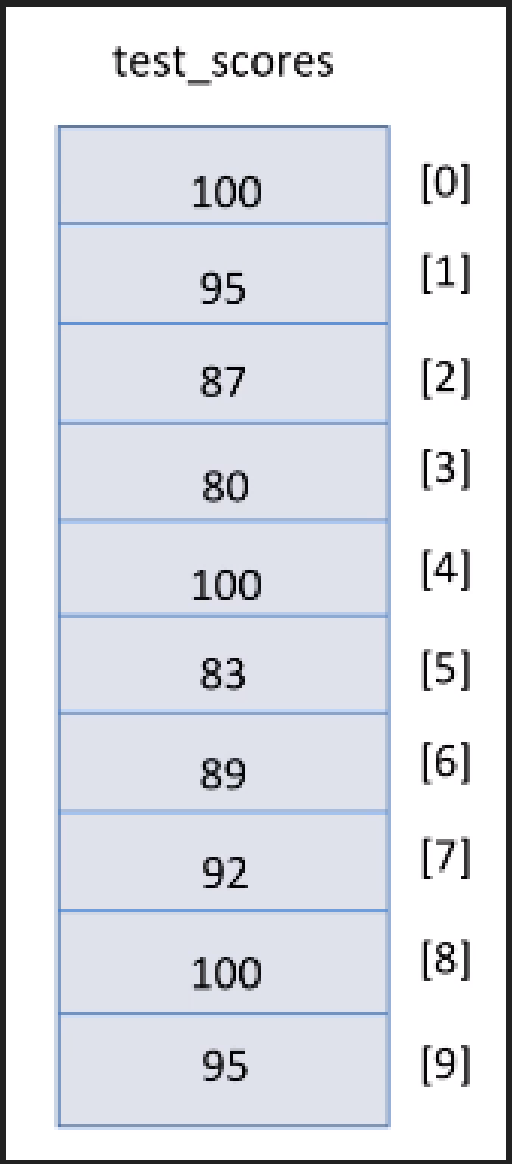
```
int test_score_1 {0};
int test_score_2 {0};
int test_score_3 {0};
int test_score_4 {0};
// ...
int test_score_100 {0};
```

  - This becomes tedious and error prone, now you would need to keep track of 100 variables with their own name. And even worse if I have 1,000,000 test scores this would get out of hand very easily.
  - In this case it's better to use an array.

7.2.1. Characteristics

- Fixed size.
- Elements are all the same type.
- Stored contigously in memory.

- Individual elements can be accessed by their position or index.
- First element is at index 0.
- Last element is at index $size - 1$.
- No checking to see if you are out of bounds. It is the responsability of the programmer to check if the program is writing data out of bounds, this can cause undefined behaviour.
- Always initialize arrays.
- Very efficient.
- Iteration (looping) is often used to process the elements stored.



7.3. Declaring and initializing arrays

7.3.1. Declaring arrays

- `element_type array_name [constant number of elements];`
    - The constant number of elements can be an expression that evaluates to a constant, a constant, a number or anything that results in a number before compiling.

```
int test_scores[5];
int high_score_per_level[10];
const double days_in_year {365};
double hi_temperature[days_in_year];
```

7.3.2. Initializing arrays

- `element_type array_name [number of elements] {init elements};`

```
int test_scores[5] {100,95,99,87,88};
int high_score_per_level[10] {3,5}; // init to 3,5 and remaining to 0.
const double days_in_year {365};
double hi_temperatures[days_in_year] {0}; // init all to zero.
double hi_temperature[days_in_year] {1,2,3,4,5}; // size automatically calculated.
```

7.4. Accessing and modifying array elements

- Accessing array elements: `array_name [element_index];`.

```
#include <iostream>
using namespace std;
int main() {
    int test_scores[5] {100,95,99,87,88};
    cout << "index 0: " << test_scores[0] << endl;
    cout << "index 1: " << test_scores[1] << endl;
    cout << "index 2: " << test_scores[2] << endl;
    cout << "index 3: " << test_scores[3] << endl;
    cout << "index 4: " << test_scores[4] << endl;
    return 0;
}
/* OUTPUT:
index 0: 100
index 1: 95
index 2: 99
index 3: 87
index 4: 88
*/
```

- Changing contents of array elements:

```
#include <iostream>
using namespace std;
int main() {
    int test_scores[5] {0};
```

```
    cin >> test_scores[0];
    cin >> test_scores[1];
    cin >> test_scores[2];
    cin >> test_scores[3];
    cin >> test_scores[4];
    test_scores[0] = 90;
    return 0;
}
```

7.4.1. How do arrays work?

- The nane of the array represents the location of the first element in the array (index 0).
- The [index] represents the offset from the beggining of the array.
- C++ simply performs a calculation to find the correct element.
- There is no bounds checking.

7.4.2. Examples

```
#include <iostream>
using namespace std;
int main() {
    char vowels[] {'a','e','i','o','u'};
    cout << vowels[0] << endl;
    cout << vowels[4] << endl;
    cin >> vowels[5]; // out of bounds.
    return 0;
}
/* OUTPUT:
    a
    u
    ... program crashes.
*/
```

```
#include <iostream>
using namespace std;
int main() {
    double hi_temps[] {90.1, 89.8, 77.5, 81.6};
    cout << "\nThe fist high temperature is: " << hi_temps[0] << endl;
    hi_temps[0] = 100.7; // sets the first element of hi_temps to 100.7.
    cout << "\nThe fist is now: " << hi_temps[0] << endl;
    return 0;
}
/* OUTPUT:

The fist high temperature is: 90.1

The fist is now: 100.7

*/
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int test_scores[5] {100};
    cout << "\nFirst score at index 0: " << test_scores[0] << endl;
    cout << "\nSecond score at index 1: " << test_scores[1] << endl;
    cout << "\nThird score at index 2: " << test_scores[2] << endl;
    cout << "\nFourth score at index 3: " << test_scores[3] << endl;
    cout << "\nFifth score at index 4: " << test_scores[4] << endl;
    cin >> test_scores[0];
    cin >> test_scores[1];
    cin >> test_scores[2];
    cin >> test_scores[3];
    cin >> test_scores[4];
    cout << "\nFirst score at index 0: " << test_scores[0] << endl;
    cout << "\nSecond score at index 1: " << test_scores[1] << endl;
    cout << "\nThird score at index 2: " << test_scores[2] << endl;
    cout << "\nFourth score at index 3: " << test_scores[3] << endl;
    cout << "\nFifth score at index 4: " << test_scores[4] << endl;
    return 0;
}
/* OUTPUT:
First score at index 0: 100

Second score at index 1: 0

Third score at index 2: 0

Fourth score at index 3: 0

Fifth score at index 4:

0 1 2 3 4 5

First score at index 0: 1

Second score at index 1: 2

Third score at index 2: 3

Fourth score at index 3: 4

Fifth score at index 4: 5
*/
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int test_scores[5] {100};
    cout << "Name of array: " << test_scores << endl;
    return 0;
}
/* OUTPUT:
```

```
Name of array: 0x61fe00
*/
```

7.5. Multidimentional arrays

- Declaring multidimentional arrays: `element_type array_name [dimension 1 size][dimension 2 size]...;`

```
int movie_rating [3][4];
```

- Some compilers do place limits on the number of dimensions you have depending on what machine they are running and the compiler being used.
- Multi-dimentional arrays are a spreadsheet concept, think of the first dimension as the rows and the second as the columns.

```
int spread_sheet[][] {
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
```

7.6. Declaring and initializing vectors

- Suppose we want to store test scores for my school.
- I have no way of knowing how many students will register next year.
- Options:
  - Pick a size that you are not likely to exceed and use static arrays.
  - Use a dynamic array sich as a vector.

7.6.1. Vectors

- A C++ vector is a container in the C++ standard template library.
- An array that can grom and shrink in size at excecution time.
- Provides similar semantics and syntax as arrays.
- Very efficient.
- Can provide bounds checking.
- Can use lots of cool functions like sort, reverse, find and more.
- When we create a C++ vector we are creating a C++ object and this means it can perform operations for us.

7.6.2. Declaring vectors

- We must include the vector library and the standard library, both allow us to create C++ vectors.
- Syntax: `vector<datatype> vector_name;`.

```
#include <vector>
using namespace std;
vector<char> vowels;
vector<int> test_scores;
```

7.6.3. Initializing vectors

- For vectors, since they are objects, we can provide initialization with the constructor intialization style:

```
vector<char> vowels (5);
vector<int> test_scores (10);
```

  - The above code creates a vector of size 5 if chars, then a vector of size 10 of ints, also all the elements of the array are intialized automatically to zero, you no longer need to do that explicitly.

- We can also use the curly brace initialization style to specify the value of the elements we want:

```
vector<char> vowels {'a','e','i','o','u'};
// Initializes vowels to the specified values in a vector of size 5.
vector<int> test_scores {100,98,89,85,93};
// Initializes test_scores to the 5 specified values.
vector<double> hi_temperatures (365, 80.0);
// The first paramenter is the initial size of the vector which os 365, the second
↪  is to which value you want to initialize all the elements in the vector.
```

7.6.4. Vector caracteristics

- Dynamic size.
- Elements are all the same type.
- Stored contiguosly in memory.
- Individual elements can be accessed by their position or index.
- First element is at index 0.
- Last element is at index $size - 1$.
- If you use the subscript operator ([index]) then vectors will provide no bounds checking, however the vector object allows you to get information at an index using methods that do provide bounds checking.
- Provides many useful functions that do bounds checking.
- Elements are automatically initialized to zero unless you specify otherwise.
- Very efficient.
- Iteration (looping) is ften used to process the elements.

7.7. Accessing and modifying vector elements

- The first way to access vector elements is to use array syntax (with the subscript operator): `vector_name[element_index` with this way no bounds checking will be done.

```
vector<int> test_scores {100,95,99,87,88};
cout << test_scores[0] << endl;
cout << test_scores[1] << endl;
cout << test_scores[2] << endl;
cout << test_scores[3] << endl;
cout << test_scores[4] << endl;
```

- We can also access vector elements with the `.at` method, the syntax is: `vector_name.at(element_index);`

```
vector<int> test_scores {100,95,99,87,88};
cout << test_scores.at(0) << endl;
cout << test_scores.at(1) << endl;
cout << test_scores.at(2) << endl;
cout << test_scores.at(3) << endl;
cout << test_scores.at(4) << endl;
```

7.7.1. Vectors changing in size dynamically

- The vector has a method called `push_back` that adds a new element (of the same type) to the back of the vector.
- This vector method will automatically allocate the required space.

7.7.2. Out of bounds errors

- What if you are out of bounds?
- Arrays never do bounds checking.
- Many vector methods provide bounds checking.
- An exception and error message is generated.

Capítulo 8

Statements and operators

8.1. Expressions and statements

- An expression is:
  - The most basic building block of a program.
  - "A sequence of operatiors and operands that specifies a computation." C++ STD.
  - Computes a value from a number of operands.
  - There is much, much more to expressions Not necessary at this level.
- Examples of expressions:

```
34 // literal.
favorite_number // variable.
1.5 + 2.8 // addition.
2 * 5 // multiplication.
a > b // relational.
a = b // assignment.
```

- Difference between a statement and expression:
  - A statement is:
    - A complete line of code that performs some action.
    - Usually terminated with a semi-colon.
    - Usually contain expressions.
    - C++ has many types of statements:
      - Expression, null, compound, selection, iteration, declaration, jump, try blocks, and others.
    - Expressions are used to make up the statement.

```
// Example of statements:
int x; // declaration.
favorite_number = 12; // assignment.
1.5 + 2.8; // expression.
x = 2 * 5; // assignment.
if ( a > b ) cout << "a is greater than b"; // if statement.
```

  - A null statement is a statement that doesn't perform any computation, it would be the equivalent of the following example:

```cpp
int a;
for (int i = 0; i < 10; i ++ ) {
    ; // null statement.
}
```

8.2. Using operators

- C++ has a rich set of operators, most of them are binary operators which means they operate on two operands, for example the multiplication operator operates on two operands (numbers). However C++'s operators aren't just binary.
- C++ has a rich set of operators.
  - Unary (operates on one operand), binary (operates on two operands), ternary (operates on three operands).
- Common operators can be grouped as follows:
  - Assignment: used for modifying the value of some object by assigning a new value to it.
  - Arithmetic: used to perform mathematical operations on operands.
  - Increment/Decrement: these operators work as an assignment and arithmetic, they increment or decrement the operand by one.
  - Relational/Comparison: allow you to compare the value of objects, examples include: $=, \neq, \geq, \leq$, etcetera.
  - Logical: used to test for logical or boolean conditions, for example: if you want to execute certain part of your code only when the temperature is less than freezing. These include the logical not, and, or operators.
  - Member access: used to allow access to a specific member. An example is the array subscript operator ([]). others that work with objects and pointers which will be introduced later.
  - Other: other operators.

8.3. The assignment operator

- Nearly all programming languages have the ability to change the value of a variable.
- In C++ we can change the value stored in a variable using the assignment operator (=).

```cpp
lhs = rhs;
```

- This does not represent equality, this represents that the value of `rhs` will be the value of `lhs`.
- We are not asserting that the left-hand side is equal to the right-hand side, nor are we comparing the left-hand side to the right-hand side. We are evaluating the value of the expression on the right-hand side and storing the value to the left-hand sign.
- `rhs` is an expression that is evaluated to a value.
- The value of the `rhs` is stored to the `lhs`.
- The value of the `rsh` must be type compatible with the `lhs`, meaning they must be of the same type.
- C++ is statcally typed which means that the compiler will be checking if it makes sense to asssign the right-hand side to the left-hand side, if it doesn't make sense you'll get a compiler error.

- In order to store the right-hand side to the left-hand side, the left-hand side must be assignable, it can't be a literal, it can't be a constant.
- An assignment expression is evaluated to what was just assigned, this makes it posible to chain more than one variable in a single assignment, such as:

```
a = b = c = d = e = 10;
```

- It is important to understand that assignment is not initialization, initialization happens only when the variable is declared and the variable gets that value for the very first time, assignment is when you change a value that already exists in the variable after initializing it.

8.3.1. Example

- Example:
  - In C++ there is a concept in assignment know as r-value and l-value, whenever we say the r-value we are denoting the content of some variable or an expression that evaluates to a value, the l-value is the a location, the statement: `lhs=rhs` means "store whatever value is in `rhs` to location `lhs`". The `rhs` could be very complex, the program will evaluate and compute that value and store it in location `lhs`.
  - In C++ all this checking is done at compile time, thus when something compiles without errors or warnings you are guaranteed to have done it correctly. This is also because C++ is statically typed.

```
#include <iostream>
using namespace std;
int main() {
    int num1 {10}; // initialization.
    int num2 {20}; // initialization.
    num1 = 100; // assignment. lhs = rhs;
    cout << "num1 is " << num1 << endl;
    cout << "num2 is " << num2 << endl;
    cout << endl;
    return 0;
}
/* OUTPUT:
num1 is 100
num2 is 20

*/
```

- Chain assignment:
  - In assignment operators they are done from right to left, the left-hand side of the expression is done last, while the further right part of the expression is done first.
  - So something like this:

$$num1 = num2 = num3 = 1'000;$$
$$num1 = num2 = \underbrace{num3 = 1'000;}_{\text{Evaluates to 1'000}}$$
$$num1 = \underbrace{num2 = num3 = 1'000;}_{\text{Evaluates to 1'000}}$$

- Finally the last part of the assignment is assigning the value of num2 to num1.

```cpp
#include <iostream>
using namespace std;
int main() {
    int num1 {10}; // initialization.
    int num2 {20}; // initialization.
    num1 = num2 = 1'000; // chained assignment.
    cout << "num1 is " << num1 << endl;
    cout << "num2 is " << num2 << endl;
    cout << endl;
    return 0;
}
/* OUTPUT:
num1 is 1000
num2 is 1000

*/
```

- The compiler will constantly be checking if it makes sense to assign something to a variable. For example the following code produces a compiler error because an `int` variable can only hold integers not strings.

```cpp
#include <iostream>
using namespace std;
int main() {
    int num1 {10}; // initialization.
    int num2 {20}; // initialization.
    num1 = "String"; // assignment to a string of an integer variable is illegal.
    cout << "num1 is " << num1 << endl;
    cout << "num2 is " << num2 << endl;
    cout << endl;
    return 0;
}
/* OUTPUT: Compiler error.
./Code/main.cpp:6:12: error: invalid conversion from 'const char*' to 'int'
 ↪  [-fpermissive]
    num1 = "String"; // assignment.
*/
```

- The following code will produce an error, though we are assigning an int to the variable which makes sense, we are declaring it as a constant, thus we cannot change the value of that constant after initialization.

```cpp
#include <iostream>
using namespace std;
int main() {
    int const num1 {10}; // initialization.
    int num2 {20}; // initialization.
    num1 = 100; // assignment to a constant is illegal.
    cout << "num1 is " << num1 << endl;
    cout << "num2 is " << num2 << endl;
    cout << endl;
    return 0;
```

```
}
/* OUTPUT: Compiler error.
./Code/main.cpp:6:12: error: assignment of read-only variable 'num1'
    num1 = 100; // assignment to a constant is illegal.
*/
```

- The following code will produce a compiler error because we are assigning a variable to a literal:

```
#include <iostream>
using namespace std;
int main() {
    int num1 {10};
    100 = num1; // assignment
    return 0;
}
/* OUTPUT: Compiler error.
./Code/main.cpp:5:11: error: lvalue required as left operand of assignment
    100 = num1; // 100 is a literal and does not have a location in memory.
*/
```

8.4. Arithmetic operators

- The arithmetic operators are:
  - Addition: +
  - Subtraction: −
  - Multiplication: ∗
  - Division: /
  - Modulo or remainder (works only with integers): %
- These operators can be overloaded, what 'overloaded' means is that they work with different types, for example you can use the addition operator to add floats, integers, doubles, etcetera.

8.4.1. Examples

- Adding variables and assigning the result to another.

```
#include <iostream>
using namespace std;
int main() {
    int num1 {100};
    int num2 {200};
    int result {num1 + num2};
    cout << num1 << " + " << num2 << " = " << result << endl;
    return 0;
}
/* OUTPUT:
100 + 200 = 300
*/
```

- Arithmetic:

- It is important to notice that multiplication is not like algebra where you can omit adding the multiplication symbol, in C++ we need to add the multiplication symbol explicitly. There are no such thing as: (3)(4) rather write it as: `(3)*(4);`.
- Another thing to watch out for is when dividing integers you don't get a decimal part, you will only get the whole number part, dividing $100/200 = 0{,}5$ however we just store the whole part, which is to say as far as C++ is concerned when you divide the integers 100/200 the result is floor division and you are left with the result being 0.
- The modulus operator is the remainder of division, for example if we divide 10 by 3 we get that we can divide 10 by 3 a total of 3 times, however we are left with a remainder, that remainder is 1 ($3 \times 3 + 1 = 10$).
- There is operator precedence in C++, it goes in the order of PEMDAS, P(Parenthesis), E(Exponents), M(multiplication), D(division), A(addition), and S(subtraction). So for example in the expression $(8 * 10 + 1) - 10$ the parenthesis are evaluated first, inside the parenthesis the multiplication of 8 and 10 are evaluated and the one is added resulting in 81, then the subtraction of 10 is done and the result is 71.

```cpp
#include <iostream>
using namespace std;
int main() {
    int num1 {100};
    int num2 {200};
    int result {0};
    result = num1 + num2;
    cout << num1 << " + " << num2 << " = " << result << endl;
    result = num1 - num2;
    cout << num1 << " - " << num2 << " = " << result << endl;
    result = num1 * num2;
    cout << num1 << " * " << num2 << " = " << result << endl;
    result = num1 / num2; // floor division
    cout << num1 << " / " << num2 << " = " << result << endl;
    result = num1 % num2;
    cout << num1 << " % " << num2 << " = " << result << endl;
    return 0;
}
/* OUTPUT:
100 + 200 = 300
100 - 200 = -100
100 * 200 = 20000
100 / 200 = 0
100 % 200 = 100

*/
```

8.5. Increment and decrement operators

- The increment and decrement operators are unary operators, `++`, `--` the `++`.
- The increment and decrement operators are simply just saying: `++` increment its operand by one, `--` decrement the operand by one.
- This operator can also be overloaded, which is to say that they will increment or decrement different types, such as incrementing or decrementing floats, doubles, integers, and even pointers.

- There are two variants to this operator, postfix and suffix.
  - Postfix notation: `++num` this means to increment num by one before using it.
  - Prefix notation: `num++` this means to increment num by one after using it.
- Don't overuse this operator, **never use it twice for the same variable in the same statement** because that will cause undefined behavior. Things such as the following are not allowed:

```
num = num1++ + ++num1; // or
cout << i++ << ++i << i << endl; // will cause undefined bahaviour.
```

- The post fix and prefix notation work exactly the same if they are alone on one line, such as: `num++;`. However, if they are accompanied such as: `arr[num++] = 10;` will work differently.

8.5.1. Example

- Incrementing:

```
#include <iostream>
using namespace std;
int main() {
    int counter {10};
    int result {0};
    cout << "Counter: " << counter << endl;
    counter = counter + 1; // 11
    cout << "Counter: " << counter << endl;
    counter++;
    cout << "Counter: " << counter << endl;
    ++counter;
    cout << "Counter: " << counter << endl;
    return 0;
}
/* OUTPUT:
Counter: 10
Counter: 11
Counter: 12
Counter: 13

*/
```

- Pre-increment example:

```
#include <iostream>
using namespace std;
int main() {
    int counter {10};
    int result {0};
    result = ++counter; // Counter will be incremented before its used.
    cout << "Result: " << result << endl;
    cout << "Counter: " << counter << endl;
    return 0;
}
/* OUTPUT:
Result: 11
```

```
Counter: 11
*/
```

- Post-increment example:

```cpp
#include <iostream>
using namespace std;
int main() {
    int counter {10};
    int result {0};
    result = counter++; // Counter will be incremented after its used.
    cout << "Result: " << result << endl;
    cout << "Counter: " << counter << endl;
    return 0;
}
/* OUTPUT:
Result: 10
Counter: 11
*/
```

- Saying `i = ++num + 10;` is the same as saying:

$$Consideringnum = 10.$$

$$i = \underbrace{++num}_{num=num+1 \rightarrow num=11} + 10;$$

$$i = \underbrace{num}_{\text{has been incremented by one}} + 10;$$

$$i = 11 + 10 = 21;$$

```cpp
#include <iostream>
using namespace std;
int main() {
    int num {10};
    int i {0};
    i = ++num + 10;
    cout << "i: " << i << endl;
    return 0;
}
/* OUTPUT:
i: 21
*/
```

- The same example with the post-increment:

```cpp
#include <iostream>
using namespace std;
int main() {
    int num {10};
    int i {0};
    i = num++ + 10; // for this addition it will use the value of num as is and
    ↪ then increment so\alpha 10 + 10 = 20.
    cout << "i: " << i << endl;
```

```
    return 0;
}
/* OUTPUT:
i: 20
*/
```

8.6. Mixed expressions and conversions

- C++ operations occur on the same type operands. For example $a + b$ where a is an integer and b is a double.
- If operands are of different types, C++ will convert one. In many cases this happens automatically.
- This is important since it could affect calculation results.
- C++ will attempt to automatically convert types (coercion). If automatic conversion or coercion is not possible a compiler error will occur.

8.6.1. Conversions

- In order to understand how these automatic conversions happen we need to understand higher vs lower types.
  - Higher type is a type that can hold higher or more values.
  - Lower type is a type that can hold lower or less values than the higher type.
- Higher vs Lower types are based on the size of the values the type can hold.
  - `long double`, `double`, `float`, `unsigned long`, `long`, `unsigned int`, `int`, `short int`, `char` are always converted to an `int`.
- Coercion always happens by converting the lower type to the higher type, because the lower type will always fit in to the higher type whereas the higher type will almost never fit in to the lower type.
- Type coercion: happens when we convert a lower type operand to a higher type operand in order to operate them. For example, adding an integer and a double, automatic conversion will occur, first the integer will get converted to a double, then added.
- Promotion: conversion to a higher type:
  - Used in mathematical expressions.
- Demotion: conversion to a lower type:
  - Used with assignment to lower types.
  - For example when performing integer division the result is operated on a double and then the decimal part is truncated until we are left with just the whole number part.

8.6.2. Example type coercion

- Lower operand to higher operand, the lower is promoted to a higher.

```
2 * 2.5; // 2 is promoted to 2.0, integer -> double.
```

- lower = higher; the higher is demoted to a lower: (this risks potentially loosing information)

```
int num {0};
num = 100.2; // float demoted to a int.
```

8.6.3. Examples explicit type casting

- We can explicitly cast or coerce to a certain type.

```cpp
#include <iostream>
using namespace std;
int main() {
    int total_amount {100};
    int total_number {8};
    double average {0.0};
    average = total_amount / total_number;
    cout << "Without explicit coercion: " << average << endl; // Here we are doing integer division.
    average = static_cast<double>(total_amount) / total_number; // now one of the operands is a double so the other will be converted to the higher and the result will be a double.
    cout << "With explicit coersion: " << average << endl;
    return 0;
}
/* OUTPUT:
Without explicit coercion: 12
With explicit coersion: 12.5
*/
```

- There are two major ways of casting, the first is the syntax we already have seen: `static_cast<double>(var1);` the second is the c-style cast which is `(double)var1;`. It is better to use the C++ style cast since it is more robust as it also checks for congruence and if it makes sense to cast the variable, rather than the C-style cast just does it and it doesn't check for congruence nor if it makes sense.

```cpp
#include <iostream>
using namespace std;
int main() {
    int total {};
    int num1 {}, num2 {}, num3 {};
    const int count {3};
    cout << "Enter 3 integers separated by spaces: ";
    cin >> num1 >> num2 >> num3;
    cout << endl;
    total = num1 + num2 + num3;
    double average {0.0};
    average = static_cast<double>(total) / count; // C++ style cast.
    // average = (double)(total) / count; // older C-style cast
    cout << average << endl;
    return 0;
}
/* OUTPUT:
Enter 3 integers separated by spaces: 15 13 12

13.3333

*/
```

8.7. Testing for equality

- These operators compare the values of two expressions and evaluate to a boolean.
- These operators are the `==` which is the equality operator and the `!=` which is the not equals operator.

```
expr1 == expr2; // evaluates to true if both values are the same, else false.
expr1 != expr2; // evaluates to false if both values are the same, else true.
100 == 200; // evaluates to false.
num1 != num2; // evaluates to true if they are different, else false.
```

- Remember to use two equals signs not just one.
- Another example:

```
bool result {false};
result = (100 == 50+50); // store the boolean in result.
```

- In C++ there is a standard library string manipulator called `std::boolalpha` once you use it all boolean output printed to console will result in the words "true" or "false" being printed instead of 0 or 1. `std::noboolalpha` will deactivate this feature and allows you to print the default of 0 and 1.

```
#include <iostream>
using namespace std;
int main() {
    int num1 {10}, num2 {10};
    cout << "num1: " << num1 << ", num2:" << num2 << endl;
    cout << "(num1 == num2) -> " << (num1 == num2) << endl; // 0 or 1
    cout << "(num1 != num2) -> " << (num1 != num2) << endl;
    cout << std::boolalpha;
    cout << "(num1 == num2) -> " << (num1 == num2) << endl; // true or false
    cout << "(num1 != num2) -> " << (num1 != num2) << endl;
    cout << std::noboolalpha;
    return 0;
}
/* OUTPUT:
num1: 10, num2:10
(num1 == num2) -> 1
(num1 != num2) -> 0
(num1 == num2) -> true
(num1 != num2) -> false
*/
```

8.7.1. Example

- Example with integers: keep in mind you can do this with any primitive type and with the correct overloading with any user defined type.

```
#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;
    int num1 {0}, num2 {0};
    bool equal_result {false};
```

```
    bool not_equal_result;
    cout << "Enter 2 integers: " << endl;
    cin >> num1 >> num2;
    equal_result = (num1 == num2);
    not_equal_result = (num1 != num2);
    cout << "Comparison result (equals): " << equal_result << endl;
    cout << "Comparison result (not equals): " << not_equal_result << endl;
    return 0;
}
/* OUTPUT:
Enter 2 integers:
10 10
Comparison result (equals):true
Comparison result (not equals): false
*/
```

- Example with characters:

```
#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;
    bool equal_result {false}, not_equal_result {false};
    char char1{}, char2{};
    cout << "Enter two characters separated by a space: ";
    cin >> char1 >> char2;
    equal_result = (char1 == char2);
    not_equal_result = (char1 != char2);
    cout << "Comparision result (equals) : " << equal_result << endl;
    cout << "Comparision result (not equals) : " << not_equal_result << endl;
    return 0;
}
/* OUTPUT:
Enter two characters separated by a space: a a
Comparision result (equals) : true
Comparision result (not equals) : false
*/
```

- Example with doubles:
  - The following evaluates to true because of the way computers store information, as far as it is concerned 11.9999999999999999 is 12.

```
#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;
    bool equal_result {false}, not_equal_result {false};
    double double1{}, double2{};
    cout << "Enter two doubles separated by a space: ";
    cin >> double1 >> double2;
    equal_result = (double1 == double2);
```

```
    not_equal_result = (double1 != double2);
    cout << "Comparision result (equals) : " << equal_result << endl;
    cout << "Comparision result (not equals) : " << not_equal_result << endl;
    return 0;
}
/* OUTPUT:
Enter two doubles separated by a space: 12 11.9999999999999999
Comparision result (equals) : true
Comparision result (not equals) : false
*/
```

- Example with integer and a double:
    - In the below example we see another example of coercion, the 10 will get promoted to 10.0 then compared with the other with is 10.0, the expression evaluates to true.

```
#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;
    bool equal_result {false}, not_equal_result {false};
    int num1 {};
    double double1 {};
    cout << "Enter an integer and a double separated by a space: ";
    cin >> num1 >> double1;
    equal_result = (num1 == double1);
    not_equal_result = (num1 != double1);
    cout << "Comparision result (equals) : " << equal_result << endl;
    cout << "Comparision result (not equals) : " << not_equal_result << endl;
    return 0;
}
/* OUTPUT:
Enter an integer and a double separated by a space: 10 10.0
Comparision result (equals) : true
Comparision result (not equals) : false
*/
```

8.8. Relational operators

- In addition to the equality operators C++ also provides another set of operators to compare objects.

Operator	Meaning
>	greater than.
>=	greater than or equal to.
<	less than.
<=	less than or equal to.
<=>	three-way comparison (C++20).

- The three-way comparison operator compares two expressions and evaluates to 0 if they are equal, less than 0 if the left-hand side is greater than the right-hand side, and greater than 0 if the right-hand side is greater than the left-hand side.

8.8.1. Example

- Example:

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;
    int num1 {}, num2 {};
    cout << "Enter 2 integers separated by a space: ";
    cin >> num1 >> num2;
    cout << num1 << " > "   << num2  << " : " << (num1 > num2) << endl;
    cout << num1 << " >= " << num2  << " : " << (num1 >= num2) << endl;
    cout << num1 << " < "  << num2  << " : " << (num1 < num2) << endl;
    cout << num1 << " <= " << num2  << " : " << (num1 <= num2) << endl;
    cout << endl;
    return 0;
}
/* OUTPUT:
Enter 2 integers separated by a space: 10 20
10 > 20 : false
10 >= 20 : false
10 < 20 : true
10 <= 20 : true

*/
```

- Checking user obedience:

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << boolalpha;
    int num1 {}, num2 {};
    const int lower {10};
    const int upper {20};
    cout << "Enter an integer that is greater than " << lower << " : " ;
    cin >> num1;
    cout << num1 << " > " << lower << " is " << (num1 > lower) << endl;
    cout << "Enter an integer that is less than or equal to " << upper << " : " ;
    cin >> num1;
    cout << num1 << " <= " << upper << " is " << (num1 <=  upper) << endl;
    cout  << endl;
    return 0;
}
/* OUTPUT:
Enter an integer that is greater than 10 : 12
12 > 10 is true
Enter an integer that is less than or equal to 20 : 16
16 <= 20 is true

*/
```

8.9. Logical operators

- C++ has three logical operators:

Operator	Meaning
`!`	negation, logical not.
`&&`	logical and.
`||`	logical or.

- These operators work on boolean expressions and evaluate to boolean values themselves.
- There are two ways to write the logical operators, you can write the keywords: `not`, `and`, `or` or the symbols `!`, `&&`, `||`.
- The `!` operator is a unary operator and it simply negates the value.

expression a	not a / $!a$
true	false
false	true

- The `&&` operator is a binary operator and it performs a logical and.

Expression a	Expression b	a and b / a `&&` b
true	true	true
true	false	false
false	true	false
false	false	false

- The `||` operator is a binary operator and it performs the logical or.

Expression a	Expression b	a or b / a `||` b
true	true	true
true	false	true
false	true	true
false	false	false

8.9.1. Precedence

- Logical operators have precedence.
- The `!` or logical *not* has higher precedence than the logical *and*.
- The logical *and* has higher precedence than the logical *or*.
- The logical *not* is a unary operator.
- The logical *and* and logical *or* are binary operators.

8.9.2. Examples

- In the below code we write the statements `10 <= 20 && 20 <= 30`, we cannot write chained logical operators such as for example: $10 \leq 20 \leq 30$ we would need to write $10 \leq 20$ and $20 \leq 30$.
- Other examples:

```cpp
num1 >= 10 && num1 < 20; // return true if num1 is greater or equal to 10 and num1 is less than 20.
num1 <= 10 || num1 >= 20; // return true if num1 is less than or equal to 10 or num1 greater or equal to 20.
!is_raining && temperature > 32.0; // return true if it is not raining and the temperature is above 32.0.
is_raining || is_snowing; // returns true if it is raining or snowing.
temperature > 100 && is_humid || is_raining; // returns true if temperature is above 100 and it is humid or raining.
```

8.9.3. Short-Circuit evaluation

- When evaluating a logical expression in C++ stops as soon as the result is known, for example:

```cpp
expr1 && expr2 && expr3; // if expr1 is false there is no way the statement is true, so it simply stops.
expr1 || expr2 || expr3; // if expr1 is true it already knows the statement is true because only one true expression is required to be true in the logical or for the statement to be true.
```

8.9.4. Example

- Example of user entering numbers within bounds:

```cpp
#include <iostream>
using namespace std;
int main() {
    int num {0};
    const int lower {10}, upper {20};
    cout << boolalpha;
    cout << "Enter an integer - the bounds are " << lower << " and " << upper << ": ";
    cin >> num;
    bool within_bounds {false};
    within_bounds = (num > lower && num < upper);
    cout << num << " is between " << lower << " and " << upper << " : " << within_bounds << endl;
    return 0;
}
/* OUTPUT:
Enter an integer - the bounds are 10 and 20: 15 13
15 is between 10 and 20 : true

*/
```

- Example outside the lower and upper bounds:

```cpp
#include <iostream>
using namespace std;
int main() {
    int num {0};
    const int lower {10}, upper {20};
    cout << boolalpha;
```

```
    cout << "Enter an integer out of bounds - the bounds are " << lower << " and "
    << upper << ": ";
    cin >> num;
    bool within_bounds {false};
    within_bounds = (num < lower || num > upper);
    cout << num << " is outside " << lower << " and " << upper << " : " <<
    within_bounds << endl;
    return 0;
}
/* OUTPUT:
Enter an integer out of bounds - the bounds are 10 and 20: 3 45
3 is outside 10 and 20 : true

*/
```

8.10. Compound assignment operators

- Compound assignment operators:

Operator	Example	Meaning
+=	lhs += rhs;	lhs = lhs + (rhs);
-=	lhs -= rhs;	lhs = lhs - (rhs);
*=	lhs *= rhs;	lhs = lhs * (rhs);
/=	lhs /= rhs;	lhs = lhs / (rhs);
%=	lhs %= rhs;	lhs = lhs % (rhs);
>>=	lhs >>= rhs;	lhs = lhs >> (rhs);
<<=	lhs <<= rhs;	lhs = lhs << (rhs);
&=	lhs &= rhs;	lhs = lhs & (rhs);
^=	lhs ^= rhs;	lhs = lhs ^ (rhs);
|=	lhs |= rhs;	lhs = lhs | (rhs);

- Example:

```
a += 1; // a = a + 1
a /= 5; // a = a / 5
a *= b + c; // a = a * (b + c)
```

8.11. Operator Precedence

You can find a table of the complete precedence and associativity here.

- What is associativity?
  - Use precedence rules when adjacent operators are different.

expr1 **op1** expr2 **op2** expr3 // precedence.

- Use associativity rules when adjacent operators have the same precedence.

expr1 **op1** expr2 **op1** expr3 // associativity.

- Use parenthesis to absolutely remove any doubt.

8.11.1. Example

- Precedence viewed with parenthesis:

$$result = num1 + num2 * num3;$$
$$result = (num1 + (num2 * num3));$$

- Use associativity from left to right since additiona and subtraction have the same precedence.

$$result = num1 + num2 - num3;$$
$$result = ((num1 + num2) - num3);$$

Capítulo 9

Controlling program flow

9.1. Section overview

- Sequence: So far what we've learned has been sequential programming, a series of statements running sequentially, our code can't make decisions.
- Selection structures: allow us to make decisions based on certain conditions being true or false.
- Iteration: looping or repeating.

With sequence, selection and iteration we can implement any algorithm.

9.1.1. Selection: Decision

- `if` statement.
- `if else` statement.
- Nested `if` statements.
- `switch` statement.
- Conditional operator `?:`;

9.1.2. Iteration: Looping

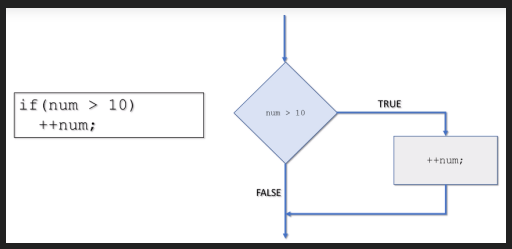
- `for` loop.
- Range-Based `for` loop.
- `while` loop.
- `continue` and `break`.
- Infinite loops.
- Nested loops.

9.2. If statement

- Syntax is: `if (expr) { statement; }`.
- The control expression must evaluate to a boolean true or false value.
- If the expression is true then execute the statement.
- If the expression is false then skip the statement.
- As a recommendation we always indent the code inside the if statement.

9.2.1. Examples of single if statements

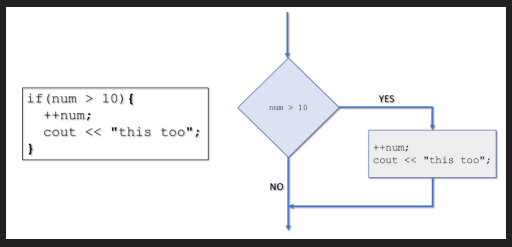
For single if statements you do not need to enclose the statements in curly braces.




```
if (selection == 'A')
    cout << "You selected A";
if (num > 10)
    cout << "num is grater than 10";
if (health < 100 && player_healed)
    health = 100;
```

9.2.2. Block if statements

- A block of code we have already seen in the main function.
- Create a block of code by including more than one statement in code block and adding {}.
- Blocks can also contain variable declarations.
- These variables are visible only within the block (local scope).



9.2.3. Example of block ifs

```cpp
#include <iostream>
using namespace std;
int main() {
    int num {0};
    const int min {10}, max {100};
    cout << "Enter a number between " << min << " and " << max << ": ";
    cin >> num;
    if (num >= min) {
        cout << "\n================if statment 1==================" << endl;
        cout << num << " is grater than " << min << endl;
        int diff {num - min};
        cout << num << " is " << diff << " greater than " << min << endl;
    }

    if (num <= max) {
        cout << "\n================if statment 2==================" << endl;
        cout << num << " is less than or equal to " << max << endl;
        int diff {max - num};
        cout << num << " is " << diff << " less than " << max << endl;
    }

    if (num >= min && num <= max) {
        cout << "\n================if statment 3==================" << endl;
        cout << num << " is in range " << endl;
        cout << "This means statement 1 and 2 must also display!" << endl;
    }
    if (num == min || num == max) {
        cout << "\n================if statment 4==================" << endl;
        cout << num << " is in boundaries " << endl;
        cout << "This means statement 1, 2 and 3 must also display!" << endl;
    }
    return 0;
}
/* OUTPUT:
Enter a number between 10 and 100: 100

================if statment 1==================
100 is grater than 10
100 is 90 greater than 10

================if statment 2==================
100 is less than or equal to 100
100 is 0 less than 100

================if statment 3==================
100 is in range
This means statement 1 and 2 must also display!

================if statment 4==================
100 is in boundaries
This means statement 1, 2 and 3 must also display!

```

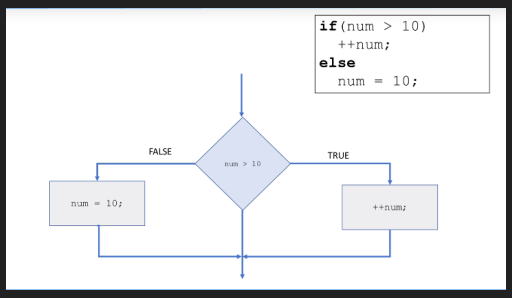
```
*/
```

9.3. If else statement

- Syntax:

```
if (expr)
    statement;
else
    statement2;
```

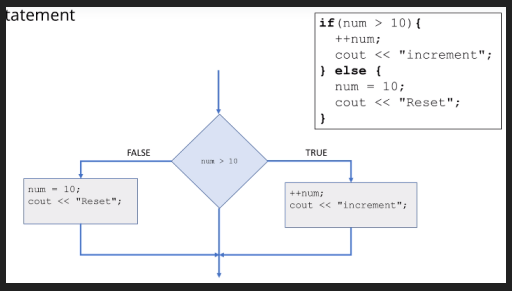
- If the expression is true then execute statement1.
- If the expression is false then execute statement2.
- Never forget the indentation.



9.3.1. Example of single if else statements

```
if (num > 10)
    cout << "num is greater than 10";
else
    cout << "num is NOT greater than 10";

if (health < 100 && heal_player)
    health = 100;
else
    ++health;
```

9.3.2. If else block



9.3.3. If else if construct

- Sometimes we need to check multiple conditions not just weather the expression is true but other characteristics, for this we use an if else if construct.

```
if (score > 90)
    cout << "A";
else if (score > 80)
    cout << "B";
else if (score > 70)
    cout << "C";
else if (score > 60)
    cout << "D";
else
    cout << "F";
cout << "Done";
```

9.3.4. Example

```
#include <iostream>
using namespace std;
int main() {
    int num{};
    const int target {10};
    cout << "Enter a number and I'll compare it to " << target << ": ";
    cin >> num;

    if (num >= target) {
        cout << "\n==========================" << endl;
        cout << num << " is greater than or equal to " << target << endl;
        int diff {num - target};
        cout << num << " is " << diff << " greater than " << target << endl;
    } else {
        cout << "\n==========================" << endl;
        cout << num << " is less than " << target << endl;
        int diff {target - num};
        cout << num << " is " << diff << " less than " << target << endl;
    }
    cout << endl;
    return 0;
}
/* OUTPUT:
Enter a number and I'll compare it to 10: -10

==========================
-10 is less than 10
-10 is 20 less than 10

*/
```

9.4. Nested if statement

- You can nest if statements within other if statements.

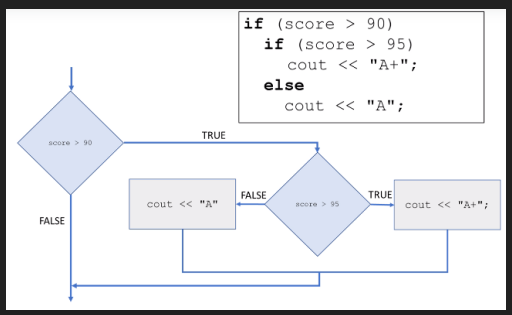
```
if (expr1)
    if (expr2)
        statement1;
    else
        statement2;
```

- If the expression is true then execute statement1.
- If the expression is false then execute statement2.
- Many times we need more logic to the if statement.
- In the above example the if statement expects one statement without the curly braces, however the if statement is a one compound statement and that is why the above code doesn't need curly braces.
- Notice, how does the computer know if the else statement belongs to the closest or farthest if? In C++ each else belogs to the closest if, so in the above example it belongs to the if holding the expr2.
- Remember to indent.

9.4.1. Example

- Example of scores:

```
if (score > 90)
    if (score > 95)
        cout << "A+";
    else
        cout << "A";
cout << "Sorry, No A";
```



- Example:

```
if (score_frank != score_bill) {
    if (score_frank > score_bill) {
        cout << "Frank wins" << endl;
    } else {
        cout << "Bill wins" << endl;
    }
} else {
    cout << "Looks like a tie!" << endl;
}
```

- Example of nested latter:

```cpp
#include <iostream>
using namespace std;
int main() {
    int score {};
    cout << "Enter your score in the exam (0-100): ";
    cin >> score;
    char letter_grade {};

    if (score >= 0 && score <= 100) {
        if (score >= 90)
            letter_grade = 'A';
        else if (score >= 80)
            letter_grade = 'B';
        else if (score >= 70)
            letter_grade = 'C';
        else if (score >= 60)
            letter_grade = 'D';
        else
            letter_grade = 'F';
        cout << "Your grade is: " << letter_grade << endl;
        if (letter_grade == 'F')
            cout << "Bad grade." << endl;
        else
            cout << "Congrats." << endl;
    } else {
        cout << "Sorry, " << score << "is not in range." << endl;
    }
    return 0;
}
/* OUTPUT:
Enter your score in the exam (0-100): 90
Your grade is: A
Congrats.

*/
```

9.5. Switch-case statement

- There are some applications where you need to execute certain blocks of code depending on the value of a constant, for this you can use the `switch` statement.

```cpp
switch (integer_control_expr) {
    case expression_1: statement_1; break;
    case expression_2: statement_2; break;
    case expression_3: statement_3; break;
    case expression_4: statement_4; break;
    ...;
    default: statement_default; break;
}
```

- Syntax: The switch keyword is followed by the control expression in parentheses. This control expression

must evaluate to an integer type or an enumeration type. Then you have a series of case statements, the value of the control expression will be compared to the values contained in each case, these also need to be constants or literals pertaining to an enumeration or integral type. If no values match the switch will execute the default, this is like the else. The break statement in C++ are optional but best practice is to include them unless you have a very good reason not to. A break statement is not needed in the default case.

- If one case is followed by another such as: the switch will act as a logical or.

```
case '3':
case '4': cout << "3 or 4 selected";
```

It is the same as saying:

```
if ('3' || '4')
    cout << "3 or 4 selected";
```

- Once the switch has hit a match, no further cases will be checked.

9.5.1. Example

- Suppose `selection` is an character initialized to the character 2.

```
switch (selection) {
    case '1': cout << "1 selected"; // compare selection to 1, not true.
            break;
    case '2': cout << "2 selected"; // compare selection to 2, is true, execute
    ↪  the code from the colon to the break.
            break;
    case '3': cout << "3 selected";
            break;
    case '4': cout << "4 selected";
            break;
    default: cout << "1,2,3,4 NOT selected";
}
```

- Example of fall-through:

```
#include <iostream>
using namespace std;
int main() {
    char selection {'2'};
    switch (selection) {
        case '1': cout << "1 selected\n";
                break;
        case '2': cout << "2 selected\n"; // this case evaluates 2 or 3 or 4 or 5.
        case '3': cout << "3 selected\n";
        case '4': cout << "4 selected\n";
        case '5': cout << "5 selected\n";
                break;
        case '6': cout << "4 selected\n";
                break;
        default: cout << "1,2,3,4 NOT selected\n";
    }
    return 0;
```

```
}
/* OUTPUT:
2 selected
3 selected
4 selected
5 selected

*/
```

- The switch statement is commonly used with enumeration, meaning the `enum`, for example:

```
#include <iostream>
using namespace std;
int main() {
    enum Color {
        red, green, blue
    };
    Color screen_color {green};
    switch (screen_color) {
        case red: cout << "red"; break;
        case green: cout << "green"; break;
        case blue: cout << "blue"; break;
        default: cout << "should never execute";
    }
    return 0;
}
/* OUTPUT:
green
*/
```

9.5.2. Review

- The control expression must evaluate to an integer type.
- The case expressions must be constant expressions that evaluate to integer or integers literals. They must be constants or literals, no variables.
- Once a match occurs all the code in the case statement is executed until a break is reached, then the switch terminates.
- Best practice is to provide break statement for each case.
- Best practice is also to provide `default` case, this is optional but should be handled in case it does get executed.

9.5.3. Example

- Switch with characters:

```
#include <iostream>
using namespace std;
int main() {
    char letter_grade{};
    cout << "Enter the letter grade you expect on the exam: ";
    cin >> letter_grade;
```

```
    switch (letter_grade) {
    case 'a':
    case 'A':
        cout << "90 or above." << endl;
        break;
    case 'b':
    case 'B':
        cout << "80-89" << endl;
        break;
    case 'c':
    case 'C':
        cout << "70-79" << endl;
        break;
    case 'd':
    case 'D':
        cout << "60-69" << endl;
        break;
    case 'f':
    case 'F': {
        char confirm{};
        cout << "Are you sure (Y/N)?";
        cin >> confirm;
        if (confirm == 'y' || confirm == 'Y') {
            cout << "0-60" << endl;
        } else {
            cout << "Illegal choice." << endl;
        }
        break;
    }
    default:
        cout << "Invalid input." << endl;
        break;
    }
    return 0;
}
/* OUTPUT:
Enter the letter grade you expect on the exam: f
Are you sure (Y/N)?y
0-60

*/
```

- Switch with enumeration (`enum`):

```
#include <iostream>
using namespace std;
int main() {
    enum Direction {
        left, right, up, down
    };
    Direction heading {left};

    switch (heading) {
```

```
        // some compilers won't let you compile if you don't handle every case. So
↪  it would not compile if we forget to  handle the 'right' case for example.
        case left:
            cout << "Going left" << endl;
            break;
        case right:
            cout << "Going right" << endl;
            break;
        case up:
            cout << "Going up" << endl;
            break;
        case down:
            cout << "Going down" << endl;
            break;
        default:
            cout << "OK" << endl;
    }
    return 0;
}
/* OUTPUT:
Going left
*/
```

9.6. Conditional operator

- Syntax is: `(cond_expr) ? expr1 : expr2;`
- This is frequently called the ternary operator.
- `cond_expr` evaluates to a boolean expression.
  - If `cond_expr` is true the value of `expr1` is returned.
  - If `cond_expr` is false the value of `expr2` is returned.
- Similar to an if-else construct in a single expression.
- Ternary operator.
- Very useful when used inline.
- Very easy to abuse! Best practice is to never nest a conditional operator within another one, this causes the code to become unreadable and difficult to manage.

9.6.1. Example

- Example:

```
#include <iostream>
using namespace std;
int main() {
    int a {10}, b {20};
    int score {92};
    int result {};
    result = (a > b) ? a : b; // false.
    cout << result << endl;
```

```
    result = (a < b) ? (a - b) : (a-b); // true.
    cout << result << endl;
    result = (b != 0) ? (a / b) : 0; // false.
    cout << result << endl;
    cout << ((score > 90)? "Excelent":"Good"); // true.
    return 0;
}
/* OUTPUT:
20
-10
0
Excelent
*/
```

- You can use conditional operators to write ifs in one single line:

```
#include <iostream>
using namespace std;
int main() {
    int num {};
    cout << "Enter an int: ";
    cin >> num;
    // usually written with ifs:
    if (num % 2 == 0)
        cout << num << " is even." << endl;
    else
        cout << num << " is odd." << endl,
    cout << endl;

    // you could do it with the conditional operator:
    cout << num << " is" << ((num % 2 == 0) ? " even." : " odd.") << endl;

    return 0;
}
/* OUTPUT:
Enter an int: 6
6 is even.
6 is even.

*/
```

- Another example:

```
#include <iostream>
using namespace std;
int main() {
    int num1 {}, num2 {};
    cout << "Enter two integers separated by a space: ";
    cin >> num1 >> num2;
    if (num1 != num2) {
        cout << "Largest: " << ((num1 > num2)?num1:num2) << endl;
    } else {
        cout << "The numbers are the same." << endl;
    }
```

```
    return 0;
}
/* OUTPUT:
Enter two integers separated by a space: 10 11
Largest: 11

*/
```

9.7. Looping

- Looping is also called iteration and is the third basic building block of programming.
  - Sequence, selection, iteration. Remember with these building blocks you can implement any algorithm.
- Iteration or repetition.
- Allows the execution of a statement or block of statements repeatedly.
- Loops are made up of a loop condition and the body which contains the statements to repeat.

9.7.1. Use cases of looping

Execute a loop:

- A specific number of times, such as the user enters the number of times.
- For each element in a collection of elements.
- While specific condition(s) remains true.
- Until a specific condition becomes false.
- Until we reach the end of some input stream.
- Forever (infinite loops) (for example operating systems).
- Many, many more use cases.

9.7.2. C++ looping constructs

- `for` loop:
  - Iterate specific number of times.
- Range-based for loop:
  - One iteration for each element in a range or collection.
  - Used with arrays and vectors for example.
- `while` loop:
  - Iterate while condition remains true.
  - Stop when the condition becomes false.
  - Check the condition at the beginning of every iteration.
- `do while` loop:
  - Iterate while a condition remains true.

- Stop when the condition is false.
- Check the condition at the end of every iteration.
- The same as the while loop only difference is that the do while checks the condition after each iteration, the while loop checks conditions before each iteration.

9.8. `for` loop

- You can omit the curly braces if only one statement will be executed, if two or more are executed that will need to be enclosed in a block of code.

```
for (initalization; condition; increment)
    statement;
for (initalization; condition; increment) {
    statement1;
    statement2;
    statementn;
}
```

- The initialization is executed exactly once. Then the condition is checked, if its true it will execute the block of code and then the increment will take place, when the condition evaluates to false the loop terminates.

9.8.1. Examples

- Count from one to five:

```
    #include <iostream>
using namespace std;
int main() {
    int i {0};
    for (i = 1; i <= 5; ++i) {
        cout << i << endl;
    }

    return 0;
}
/* OUTPUT:
1
2
3
4
5
*/
```

- You can declare the conter variable (if you must use one) and initialize all inside the for loop, the advantage to this is that you don't need to declare it outside (like the previous example), however that variable will remain only within the score of the for loop, you cannot use it else where.

```
#include <iostream>
using namespace std;
int main() {
    for (int i {1}; i <= 5; ++i) {
        cout << i << endl;
```

```
    }
    for (int i = 1; i <= 5; ++i) {
        cout << i << endl;
    }
    cout << i << endl; // compiler error, i was not declared in this scope.
    return 0;
}
```

Correction:

```
#include <iostream>
using namespace std;
int main() {
    for (int i {1}; i <= 5; ++i) {
        cout << i << endl;
    }
    for (int i = 1; i <= 5; ++i) {
        cout << i << endl;
    }
    return 0;
}
/* OUTPUT:
1
2
3
4
5
1
2
3
4
5
*/
```

- Display even numbers from one to ten:

```
#include <iostream>
using namespace std;
int main() {
    for (int i {1}; i <= 10; ++i) {
        if (i % 2 == 0) {
            cout << i << endl;
        }
    }
    return 0;
}
/* OUTPUT:
2
4
6
8
10
*/
```

- Use for loops to iterate an array:

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,90,87};
    for (int i {0}; i < 3; ++i) {
        cout << scores[i] << endl;
    }
    return 0;
}
/* OUTPUT:
100
90
87
*/
```

  - Very careful with array out of bounds errors.

- You can initialize many variables using the comma operator:

```cpp
#include <iostream>
using namespace std;
int main() {
    for (int i {1}, j {5}; i <= 5; ++i, ++j) {
        cout << i << " * " << j << " : " << (i * j) << endl;
    }
    return 0;
}
/* OUTPUT:
1 * 5 : 5
2 * 6 : 12
3 * 7 : 21
4 * 8 : 32
5 * 9 : 45
*/
```

  - Note that the associativity is right to left, and the result of the comma operator is the left most expression.
  - In this case we just use them to initialize two variables and then to increment those two variables.

9.8.2. Other details

- The basic for loop is very clear and concise.
- Since the for loop's expressions are all optional, it is possible to have:
  - No initialization, No condition, and No increment.
  - This creates an infinite loop, and as we will see later on is the same as using the while loop with the condition being just true always.
  - You can also use doubles and other types as counters.
  - Best practice is to never include any expressions too complicated to document, if the expressions are too complicated consider using another looping construct.

```cpp
for (;;) {
    cout << "Endless loop" << endl;
}
```

9.8.3. Examples

- Count from one to ten:

```cpp
#include <iostream>
using namespace std;
int main() {
    for (int i {1}; i <= 10; ++i) {
        cout << i << endl;
    }
    return 0;
}
/* OUTPUT:
1
2
3
4
5
6
7
8
9
10

*/
```

- Count from one to ten every two:

```cpp
#include <iostream>
using namespace std;
int main() {
    for (int i {1}; i <= 10; i += 2) {
        cout << i << endl;
    }
    return 0;
}
/* OUTPUT:
1
3
5
7
9

*/
```

- For decrementing:

```cpp
#include <iostream>
using namespace std;
```

```
int main() {
    for (int i {10}; i > 0; --i) {
        cout << i << endl;
    }
    return 0;
}
/* OUTPUT:
10
9
8
7
6
5
4
3
2
1

*/
```

- Iterating a vector:

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> nums {10,20,30,40,50};
    for (unsigned i{0}; i < nums.size(); ++i) {
        cout << nums[i] << endl;
    }
    return 0;
}
/* OUTPUT:
10
20
30
40
50

*/
```

9.9. Range based for loop

- The idea with the range based for loop is to loop through a collection of elements and be able to easily access each element, without having to worry about the length of the collection or incrementing or decrementing looping variables or subscripting indexes.

- The syntax is:

```
for (var_type var_name : collection_name)
    statement1; // can use var_name.
    for (var_type var_name : collection_name) {
        statements; // can use var_name.
```

```
}
```

- `var_type` will be bound to each element of the collection so it should be of the same type the collection of elements.
- Now when we access each variable name in the loop it will have a specific element in the collection.
- For example:

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,90,97};
    for (int score : scores) {
        cout << score << endl;
    }
    return 0;
}
/* OUTPUT:
100
90
97

*/
```

- Instead of providing the `var_type` you can use the `auto` keyword, this allows the C++ compiler to deduce the type itself.

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,90,97};
    for (auto score : scores) {
        cout << score << endl;
    }
    return 0;
}
/* OUTPUT:
100
90
97

*/
```

9.9.1. Examples

- Vector example:

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<double> temps {87.2, 77.1, 80.0, 72.5};
    double average_temp {};
```

```
    double running_sum {};
    for (auto temp : temps) {
        running_sum += temp;
    }
    average_temp = running_sum / temps.size();
    cout << average_temp << endl;
    return 0;
}
/* OUTPUT:
79.2
*/
```

- Intializer list:

```
#include <iostream>
using namespace std;
int main() {
    double average_temp {};
    double running_sum {};
    int size {};
    for (auto temp : {60.2, 80.1, 90.0, 78.2}) {
        running_sum += temp; size++;
    }
    average_temp = running_sum / size;
    cout << average_temp << endl;
    return 0;
}
/* OUTPUT:
77.125
*/
```

- Looping a string:

```
#include <iostream>
using namespace std;
int main() {
    for (auto c: "C++") {
        cout << c << endl;
    }
    return 0;
}
/* OUTPUT:
C
+
+

*/
```

9.10. While

- The while loop is an example of a pre-test loop because the expression is evaluated before executing the code.
- If the expression evaluates to true the code is executed, else it is not executed.

- Syntax is:

```
while (expression)
    statement;
while (expression) {
    statement (s);
}
```

- The `expression` has to evaluate to a boolean value. Followed by the statement or the block of statements.

9.10.1. Example

- Counting from 1-5.

```cpp
#include <iostream>
using namespace std;
int main() {
    int i {1};
    while (i <= 5) {
        cout << i << endl;
        ++i; // increment
    }
    return 0;
}
/* OUTPUT:
1
2
3
4
5

*/
```

- Even numbers between 1 and 10.

```cpp
#include <iostream>
using namespace std;
int main() {
    int i {1};
    while (i <= 10) {
        if (i % 2 == 0)
            cout << i << endl;
        ++i;
    }
    return 0;
}
/* OUTPUT:
2
4
6
8
10
```

```
18
19 */
```

- Array example:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int scores[] {100,90,87};
5      int i {0};
6      while (i < 3) {
7          cout << scores[i] << endl;
8          ++i;
9      }
10     return 0;
11 }
12 /* OUTPUT:
13 100
14 90
15 87
16
17 */
```

- Input validation:

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int number {};
5      cout << "Enter an integer less than 100: ";
6      cin >> number;
7      while (number < 100) {
8          cout << "Enter an integer less than 100: ";
9          cin >> number;
10     }
11     cout << "Thanks" << endl;
12     return 0;
13 }
14 /* OUTPUT:
15 Enter an integer less than 100: 100
16 Enter an integer less than 100: 99
17 Thanks
18
19 */
```

9.11. do-while loop

- Syntax is:

```
do {
    statements;
} while (expression);
```

- The do-while loop is a post-test loop, because the expression is tested at the end of each iteration.
- This means that the loop body is garanteed to be executed at least once.

9.11.1. Examples

- Input validation:
  - In the below code, notice that variable declarations cannot be done inside the do-while code body, if you do this you will get a compiler error.
  - You cannot declare variables that are involved in the expression inside the code body.

```cpp
#include <iostream>
using namespace std;
int main() {
    int number {};
    do {
        cout << "Enter an integer between 1 and 5: ";
        cin >> number;
    } while (number <= 1 || number >= 5);
    cout << "Thanks" << endl;
    return 0;
}
/* OUTPUT:
Enter an integer between 1 and 5: 0
Enter an integer between 1 and 5: 9
Enter an integer between 1 and 5: 3
Thanks

*/
```

- Input validation:
  - Remember to declare the variable used in the expression outside the do-while loop. You can declare any variable you want inside the du-while loop as long as you don't use it in the expression.

```cpp
#include <iostream>
using namespace std;
int main() {
    char selection {};
    do {
        double width {}, heigth {};
        cout << "Enter width and heigth: ";
        cin >> width >> heigth;
        double area {width * heigth};
        cout << "The area is " << area << endl;
        cout << "Calculate another? (Y/N): ";
        cin >> selection;
    } while (selection == 'Y' || selection == 'y');
    cout << "Thanks" << endl;
    return 0;
}
/* OUTPUT:
Enter width and heigth: 1 2
The area is 2
```

```
Calculate another? (Y/N): y
Enter width and heigth: 2 3
The area is 6
Calculate another? (Y/N): n
Thanks
*/
```

9.12. `continue` and `break`

- The continue and break statements can be used within all C++ loop constructs, they add more explicit control over the looping behaviour.
- `continue`:
  - When a continue statement is executed inside a loop, no further statements in the body of the loop are executed.
  - Control immediately goes directly to the beggining of the loop for the next iteration.
  - "Skip this iteration and continue with the next one."
- `break`
  - When a break statement is executed in a loop no further statements in the body of the loop are executed.
  - Loop is immedietly terminated.
  - Control immediately goes to the statement following the loop construct.
- Don't overuse the break and continue statement.

9.12.1. Example

- We have data and if a piece of data is -1 we don't want to process it. We want to stop if we find -99 in the vector.

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> values {1,2,-1,-99,7,8,10};
    for (auto val : values) {
        if (val == -99) {
            break;
        } else if (val == -1) {
            continue;
        } else {
            cout << val << endl;
        }
    }
    return 0;
}
/* OUTPUT:
1
2

*/
```

9.13. Infinite loops

- An infinite loop is a loop whose condition expression always evaluates to true.
- Usually this is unintended and a programmer error.
- Sometimes programmers use infinite loops and include and break statements in the body to control them.
- Sometimes infinite loops are exactly what we need, such as in:
  - Even loop in an event-driven program: common in embeded systems or mobile devices where the program listens and reacts to movement detected by the mouse or other such sensors, and this continues as long as the program is running.
  - Operating system: an infinite loop executes and breaks until you shut down your computer.

9.13.1. Example

- Infinite for loop:

```
for (;;) {
    do_something;
}
```

- Infinite while loop:

```
while (true) {
    do_something;
}
// or
while (10 < 12) { // 10 is always les than 12.
    do_something;
}
```

- Infinite do-while loop:

```
do {
    do_something;
} while (true);
```

- Breaking the infinite while loop:

```
#include <iostream>
using namespace std;
int main() {
    while (true) {
        char again {};
        cout << "Do you want to loop again? (Y/N): ";
        cin >> again;
        if (again == 'N' || again == 'n') {
            break;
        }
    }
    return 0;
}
```

```
/* OUTPUT:
Do you want to loop again? (Y/N): y
Do you want to loop again? (Y/N): y
Do you want to loop again? (Y/N): y
Do you want to loop again? (Y/N): n

*/
```

9.14. Nested loops

- Loops can be nested one inside another.
- We can nest loops as deep as we need.
- Nested loops can be as many levels deep as the program needs.
- Very useful with multi-dimensional data structures. Such as 2D arrays, 2D vectors, etcetera.
- Outer loop vs. Inner loop.

9.14.1. Examples

- 2D Looping:

```
#include <iostream>
using namespace std;
int main() {

    for (int outer_val {1}; outer_val <= 2; ++outer_val) {
    for (int inner_val {1}; inner_val <= 3; ++inner_val) {
                cout << outer_val << ", " << inner_val << endl;
            }
        }
    return 0;
}
/* OUTPUT:
1, 1
1, 2
1, 3
2, 1
2, 2
2, 3

*/
```

- Multiplication table:

```
#include <iostream>
using namespace std;
int main() {
    for (int num1 {1}; num1 <= 10; ++num1) {
        for (int num2 {1}; num2 <= 10; ++num2) {
            cout << num1 << " * " << num2 << " = " << num1 * num2 << endl;
        }
    }
```

```
    return 0;
}
/* OUTPUT:
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
4 * 10 = 40
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
6 * 1 = 6
```

```
6 * 2 = 12
6 * 3 = 18
6 * 4 = 24
6 * 5 = 30
6 * 6 = 36
6 * 7 = 42
6 * 8 = 48
6 * 9 = 54
6 * 10 = 60
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
7 * 10 = 70
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48
8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
8 * 10 = 80
9 * 1 = 9
9 * 2 = 18
9 * 3 = 27
9 * 4 = 36
9 * 5 = 45
9 * 6 = 54
9 * 7 = 63
9 * 8 = 72
9 * 9 = 81
9 * 10 = 90
10 * 1 = 10
10 * 2 = 20
10 * 3 = 30
10 * 4 = 40
10 * 5 = 50
10 * 6 = 60
10 * 7 = 70
10 * 8 = 80
10 * 9 = 90
10 * 10 = 100

*/
```

- 2D arrays - set all elements to 1000:

```cpp
#include <iostream>
using namespace std;
int main() {
    int grid[5][3] {};
    // initialize everything.
    for (int row {0}; row < 5; ++row) {
        for (int col {0}; col < 3; ++col) {
            grid[row][col] = 1'000;
        }
    }
    // display elements.
    for (int row {0}; row < 5; ++row) {
        for (int col {0}; col < 3; ++col) {
            cout << grid[row][col] << ", ";
        }
        cout << endl;
    }
    return 0;
}
/* OUTPUT:
1000, 1000, 1000,
1000, 1000, 1000,
1000, 1000, 1000,
1000, 1000, 1000,
1000, 1000, 1000,

*/
```

- 2D vector - display elements:

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<vector<int>> vector_2d {
        {1,2,3},
        {10,20,30,40},
        {100,200,300,400,500}
    };
    for (auto vec: vector_2d) {
        for (auto val: vec) {
            cout << val << " ";
        }
        cout << endl;
    }
    return 0;
}
/* OUTPUT:
1 2 3
10 20 30 40
100 200 300 400 500

*/
```

Capítulo 10

Characters and strings

10.1. Character functions

- The cctype library includes very useful functions that allow us to work with characters, such as the following:
  - Functions for testing characters.
  - Functions for converting character case.
- In order to use those functions you must include the library.

```
1 #include <cctype>
```

- The testing functions accept a single character and all evaluate to a boolean value, the conversion functions return the converted character.
- Some of the testing functions are:
  - To use them you must pass the character to evaluate.

`isalpha(c)`	True if c is a letter.
`isalnum(c)`	True if c is a letter or digit.
`isdigit(c)`	True if c is a digit.
`islower(c)`	True if c is lowercase letter.
`isprint(c)`	True if c is a printable character.
`ispunct(c)`	True if c is a punctuation character.
`isupper(c)`	True if c is an uppercase letter.
`isspace(c)`	True if c is whitespace.

- Character conversion functions:

`tolower(c)`	returns lowercase of c, if the function can't convert, it will return to the character that was passed in.
`toupper(c)`	returns uppercase of c, if the function can't convert, it will return to the character that was passed in

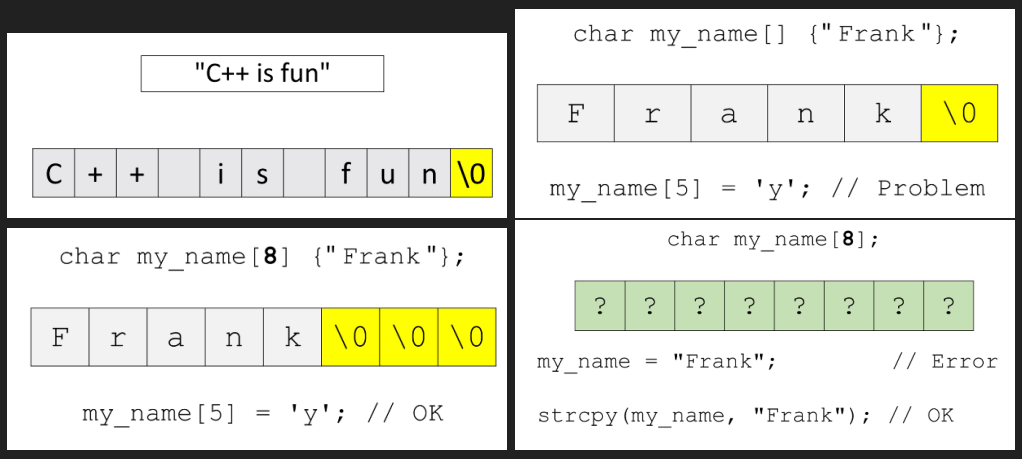
10.2. C-Style strings

- A C-Style string is a sequence of characters:
  - Stored contiguous in memory.

- Implemented as an array of characters. You can access each character using the array subscript syntax.
- Terminated by a null character (null).
  - Null character with a value of zero.
- Refered to as zero or null terminated strings.

- String literal:
  - Sequence of characters in double quotes.
  - Constant.
  - Terminated with a null character.

- How C++ stores strings in memory:



- The cstring library contains functions that work with C-style strings, these serve for: (include the library `#include <cstring>`)
  - Copying.
  - Concatenation.
  - Comparison.
  - Searching.
  - And others.

- C++ has a library called cstdlib, containing functions that convert C-Style strings to other types, such as:
  - Integer.
  - Float.
  - Long.
  - Etc.

10.2.1. Examples

- Example of string functions:

```cpp
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char str[80];
    strcpy(str, "Hello "); // copy.
    strcat(str, "there "); // concatenate.
    cout << strlen(str); // 10
    strcmp(str, "Another"); // -1
    return 0;
}
/* OUTPUT:
12
*/
```

10.3. Working with C-Style strings examples

- Example:

```cpp
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char first_name[20] {};
    char last_name[20] {};
    char full_name[20] {};
    char temp[50] {};
    cout << "Please enter your first and last name: ";
    cin >> first_name >> last_name;
    cout << "--------------------------" << endl;
    cout << "Hello, " << first_name << " your first name has " <<
 ↪  strlen(first_name) << " characters." << endl;

    strcpy(full_name, first_name);
    strcat(full_name, " ");
    strcat(full_name, last_name);

    cout << full_name << endl;
    return 0;
}
/* OUTPUT:
Please enter your first and last name: David Corzo
--------------------------
Hello, David your first name has 5 characters.
David Corzo

*/
```

- In the last example we read in the firs name and the last name separately, the program delimited the space entered by the user. In this example we will read in the full name with the space.

```cpp
#include <iostream>
```

```cpp
#include <cstring>
using namespace std;
int main() {
    char full_name[50] {};
    cout << "Please enter your first and last name: ";
    cin.getline(full_name, 50); // it will stop at 50 chars or when it finds the return character.
    cout << full_name << endl;
    return 0;
}
/* OUTPUT:
Please enter your first and last name: David Corzo
David Corzo

*/
```

- Comparing strings:

```cpp
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char str1[] {"Hello world\n"};
    char str2[] {"Hello world\n"};
    if (strcmp(str1, str2) == 0) {
        cout << "The strings are the same." << endl;
    }
    return 0;
}
/* OUTPUT:
The strings are the same.

*/
```

- Iterating and changing strings:

```cpp
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char str1[] {"Hello world."};
    for (size_t i {0}; i < strlen(str1); ++i) {
        if (isalpha(str1[i])) {
            str1[i] = toupper(str1[i]);
        }
    }
    cout << str1 << endl;
    return 0;
}
/* OUTPUT:
HELLO WORLD.

*/
```

10.4. C++ strings

- `std::string` is a class in the C++ Standard template library.
  - You must include the library `#include <string>`.
  - You must include the `std` namespace.
  - Stored contiguous in memory.
  - Dynamic size.
  - Work with input and output streams.
  - There are lots of useful member functions.
  - Our familiar operators can be used (`+`,`=`,`<`,`<=`,`>`,`>=`,`+=`,`==`,`!=`,`[]`,...). They also work with the insertion and extraction operator. This is another example of operator overloading.
  - Can be converted to C-Style strings if needed.
  - Safer to use.
- Declaring and initializing:
  - You can initialize a C++ string using any initialization style.
  - C++ strings are always initialized, unlike C-style strings that were never initialized.

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1; // Empty.
    string s2 {"Frank"}; // Frank
    string s3 {s2}; // Frank
    string s4 {"Frank", 3}; // Fra
    string s5 {s3,0,2}; // Fr
    string s6 (3, 'X'); // XXX
    return 0;
}
```

- With C++ strings you can use the assignment operator.

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1;
    s1 = "C++";
    string s2;
    s2 = s1;
    return 0;
}
```

- Concatenation can be done with the `+` operator.
  - In the below example if we would have tried to do something as:

```cpp
sentence = "C++" + " is powerful";
```

- We would have had a compiler error, because as far as C++ is concerned these are two C-Style literals, not `std::string` objects, thus addition is not concatenation.
- A combination of C-style strings and C++ strings is ok, the code below does this, however you cannot add two C-Style strings.

```cpp
#include <iostream>
using namespace std;
int main() {
    string part1 {"C++"};
    string part2 {"is powerful"};
    string sentence;
    sentence = part1 + " " + part2 + " language";
    cout << sentence << endl;
    return 0;
}
/* OUTPUT:
C++ is powerful language
*/
```

- Accessing characters [] and at() method:
  - The difference between the [] and the .at() method is that the .at() method in the `std::string` performs bounds checking and the [] does not. As a best practice, always use the .at() method.

```cpp
#include <iostream>
using namespace std;
int main() {
    string s1;
    string s2 {"Frank"};
    cout << s2[0] << endl; // F
    cout << s2.at(0) << endl; // F
    return 0;
}
/* OUTPUT:
F
F

*/
```

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1 {"Frank"};
    for (char c : s1) {
        cout << c << endl;
    }
    return 0;
}
/* OUTPUT:
F
r
a
```

```
n
k

*/
```

- Comparing `std::string` objects can be done with the `==`, `!=`, `>`, `>=`, `<`, `<=` operators. We can perform comparisons in the following cases:
  - When we have two `std::string` objects.
  - When we have one `std::string` object and C-style string literal.
  - When we have `std::string` object and C-style string variable.
  - We cannot perform this comparison if both are string literals.
  - Greater than, less than, or equal are done by comparing the values in the ASCII table.

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    cout << boolalpha;
    string s1 {"Apple"};
    string s2 {"Banana"};
    string s3 {"Kiwi"};
    string s4 {"apple"};
    string s5 {s1};
    cout << (s1 == s5) << endl; // true
    cout << (s1 == s2) << endl; // false
    cout << (s1 != s2) << endl; // true
    cout << (s2 > s1) << endl; // true
    cout << (s4 < s5) << endl; // false
    cout << (s1 == "Apple") << endl; // true
    return 0;
}
/* OUTPUT:
true
false
true
true
false
true

*/
```

- You can extract substrings from a string using the `substr()` method.
  - This method extracts a substring from a `std::string`.

```
object.substr(start_index, lenght);
```

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1 {"This is a test"};
```

```
    cout << s1.substr(0,4) << endl; // This
    cout << s1.substr(5,2) << endl; // is
    cout << s1.substr(10,4) << endl; // test
    return 0;
}
/* OUTPUT:
This
is
test

*/
```

- To find a substring in a `std::string` you can use the `find()` method.
  - This method returns the index of a substring in a `std::string`.

```
object.find(search_string);
```

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1 {"This is a test"};
    cout << s1.find("This") << endl;
    cout << s1.find("This") << endl;
    cout << s1.find("is") << endl;
    cout << s1.find("test") << endl;
    cout << s1.find('e') << endl;
    cout << s1.find("is",4) << endl; // The 4 represents from what index forward you want to start to look.
    cout << s1.find("XX") << endl; // its not found so the value returned is string::npos
    return 0;
}
/* OUTPUT:
0
0
2
10
11
5
18446744073709551615

*/
```

  - There is also an `.rfind` method that starts searching from the end of the string.

- You can also remove characters using the `erase()` and `clear()` methods.
  - This methods removes a substring of characters from a `std::string`.

```
object.erase(start_index, lenght);
```

```
#include <iostream>
#include <string>
```

```cpp
using namespace std;
int main() {
    string s1 {"This is a test"};
    cout << s1.erase(0,5) << endl;
    cout << s1.erase(5,4) << endl;
    s1.clear(); // clears the string.
    return 0;
}
/* OUTPUT:
is a test
is a

*/
```

- To obtain the length of the string use:

```cpp
object.length();
```

- To cocatenate you can use the `+=` operator:

```cpp
s1 += " James"; // concatenates s1 and " James".
```

- Input with C++ strings:
    - The `cin` reads until it finds a space, sometimes we want to read up until the user enters the return character. For this we can use `getline(cin, s1);`.

```cpp
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s1;

    getline(cin, s1); // reads entire line.
    cout << s1 << endl;

    getline(cin, s1, 'x'); // you can add a delimeter.
    cout << s1 << endl;
    return 0;
}
/* OUTPUT:
Hello world
Hello world
Hello world x
Hello world

*/
```

Capítulo 11

Functions

11.1. What is a function?

- In C++ programs we typically make use of:
  - C++ standard library (functions and classes).
  - Third-party library (functions and classes).
  - Our own functions and classes.
- Functions allow the modularization of a program.
  - Which allow us to separate code into logical self-contained units.
  - These units can be reused.
- Why we want to use functions?
  - Which code is better?
  - Without functions:

```cpp
int main() {
    // read input
    statement1;
    statement2;
    statement3;
    statement4;
    // process input
    statement5;
    statement6;
    statement7;
    // provide input
    statement8;
    statement9;
    statement10;
    return 0;
}
```

  - With functions:

```cpp
int main() {
    // read input
    read_input();
```

```
    // process input
    process_input();
    // provide input
    provide_input();
    return 0;
}
```

- Functions allow for abstraction.
- Functions allow us to use the same code again and again and not having to be copying and pasting everything.

- What is a function? The boss/worker analogy.
  - Write your code to the function specification.
  - Understand what the function does.
  - Understand what information the function needs.
  - Understand what the function returns.
  - Understand any errors the function my produce.
  - Understand any performance constraints.
  - Don't worry about how the function works internally:
    - Unless you are the one writing the function.
    - This concept is called information hidding.
- Example: `<cmath>`
  - Common mathematical calculations.
  - Global functions called as:
    - Global functions mean they are available to the entire program.

```
function_name(argument);
function_name(argument1, argument2, ...);
```

- User defined functions:
  - Implement only the functions you need, use the functions already implemented if posible.
  - `https://en.cppreference.com/w/cpp/header` You can check functions.

11.1.1. Example

- Math:

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double num {12.5};
    cout << sqrt(num) << endl;
    return 0;
}
/* OUTPUT:
3.53553
*/
```

11.2. Function definition

Functions follow four main parts:

- Functions perform operations.
- Once a function is defined you can call it from anywhere in the program.
- Functions need to be declared before they are used.
- Name:
  - The name of the function.
  - Function names must be descriptive.
  - Same rules as for variable names.
  - Should be meaningful.
  - Usually a verb or verb phrase.
- Parameter list:
  - The variables passed into the function.
  - You can pass no variables.
  - Their types must be specified.
- Return type:
  - The type of the data that is returned from the function.
  - It's possible that the function does not return anything, in that case the type is `void`.
- Body:
  - The statements that are executed when the function is called.
  - In curly braces {}.

11.2.1. Example

- Example of function:

```
int function_name(int a) {
    return a;
}
```

- Example of area of the circle:

```
#include <iostream>
using namespace std;

const double pi {3.4159};

double calc_area_circle(double radious) {
    return pi * radious * radious;
}

void area_circle() {
    double radious {};
    cout << "Enter the radious of the circle: ";
```

```
13      cin >> radious;
14
15      cout << "The area of a circle with radious " << radious << " is " <<
    ↪   calc_area_circle(radious) << endl;
16  }
17
18  double calc_volume_cylinder(double radious, double height) {
19      return calc_area_circle(radious) * height;
20  }
21
22  void volume_cylinder() {
23      double radious {};
24      double height {};
25      cout << "Enter the radious of the cylinder: ";
26      cin >> radious;
27      cout << "Enter the height of the cylinder: ";
28      cin >> height;
29      cout << "The volume of a cylinder with radious " << radious << " and height "
    ↪   << height << " is " << calc_volume_cylinder(radious, height) << endl;
30  }
31
32  int main() {
33      area_circle();
34      volume_cylinder();
35      return 0;
36  }
37  /* OUTPUT:
38  Enter the radious of the circle: 78
39  The area of a circle with radious 78 is 20782.3
40  Enter the radious of the cylinder: 78
41  Enter the height of the cylinder: 90
42  The volume of a cylinder with radious 78 and height 90 is 1.87041e+06
43
44  */
```

11.3. Function prototypes

- The compiler must know about a function before it is used.
- This implies that either you define functions before calling them or you can use *function prototypes.*
- Defining functions before they are called is OK for small programs, however not at all practical for larger programs.
- Using function prototypes allows us to define the functions anywhere in the code, because they are technically being 'declared' beforehand.
    - Function prototypes tells the compiler what it needs to know without a full function definition.
    - Also called "forward declarations" since you are telling the compiler how the function is going to look like and the compiler will enforce those rules.
    - Function prototypes are placed at the beginning of a program.
    - Also used in out own header files (.h). Functions defined in header files must contain their corresponding function prototypes in the same file.

- Syntax of function prototype:

```
int function_name(); // prototype.

int function_name() { // function definition.
    statement(s);
    return 0;
}
```

- Function prototypes can include parameter names.

```
int function_name(int); // prototype with out names.
int function_name(int a); // prototype with parameter names.
int function_name(int a) {
    statement(s);
    return 0;
}
```

- With `void` the return type is optional:

```
void function_name(); // prototype.
void function_name() {
    statement(s);
    return; // optional
}
```

- Function parameters help the compiler enforce the rules, for example what parameters it takes in, what types are those parameters, etcetera.

```
#include <iostream>
using namespace std;

void say_hello();

int main() {
    say_hello(); // OK
    say_hello(100); // error
    cout << say_hello(); // error, cout expects to print something and there is no
    ↪ return value
    return 0;
}
/* OUTPUT:

*/
```

11.4. Function parameters and the return statement

11.4.1. Function parameters

- Function parameters:

- When we call a function we can pass in data to that function.
- In the function call they are called arguments.
- In the function definition they are called parameters.
- They must match in number, order, and type.

```cpp
#include <iostream>
using namespace std;

int add_numbers(int, int); // prototype.

int main() {
    int result {0};
    result = add_numbers(100,200); // call.
    cout << result << endl;
    return 0;
}

int add_numbers(int a, int b) { // function definition.
    return a + b;
}

/* OUTPUT:
300
*/
```

- If the compiler knows how to convert a type it will convert.

```cpp
#include <iostream>
using namespace std;

void say_hello(string);

void say_hello(string name) {
    cout << "Hello " << name << endl;
}

int main() {
    say_hello("C++"); // Sending a string literal not a std::string. The compiler
    ↪ will convert.
    return 0;
}
/* OUTPUT:
Hello C++

*/
```

- Pass-by-value:
  - When you pass data into a function it is passeded-by-value.
  - A copy of the data is passed to the function.
  - Whatever changes you make to the parameter in the function does NOT affect the argument that was passed in.

- This is good because the original data is not touched by the function if something goes wrong, and bad because sometimes copying the function parameters is expensive in terms of memory space, time and money. And sometimes you really do need to change the data that is passed in.
- Formal vs. Actual parameters:
  - Formal parameters: the parameters defined in the function header.
  - Actual parameters (arguments): the parameters used in the function call.
- In C++ the actual parameters are passed by value or copied to the formal parameters.

```cpp
#include <iostream>
using namespace std;

void param_test(int formal) { // formal is a copy if actual.
    cout << formal << endl; // 50
    formal = 100; // only changes the local (formal) copy.
    cout << formal << endl; // 100.
}

int main() {
    int actual {50};
    cout << actual << endl; // 50.
    param_test(actual); // pass in 50 to param_test.
    cout << actual << endl; // actual (50) did not change.
    return 0;
}
/* OUTPUT:
50
50
100
50

*/
```

11.4.2. Function return statement

- If a function returns a value of a specific type then it must use a `return` statement that returns a value.
- If a function does not return a value (`void`) then the `return` statement is optional.
- `return` statement can occur anywhere in the body of the function, but it is usually used in the end of the function.
- `return` statement immediately exits the function.
- We can have multiple `return` statements in a function:
  - Avoid many return statements in a function.
- The return value is the result of the function call.

11.5. Default arguments

- When a function is called, all arguments must be supplied.
- Sometimes some of the arguments have the same values most of the time.

- We can tell the compiler to use default values if the arguments are not supplied.
- Default values can be in the prototype or definition, not both:
  - Best practice is to have them in the prototype.
  - Must appear at the tail end of the parameter list.
- Can have multiple default values.
  - Must appear consecutively at the tail end of the parameter list.

11.5.1. Example

- Example:

```cpp
#include <iostream>
using namespace std;

double calc_cost(double base_cost, double tax_rate = 0.06);

double calc_cost(double base_cost, double tax_rate) {
    return base_cost += (base_cost * tax_rate);
}

int main() {
    double cost {0};
    cost = calc_cost(200.0); // will use the default tax.
    cout << cost << endl;
    cost = calc_cost(100.0, 0.08); // will use 0.08 not the default
    cout << cost << endl;
    return 0;
}
/* OUTPUT:
212
108

*/
```

11.6. Overloading functions

- Sometimes we want to use a function to perform operations in several data types, such as the print function would need to admit any type of parameter.
- We can have functions that have different parameter lists that have the same name.
- This is a great example of abstraction, since we just think of “print” and not worry about the data type being passed in.
- This is a type of polymorphism.
  - We can have the same name work with different data types to execute similar behavior.
- The compiler must be able to tell the functions apart based on the parameter list and argument supplied.
- All overloaded functions must be implemented separately, this could be solved with templates.

- If the function don't have any parameters and just vary on the return type a compiler error will be produced.
- The ease of use of overloading functions is just the abstraction, you can just think of a concept and not have to worry in which data type you implemented the function in, the compiler will deduce that for you.
- Be careful when using default values in more than one overloaded function, the compiler will not know which one to use, and this can cause a compiler error.
- Consider the following:

```cpp
#include <iostream>
using namespace std;

int add_numbers(int a, int b);
double add_numbers(double a, double b);

double add_numbers(double a, double b) {
    return a + b;
}
int add_numbers(int a, int b) {
    return a + b;
}

int main() {
    cout << add_numbers(10,20) << endl;
    cout << add_numbers(10.1,67.9) << endl;
    return 0;
}
/* OUTPUT:
30
78

*/
```

11.6.1. Example

```cpp
#include <iostream>
#include <vector>
using namespace std;

void print(int);
void print(double);
void print(string);
void print(string, string);
void print(vector<string>);

void print(int num) {
    cout << "int:" << num << endl;
}
void print(double num) {
    cout << "double: " << num << endl;
}
void print(string str) {
```

```
    cout << "string: " << str << endl;
}
void print(string str1, string str2) {
    cout << "str1: " << str1 << ", str2: " << str2 << endl;
}
void print(vector<string> vec) {
    for (auto v : vec) {
        cout << v << endl;
    }
}

int main() {
    print(100); // int -> print(int)
    print('A'); // char -> print(int) char is promoted to int.
    print(123.5); // double -> print(double)
    print(123.3F); // float -> print(double) float is promoted to double.
    print("C-style string"); // C-style string is converted to a C++ string.
    vector<string> vec {"ABC", "DEF", "GHI"};
    print(vec);
    return 0;
}
/* OUTPUT:
int:100
int:65
double: 123.5
double: 123.3
string: C-style string
ABC
DEF
GHI

*/
```

11.7. Passing arrays to functions

- We can pass an array to a function by providing square brackets in the formal parameter description.

```
void print_array(int numbers[]);
```

- The array elements are NOT copied, the function does not know how many times to iterate through the array because what is passed in is an address to the array not a copy of the array.
- Since the array name evaluates to the location of the array in memory, this address is what is copied.
- So the function has no idea how many elements are in the array since all it knows is the location of the first element (the name of the array).
- So when we pass arrays to functions we must also pass in the size of the array in order to know how much time to iterate.
- Example:

```
#include <iostream>
using namespace std;
```

```
3
4  void print_array(int numbers[], size_t size);
5
6  int main() {
7      int my_numbers[] {1,2,3,4,5};
8      print_array(my_numbers, 5);
9      return 0;
10 }
11
12 void print_array(int numbers[], size_t size) {
13     for (size_t i{0}; i < size; ++i) {
14         cout << numbers[i] << endl;
15     }
16 }
17 /* OUTPUT:
18 1
19 2
20 3
21 4
22 5
23
24 */
```

- In this case since the array is not copied but instead C++ works with the original array, we sometimes don't want to risk changing or altering the array passed in. Since we are passing the location of the array the function can modify the actual array.
    - For this we can tell the compiler that function parameters are `const` (read-only).
    - This could me useful in the `print_array` function since if should NOT modify the array.

```
1  void print_array(const int numbers[], size_t size) {
2      for (size_t i{0}; i < size; ++i) {
3          cout << numbers[i] << endl;
4      }
5  }
```

11.8. Pass by reference

- Sometimes we want to be able to change the actual parameter from within the function body.
- In order to achieve this we need the location or address of the actual parameter.
- We say how this is the effect with array, but what about other variable types?
- We can use reference parameters to tell the compiler to pass in a reference to the actual parameter.
- The formal parameter will now be an alias for the actual parameter.
- Syntax is: `return_type func_name(type &var_name);`
- Example:

```
1  #include <iostream>
2  using namespace std;
3
```

```cpp
void scale_number(int &num);

int main() {
    int number {1000};
    scale_number(number);
    cout << number << endl;
    return 0;
}

void scale_number(int &num) {
    if (num > 100) {
        num = 100;
    }
}

/* OUTPUT:
100

*/
```

- Example:

```cpp
#include <iostream>
using namespace std;

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x{10}, y{20};
    cout << x << " " << y << endl;
    swap(x,y);
    cout << x << " " << y << endl;
    return 0;
}
/* OUTPUT:
10 20
20 10

*/
```

- In the following we intend to pass a vector by value in to the function print which would mean that it will make a copy of all that data inside the function. We could better pass the location of the vector into the function and modify the original data:
    - The issue now would be that we could pass by reference and risk modifying the vector in an unintended way, for this we can make the data being passed in constant.
    - This is the best of both worlds because we are passing the reference (which is more efficient) and at the same time ensuring no changes are made to the vector.

```cpp
#include <iostream>
#include <vector>
```

```cpp
using namespace std;

void print(const vector<int> &v) {
    for (auto num : v) {
        cout << num << endl;
    }
}

int main() {
    vector<int> data {1,2,3,4};
    print(data);
    return 0;
}
/* OUTPUT:
1
2
3
4

*/
```

11.9. Scope rules

- C++ uses scope rules to determine where an identifier can be used.
- C++ uses static or lexical scoping.
- C++ has two main scopes: Local or block scope:
  - Local or block scopes:
    - Identifiers declared in a block {}.
    - Function parameters have block scope.
    - Only visible within the block {} where declared.
    - Function local variables are only active while the function is executing.
    - Local variables are not preserved between function calls.
    - With nested blocks inner blocks can 'see' but outer blocks cannot.
    - When a function is called you can think of the function as being activated and the function parameters are bound to storage. The parameters become alive and the their lifetime is the time the function executes, once the function completes the function is deactivated and these variables and parameters are no longer alive.
- Static local variables:
  - There is one kind of variable whose value is preserved between many function calls.

```cpp
static int value {10};
```

  - It is a variable whose lifetime is the lifetime of the program.
  - It is only visible to the statements in the function body.
  - These variables can be very helpful when you need some values to be preserved in a function without having to pass them in each time.
  - Static local variables are only initialized once, if no initialization is provided they are set to 0.
- Identifiers with global scope.

- Identifiers declared outside any function or class.
- Visible to all parts of the program after the global identifier has been declared.
- Global constants are OK.
- Best practice is not to use global variables.

- You can create a new scope with the {}.

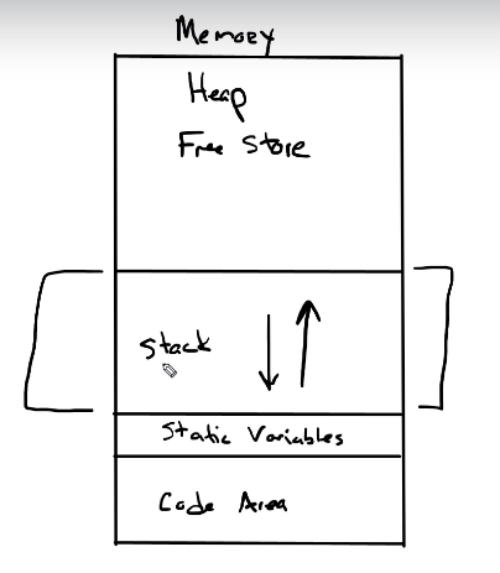
```cpp
#include <iostream>
using namespace std;
int main() {
    int num1 {100}; // local to main.
    int num2 {500}; // local to main.
    cout << "num1: " << num1 << " {" << endl;
    {
        int num1 {200};
        cout << "\tnum1: " << num1 << endl;
        cout << "\tnum2: " << num2 << endl;
    }
    cout << '}' << endl;
    return 0;
}
/* OUTPUT:
num1: 100 {
        num1: 200
        num2: 500
}

*/
```

- One particular block can see outwardly while the out most blocks cannot see in to the most nested blocks.

11.10. How do function calls work?

- Functions use an area in memory called the function call stack:
  - Analogous to a stack of books.
  - Operations on a stack push and pop.
  - In a program a push element in to the call stack is called a stack frame.
- Stack frame or activation record:
  - Functions must return control to function that called it.
  - Each time a function is called we create an new activation record and push it on to the stack.
  - When a function terminates we pop the activation record and return.
  - Local variables and function parameters are located on the stack.
- The stack size if finite:
  - If you call too many functions you might run out of stack space producing a stack overflow error which will crash your program.

- Illustration:



11.11. Inline functions

- As we've seen, a function requires space in the call stack, the creation of a stack frame and the deletion of the stack frame once the function is done.
- Sometimes the function is so simple the process a function has to undergo to become a function in memory is ineficient and not worth the code.
- Function calls have certaing amount of overhead.
- You saw what happens on the call stack.
- Sometimes we have simple functions.
- We can suggest to the compiler to compile them 'inline':
  - Avoid function call overhead.
  - Generate inline assembly code.
  - Faster.
  - Could cause code bloat.
- Compilers optimizations are very sophisticated:
  - Will likely make simple functions inline even without your suggestion.
- Syntax is:

```
inline int add_numbers(int a, int b) {
    return a + b;
}
int main() {
    int result{0};
    result = add_numbers(100,200);
    return 0;
}
```

11.12. Recursive functions

- A recursive function is a function that calls itself.
  - Either directly or indirectly through another function.
- Recursive problem-solving:
  - Base case.
  - Divide the rest of the problem into subproblems and do a recursive call.
- There are many problems that lend themselves to recursive solutions.
- Mathematics: factorial, Fibbonacci, fractals.
- Searching and sorting: binary search, search trees, ...
- Example of Factorial:

$$n! = n \times (n - 1);$$

```cpp
#include <iostream>
using namespace std;
unsigned long long factorial(unsigned long long n) {
    if (n == 0) {
        return 1; // base case.
    }
    return n * factorial(n-1); // recursive call.
}
int main() {
    cout << factorial(8) << endl;
    return 0;
}
/* OUTPUT:
40320

*/
```

- Important notes:
  - If recursion doesn't eventually stop you will have infinite recursion.
  - Recursion can be resource intensive (take up too much memory).
  - Remember the base case(s):
    - It terminates the recursion.
  - Only use recursive solutions when it makes sense.
  - Anything that can be done recursively can be done iteratively.
    - Stack overflow error.

Capítulo 12

Pointers and references

12.1. What is a pointer?

- A variable.
  - Whose value is an address.
- What can be at that address?
  - Another variable.
  - A function.
- Pointers have a memory location that is bound to, type and has a value, this value is address in memory.
- If x is an integer variable and its value is 10 then I can declare a pointer that points to it.
- To use the data that the pointer is pointing to you must know its type.

12.1.1. Why use pointers?

- Can't I just use the variable or the function itself?
  - Yes, but you can't always do that.
- Inside functions, pointers can be used to access data that are defined outside the function. Those values may not be in scope so you can't access them by their name.
- Pointers can be used to operate on arrays very efficiently.
- We can allocate memory dynamically on the heap or free store.
  - This memory doesn't even have a variable name.
  - The only way to get to it is via a pointer.
- With object-oriented programming, pointers are how polymorphism works.
- Can access specific addresses in memory.
  - Useful in embedded and system applications.

12.2. Declaring pointer variables

- We declare pointer variables in very much the same way any other variable would, except an asterisk must be between the type and the identifier:

```
variable_type *pointer_name;
```

```
int *int_ptr; // pointer to int.
double *double_ptr; // pointer to double.
char *char_ptr; // pointer char.
std::string *string_ptr; // pointer to a std::string.
```

  - Keep in mind that there are many conventions to declaration of pointers, such as the one where the asterisk is next to the type `int* name;`, others where the asterisk is previous to the name `int *name;` or even a space from each `int * name;`

- Intializing pointer variables to "point no where" or null:
  - If you don't initialize your pointers they will have garbage data.
  - In this case that data will be addresses.
  - Just as an int was as convention initialized to a value, say 0, a pointer is usually initialized to null which means it doesn't point to any memory location.
  - You do this with the `nullptr` or the `NULL` keywords.

```
variable_type *pointer_name {nullptr};
```

```
int *int_ptr {};
double *double_ptr {nullptr};
char *char_ptr {nullptr};
string *string_ptr {nullptr};
```

- Always initialize variable pointers to 'point no where 'or null.
  - Always initialize.
  - Uninitialized pointers contain garbage data and can 'point anywhere' in memory.
  - Initializing to zero or `nullptr` (C++11) represents address zero.
    - Implies that the pointer is 'pointing nowhere'.
  - If you don't initialize a pointer to point to a variable or function then you should initialize it to `nullptr` to 'make it null'.

12.3. Accessing the pointer address and storing address in a pointer

- The address operator.
  - Variables are stored in unique addresses.
  - Unary operator.
  - Evaluates to the address of its operand.
    - Operand cannot be a constant or expression that evaluates to temp values.

```cpp
#include <iostream>
using namespace std;
int main() {
    int num{10};
    cout << "Value of num is: " << num << endl;
    cout << "Size of num is: " << sizeof num << endl;
    cout << "Address of num is: " << &num << endl;
    return 0;
}
/* OUTPUT:
Value of num is: 10
Size of num is: 4
Address of num is: 0x61fe1c
*/
```

- Example:

```cpp
#include <iostream>
using namespace std;
int main() {
    int *p;
    cout << "Value of p is: " << p << endl; // garbage data.
    cout << "Address of p is: " << &p << endl; // value of p.
    cout << "Size of p is: " << sizeof p << endl; // size of p.
    p = nullptr;
    cout << "Value of p is: " << p << endl;
    return 0;
}
/* OUTPUT:
Value of p is: 0x10
Address of p is: 0x61fe18
Size of p is: 8
Value of p is: 0

*/
```

12.3.1. sizeof a pointer variable

- Don't confuse the size of a pointer variable and the size of what it points to.
- All pointers in a program have the same size.
- They may be pointing to a very large or very small types.

```cpp
int *p1 {nullptr};
double *p2 {nullptr};
unsigned long long *p3 {nullptr};
vector<string> *p4 {nullptr};
string *p5 {nullptr};
```

12.3.2. Storing an address in a pointer variable?

Typed pointers.

- The compiler will make sure that the address stored in a pointer variable is of the correct type.

```
int score {10};
double high_temp {100.7};
int *score_ptr {nullptr};
score_ptr = &score;
score_ptr = &high_temp; // compiler error because high_temp is a double and score_ptr
↪  is an int.
```

12.3.3. & the address of operator

- Pointers are variables so they can change.
- Pointers can be null.
- Pointers can be uninitialized.

12.4. Dereferencing the pointer

- Access the data we're pointing to is called dereferencing a pointer.
- If `score_ptr` is a pointer and has a valid address.
- Then you can access the data at the address contained in the `score_ptr` using the dereferencing operator.

```
#include <iostream>
using namespace std;
int main() {
    int score {100};
    int *score_ptr {&score};
    cout << *score_ptr << endl; // 100
    *score_ptr = 200;
    cout << *score_ptr << endl;
    cout << score << endl;
    return 0;
}
/* OUTPUT:
100
200
200
*/
```

  - Declaring and dereferencing is done using the asterisk, C++ has received some criticism about this, but once you understand where and how to use the asterisk you'll be fine.

- Example:

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<string> stooges {"Larry", "Moe", "Curly"};
    vector<string> *vector_ptr {nullptr};
    vector_ptr = &stooges;
```

```
    cout << "First stooge: " << (*vector_ptr).at(0) << endl;
    cout << "Stooges: ";
    for (auto stooge: *vector_ptr) {
        cout << stooge << " ";
    }
    cout << endl;
    return 0;
}
/* OUTPUT:
First stooge: Larry
Stooges: Larry Moe Curly
*/
```

12.5. Dynamic Memory Allocation

- Allocating storage from the heap at runtime.
- We often don't knot how much storage we need until we need it.
- We can allocate storage for a variable at run time.
- Recall C++ arrays:
  - We had to explicitly provide the size and it was fixed.
  - But vectors grow and shrink dynamically.
- We can use pointers to access newly allocated heap storage.

12.5.1. Allocating and deallocating memory

- The `new` keyword
  - Using the new keyword to allocate storage.

```
#include <iostream>
using namespace std;
int main() {
    int *int_ptr {nullptr};
    int_ptr = new int; // allocate an integer on the heap.
    cout << int_ptr << endl; // address of int_ptr.
    cout << *int_ptr << endl; // garbage. Notice the dereferencing.
    *int_ptr = 100;
    cout << *int_ptr << endl; // 100.
    return 0;
}
/* OUTPUT:
0x25824f0
39331936
100
*/
```

  - If you loose the pointer because it goes out of scope or other such incidents, that is called a memory leak and you lost the only way you have to access that memory.
  - You also must deallocate the pointer after you are done using it.
- The `delete` keyword

- The delete keyword is used to deallocate allocated space.

```
int *int_ptr {nullptr}; // allocate an integer on the heap.
delete int_ptr; // frees the allocated storage.
```

- The `new[]` keyword
  - The `new[]` is used to allocate an array.

```
int *array_ptr {nullptr};
int size {};
cout << "How big do you want the array: ";
cin >> size;
array_ptr = new int[size]; // allocate array on the heap.
```

  - Keep in mind that these brackets must be empty.
- The `delete[]` keyword is used to deallocate storage of an array.

```
int *array_ptr {nullptr};
int size {};
cout << "How big do you want the array: ";
cin >> size;
array_ptr = new int[size]; // allocate array on the heap.
delete[] array_ptr; // free allocated storage.
```

  - Keep in mind that these brackets must be empty.
- When you allocate dynamically you are allocating into the heap or the free store, the stack houses the pointer to the dynamically allocated data.
  - Example:

```
#include <iostream>
using namespace std;
int main() {
    size_t size{0}; // allocated on the stack.
    double *temp_ptr {nullptr}; // allocated on the stack.
    cout << "How many temps: ";
    cin >> size;
    temp_ptr = new double[size]; // allocated on the heap.
    cout << temp_ptr << endl;
    delete[] temp_ptr; // dealocated on the heap.
    return 0;
}
```

12.6. Relationship between arrays and pointers

- The value of an array name is the address of the first element in the array.
- The value of a pointer variable is an address.
- If the pointer points to the same data types as the array element then the pointer and array name can be used interchangeably (almost).

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,95,89};
    cout << scores << endl;
    cout << *scores << endl;
    int *score_ptr {scores};
    cout << score_ptr << endl;
    cout << *score_ptr << endl;
    return 0;
}
/* OUTPUT:
0x61fe0c
100
0x61fe0c
100
*/
```

- We can also use array subscripting on a pointer using the square brackets operator.

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,95,89};
    int *score_ptr {scores};
    cout << score_ptr[0] << endl;
    cout << score_ptr[1] << endl;
    cout << score_ptr[2] << endl;
    return 0;
}
/* OUTPUT:
100
95
89
*/
```

- You can perform pointer arithmetic, which is adding numbers to a pointer:

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,95,89};
    int *score_ptr {scores};
    cout << score_ptr << endl;
    cout << (score_ptr + 1) << endl;
    cout << (score_ptr + 2) << endl;
    return 0;
}
/* OUTPUT:
0x61fe0c
0x61fe10
0x61fe14
*/
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,95,89};
    int *score_ptr {scores};
    cout << *score_ptr << endl;
    cout << *(score_ptr + 1) << endl;
    cout << *(score_ptr + 2) << endl;
    return 0;
}
/* OUTPUT:
100
95
89
*/
```

12.6.1. Subscript and offset notation equivalence

- You can write this in two ways:

```cpp
int array_name[] {1,2,3,4,5};
int *pointer_name {array_name};
```

- Subscript notation:

```cpp
array_name[index];
pointer_name[index];
```

- Offset notation:

```cpp
*(array_name + index);
*(pointer_name + index);
```

12.7. Pointer arithmetic

- Pointers can be used in:
  - Assignment expressions.
  - Arithmetic expressions.
  - Comparison expressions.
- C++ allows pointer arithmetic.
- Pointer arithmetic only makes sense with raw arrays.

12.7.1. ++ and −

- ++ increments a pointer to point to the next array element.

```cpp
int_ptr++;
```

- − decrements a pointer to point to the previous array element.

```
int_ptr--;
```

- Keep in mind that in pointer arithmetic, adding one means adding whatever number of bytes the elements occupy in order to get to the next element.

12.7.2. + and -

- + increment pointer by some number: (`n * sizeof(type)`)

```
int_ptr += n; or int_ptr = int_ptr + n;
```

- - decrement pointer by some number: (`n * sizeof(type)`)

```
int_ptr -= n; or int_ptr = int_ptr - n;
```

12.7.3. Subtracting two pointers

- Determine the number of elements between the pointers.
- both pointers must point to the same data type:

```
int n = int_ptr2 - int_ptr1;
```

12.7.4. Compare two pointers == and !=

- Determine if two pointers point to the same location.
    - Does not compare the data where they point.

```
#include <iostream>
using namespace std;
int main() {
    string s1 {"Frank"};
    string s2 {"Frank"};
    string *p1 {&s1};
    string *p2 {&s2};
    string *p3 {&s1};
    cout << boolalpha;
    cout << (p1 == p2) << endl;
    cout << (p1 == p3) << endl;
    return 0;
}
/* OUTPUT:
false
true
*/
```

12.7.5. Comparing the data pointers point to

- Determine if two pointers point to the same data.
- You must compare the referenced pointers.

```cpp
#include <iostream>
using namespace std;
int main() {
    string s1 {"Frank"};
    string s2 {"Frank"};
    string *p1 {&s1};
    string *p2 {&s2};
    string *p3 {&s1};
    cout << boolalpha;
    cout << (*p1 == *p2) << endl;
    cout << (*p1 == *p3) << endl;
    return 0;
}
/* OUTPUT:
false
true
*/
```

12.7.6. Examples

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[] {100,95,89,68,-1};
    int *score_ptr {scores};
    while (*score_ptr != -1) {
        cout << *score_ptr << endl;
        score_ptr++;
    }
    return 0;
}
/* OUTPUT:
100
95
89
68
*/
```

```cpp
#include <iostream>
using namespace std;
int main() {
    char name[] {"Frank"};
    char *char_ptr1 = &name[0];
    char *char_ptr2 = &name[3];
    cout << "In the string " << name << ", " << *char_ptr2 << " is " << (char_ptr2 -
    ↪ char_ptr1) << " characters away from " << *char_ptr1 << endl;
    return 0;
}
/* OUTPUT:
In the string Frank, n is 3 characters away from F
*/
```

12.8. Passing pointers to a function

const and pointers:

- There are several ways to qualify pointers using const.
  - Pointers to constants.
  - Constant pointers.
  - Constant pointers to constants.
- Pointers to constants:
  - The data pointed to by the pointers is constant and cannot be changed.
  - The pointer itself can change and point somewhere else.
  - Pointers to constants follow the syntax: `const type *var_name`.

```
int high_score {100};
int low_score {65};
const int *score_ptr {&high_score};
*score_ptr = 86; // error, changing the data being pointed to.
score_ptr = &low_score; // Ok.
```

- Constant pointers:
  - The data pointed to by the pointers can be changed.
  - The pointer itself cannot change and point somewhere else.
  - Const pointers follow the syntax: `type const *var_name`.

```
int high_score {100};
int low_score {65};
int const *score_ptr {&high_score};
*score_ptr = 86; // Ok.
score_ptr = &low_score; // Error, changing the pointer.
```

12.9. Passing pointers to functions

- Pass-by-reference with pointer parameters.
- We can use pointers and the dereference operator to achieve pass-by-reference.
- The function parameter is a pointer.
- The actual parameter can be a pointer of address of a variable.
- Example:

```
#include <iostream>
using namespace std;

void double_data(int *int_ptr);

void double_data(int *int_ptr) {
    *int_ptr *= 2;
}
```

```cpp

int main() {
    int value {10};
    cout << value << endl; // 10
    double_data(&value);
    cout << value << endl; // 20
    return 0;
}
/* OUTPUT:
10
20
*/
```

- Example swap two variables:

```cpp
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int temp {*a};
    *a = *b;
    *b = temp;
}

int main() {
    int x {100}, y {200};
    cout << "x: " << x << endl;
    cout << "y: " << y << endl;
    swap(&x, &y);
    cout << "x: " << x << endl;
    cout << "y: " << y << endl;
    return 0;
}
/* OUTPUT:
x: 100
y: 200
x: 200
y: 100
*/
```

- Example of passing pointer to vector:

```cpp
#include <iostream>
#include <vector>
using namespace std;

void display(const vector<string> const *v) { // the pointer is constant and the
↪  data is constant.
    for (auto str : *v) {
        cout << str << ' ';
    }
    cout << endl;
}

```

```
int main() {
    vector<string> stooges {"Larry", "Moe", "Curly"};
    display(&stooges);
    return 0;
}
/* OUTPUT:
Larry Moe Curly
*/
```

- Example or updating pointers:

```
#include <iostream>
using namespace std;

void display(int *array, int sentinel) {
    while (*array != sentinel) {
        cout << *array++ << endl; // dereference the pointer and print it, then increment the pointer.
    }
    cout << endl;
}

int main() {
    int scores[] {100,98,97,79,85,-1};
    display(scores, -1);
    return 0;
}
/* OUTPUT:

*/
```

12.10. Returning a pointer from a function

- In C++ functions can return pointers.

```
type *function();
```

- Should return pointers to:
  - Memory dynamically allocated in the function.
  - To data that was passed in.
- Never return a pointer to a local function variable.
- Example, return the pointer to the largest int:

```
#include <iostream>
using namespace std;

int *largest_int(int *int_ptr1, int *int_ptr2) {
    if (*int_ptr1 > *int_ptr2) {
        return int_ptr1;
    } else {
```

```
        return int_ptr2;
    }
}

int main() {
    int a {100}, b {200};
    int *largest_ptr {nullptr};
    largest_ptr = largest_int(&a, &b);
    cout << *largest_ptr << endl; // 200
    return 0;
}
/* OUTPUT:
200
*/
```

- Example, returning dynamically allocated memory:

```
#include <iostream>
using namespace std;

int *create_array(size_t size, int init_value = 0) {
    int *new_storage {nullptr};
    new_storage = new int[size];
    for (size_t i {0}; i < size; ++i ) {
        *(new_storage + i) = init_value;
    }
    return new_storage;
}

int main() {
    int *my_array;
    my_array = create_array(100,200);
    delete[] my_array;
    return 0;
}
/* OUTPUT:
*/
```

- Never return a pointer to a local variable:

```
int *dont_do_this() {
    int size {};
    return &size;
}

int *or_this() {
    int size {};
    int *int_ptr {&size};
    return int_ptr;
}
```

    - Notice this will compile just fine, since the address returned is the address of an integer, however, all variables are deleted when they go out of scope and you return a pointer to a deleted memory location.

12.11. Potential pointer pitfalls

- Uninitialized pointers.
  - Contain garbage, the pointer might be pointing to a very important area in memory and cause serious crashes even in the OS.
  - Sometimes makes debugging much harder because sometimes your program does not crash and when you add somethin it unexpectedly crashes and you think it is because of the new change but its actually a bug that has been in the code for a long time.

```
int *int_ptr; // pointing anywhere.
int *int_ptr = 100; // Hopefully a crash.
```

- Dangling pointers.
  - Pointers that are pointing to memory that is no longer valid or released memory:
    - For example: 2 pointers point to the same data.
    - 1 pointer releases the data with delete.
    - The other pointer accesses the release data.
    - This causes undefined behaviour.
  - Pointer that points to memory that is invalid:
    - We saw this when we returned a pointer to a function's local variable.
- Not checing if new failed to allocate memory.
  - If `new` fails an exception is thrown.
  - We can use exception handling to catch exceptions.
  - Dereferencing a null pointer will cause your program to crash.
  - This is good in testing but bad in production.
- Leaking memory.
  - Forgetting to release dynamically allocated memory with delete.
  - If you lose your pointer to the storage allocated on the heap you have not way to get to that storage again.
  - The memory is orphaned or leaked.
  - One of the most common pointer problems.

12.12. What is a reference?

- An alias for a variable, not a pointer.
- Must be initialized to a variable when declared.
- Cannot be null.
- Once initialized cannot be made to refer to a different variable.
- Very useful as function parameters.
- Might be helpful to think of a reference as a constant pointer that is automatically dereferenced.

12.12.1. Using references in range-based for loops

- In the next example, we iterate a vector but are unable to change the data.

```cpp
#include <iostream>
#include <vector>
using std::vector;
using std::string;
using std::cout;
using std::endl;

int main() {
    vector<string> stooges {"Larry", "Moe", "Curly"};
    for (auto str: stooges) {
        str = "Funny"; // changes the copy.
    }
    for (auto str: stooges) {
        cout << str << endl; // Larry, Curly, Moe
    }
}

/* OUTPUT:
Larry
Moe
Curly
*/
```

- Using references we can change the data.

```cpp
#include <iostream>
#include <vector>
using std::vector;
using std::string;
using std::cout;
using std::endl;

int main() {
    vector<string> stooges {"Larry", "Moe", "Curly"};
    for (auto &str: stooges) {
        str = "Funny"; // changes the actual.
    }
    for (auto str: stooges) {
        cout << str << endl; // Funny, Funny, Funny
    }
}
/* OUTPUT:
Funny
Funny
Funny
*/
```

- Be careful using the `const` qualifier in range based for loops if you plan to change the data.

```cpp
vector<string> stooges {"Larry", "Moe", "Curly"};
for (auto const &str: stooges) {
```

```
    str = "Funny"; // compiler error.
}
```

- It makes sense to use const if you don't want to change the data, using references you do not copy the elements, just the elements.

```
vector<string> stooges {"Larry", "Moe", "Curly"};
for (auto const &str: stooges) {
    cout << str << endl; // Larry, Moe, Curly
}
```

12.12.2. Examples

- Example: references are aliases to a variable.

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int num {100};
    int &ref {num};
    cout << num << endl;
    cout << ref << endl;

    num = 200;
    cout << num << endl;
    cout << ref << endl;

    ref = 300;
    cout << num << endl;
    cout << ref << endl;
}
/* OUTPUT:
100
100
200
200
300
300
*/
```

- References in range based for loops:

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<string> stooges {"Larry", "Moe", "Curly"};
    for (auto str: stooges) {
        str = "Funny"; // changing a copy.
    }
    for (auto str: stooges) {
        cout << str << " ";
    }
```

```
    cout << endl;
    for (auto &str: stooges) {
        str = "Funny"; // changing the actual.
    }
    for (auto str: stooges) {
        cout << str << " ";
    }
    cout << endl;
    return 0;
}
/* OUTPUT:
Larry Moe Curly
Funny Funny Funny
*/
```

12.13. L-values and R-values

12.13.1. L-values

- Values that have names and are addressable.
- Modifiable if they are not constants.

```
int x {100}; // x is an l-value.
x = 1000;
x = 1000 + 20;
string name; //name is an l-value.
name = "Frank";
```

- l-value:
  - Values that have names and are addressable.
  - Modifiable if they are not constants.

```
100 = x; // 100 is not an r-value.
(1000+20) = x; // (1000 + 20) is an l-value.
```

- r-values:
  - A value that is not an l-value. We can define it like this using exclusion.
  - On the right hand side of an assignment expression.
  - A literal.
  - A temporary which is intended to be non-modifiable.

```
string name;
name = "Frank";
"Frank" = name; // "Frank" is an r-value.
int max_num = max(20,30); // max(20,30) is an r-value.
```

  - r-values can be assigned to l-values explicitly.

```
int x {100};
int y {0};
y = 100; // r-value 100 assigned to l-value y.
x = x + y; // r-value (x-y) assigned to l-value x.
```

- References from the perpective of l and r values:
  - The references we've used are l-value references.
  - Because we are referencing l-values.

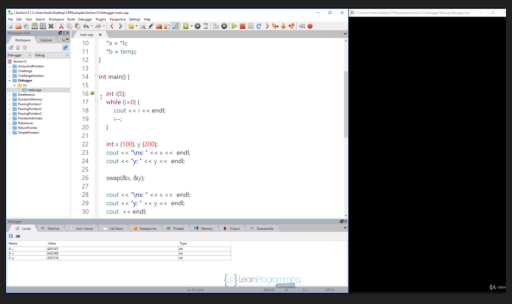
```
int x {100};
int &ref1 = x; // ref1 is reference to l-value.
ref1 = 1000;
int &ref2 = 100; // error: 100 is an r-value.
```

- The same is true when we pass-by-reference to a function:

```
int square(int &n) {
    return n*n;
}
int num {10};
square(num); // ok.
square(5); // error: can't reference r-value 5.
```

12.14. Using the debugger

- Using F5 you will compile and run your program, your program will stop when it encounters a break point.
- On the break point you can see the local variables that exist, the functions as well as the locations of these elements.



- Step in: if the line is a function it allows us execute a break point inside the function.
- Step out: if you've pressed step in, steping out will continue where you left off.
- Continue/Next: will go to the next line.
- Watches: alloc you to check for conditions.

12.15. Section recap

- When to use pointers vs. reference parameters:
  - Pass by value.
    - When the function does not modify the actual parameter, and the parameter is small and efficient to copy like simple types (int, char, double, etc.)
  - Pass by reference using a pointer:
    - When the function does moddify the actual parameter, and the parameter is expensive to copy.
    - Its OK for a pointer to be null, references cannot be null.
  - Pass by reference usign a pointer to const:
    - Pass by reference using a pointer to `const`
      - When the function does not modify the actual parameter, and the parameter is expensive to copy, and its ok for the pointer to be a null value.
  - Pass by reference using a const pointer to const:
    - When the fucntion does not modify the actual parameter, adn the parameter is expensive to copy, and its OK for the pointer to be a null value, you don't want to modify the pointer itself.
  - Pass by reference using a reference:
    - When the function does modify the actual parameter, and the parameter is expensive to copy, and the parameter will never be a null value.
  - Pass by reference using a const reference:
    - When the function does not modify the actual parameter, and the parameter is expensive to copy, and the parameter will never be a null value.

Capítulo 13

OPP: Classes and objects

13.1. What is object-oriented programming?

13.1.1. Procedural programming and its limitations:

- Procedural programming is pretty much what we've been doing throughout the course.
- Focus is on processes or actions that a program takes.
- Programs are typically a collection of functions.
- Data is declared separately.
- Data is passed as arguments to into functions.
- Fairly easy to learn.

Functions need to know the structure of the data.

- If the structure of the data changes many functions must be changed.
- Imagine a program with thousands of functions that all depend on a data structure being passed in to it, imagine that that data structure needs to change you would need to change every function, functions might even need to be modified to handle such change and a grave impact is in order, then test all the functions which makes development much more expensive.

As programs get larger they become more:

- Difficult to understand.
- Difficult to maintain.
- Difficult to extend.
- Difficult to debug.
- Difficult to reuse code.
- Fragile and easier to break.

13.1.2. What is objected-oriented programming:

- Object oriented denotes many things such as: OO analysis, OO design, OO testing, etcetera.
- Object-oriented is all about modeling you software in terms of objects and classes.
- Classes and objects:
  - Focus is on classes that model real-world domain entities.
  - Allows developers to think at a higher level of abstraction.
  - Used successfully in very large programs.

Why is it such a big deal?

- As our programs grow more and more complex we need ways to deal with complexity, classes and objects are one way to do just that, rather than thinking of details we can think of just that object.

Encapsulation:

- Objects contain data AND operations that work on that data.
- Abstract Data Type (ADT).

Information hiding:

- OO allows us to hide implementation specific logic in a class that is available only within the class, now we know that the user can mess with the user implementation specific code since they never knew about it in the first place.
- Users of the class code to the interface since they don't need to know the implementation.
- More abstraction.
- Easier to test, debug, maintain and extend.

Reusability:

- Easier to reuse in other applications.
- Faster development.
- Higher quality.

Inheritance:

- Can create new classes in terms of existing classes.
- Reusability.
- Polymorphic classes.

13.1.3. OO limitations

- Not a panacea:
  - OO programming won't make bad code better.
  - Not suitable for all types of problems.
  - Not everything decomposes to a class.
- Learning curve:
  - Usually a steeper learning curve, especially in C++.

    - Many OO languages, many variations of OO concepts.
- Design:
    - Usually more up-front design is necessary to create good models and hierarchies.
- Programs can be:
    - Larger in size.
    - Slower.
    - More complex.

13.2. What are classes and objects?

13.2.1. Classes

- Blueprints from which objects are created.
- A user-defined data-type.
- Has attributes (data).
- Has methods (functions).
- Can hide data and methods.
- Provides a public interfase.

Example classes:

- Account.
- Employee.
- Image.
- `std::vector`.
- `std::string`.

13.2.2. Objects

- Objects are created from classes.
- Represent a specific instantce of a class.
- Can create many, many objects.
- Each has its own identity.
- Each can use the defined class methods.

Examples of objects:

- Frank's account is an instance of an account.
- Jim's account is an instance of an account.
- Each has its own balance, can make deposits, withdrawls, etc.

13.2.3. Example

- int are not classes but an object is declared in very much the same way any variable is.

```
int high_score;
int low_score;
Account frank_account; // user defined type.
Account jim_account;
std::vector<int> scores;
std::string name;
```

13.3. Declaring a class and creating objects

```
#include <iostream>
class Player {
    // attributes
    std::string name;
    int health;
    int xp;

    // methods
    void talk(std::string text_to_say);
    bool is_dead();
};
int main() {
    // Now that we have declared the class, we can create objects or instances of that
    ↪ class.
    Player *enemy = new Player();
    delete enemy;
    return 0;
}
```

Take the following example:

```
#include <iostream>
class Account {
    // attributes
    std::string name;
    double balance;
    // methods
    bool withdrawl(double amount);
    bool deposit(double amount);
};
int main() {
    Account frank_account;
    Account jim_account;
    Account *mary_account = new Account();
    delete mary_account;
    return 0;
}
```

13.3.1. Creating objects

- Once we have objects we can use it like any other variable, for example.

```
Account frank_account, jim_account;
std::vector<Account> account1 {frank_account, jim_account};
account1.push_back(jim_account);
Account accounts[] {frank_account};
```

13.3.2. Example

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
class Player { // everything inside here is 'encapsulated'.
    // attributes
    string name;
    unsigned int health;
    int xp;
    // methods prototypes
    void talk(string);
    bool is_dead();
};
class Account {
    string name;
    double balance;

    bool deposit(double);
    bool withdraw(double);
};
int main() {
    Player frank, hero;
    Player * enemy {nullptr};
    enemy = new Player;
    delete enemy;

    Player players[] {frank, hero};

    vector<Player> player_vec;
    player_vec.push_back(hero);

    Account frank_account, jim_account;
    return 0;
}
```

13.4. Accessing class members

- We can access:
    - Class attributes.
    - Class methods.

- Some class members will not be accessible (because of the access modifiers).
- We need an object to access instance variables.

13.4.1. Access members

- If we have an object then we can use the dot operator to access the members.

```
frank_account.balance;
frank_account.deposit(1000.00);
```

- If we have a pointer to an object then we can use the -> operator or dereference and use the dot operator to access the members.

```
    Account *frank_account = new Account();
(*frank_account).balance;
(*frank_account).deposit(1000.00);
```

```
Account *frank_account = new Account();
frank_account->balance;
frank_account->deposit(1000.00);
```

13.4.2. Example

```
#include <iostream>
using namespace std;

class Player {
    public: // allows us to see the attributes and methods from main.
    string name;
    double health;
    unsigned int xp;

    void talk(string text_to_say) {
        cout << name << " says " << text_to_say << endl;
    }
    bool is_dead(); // not implemented, if executed a linker error is going to be
 ↪  thrown.
};

int main() {
    Player frank;
    frank.name = "Frank"; // access the name and change it to "Frank"
    frank.health = 100; // acces the health and change it to 100.
    frank.xp = 12; // access the xp and change it 12.
    frank.talk("Hello");

    Player *enemy = new Player;
    (*enemy).name = "Enemy";
    enemy->name = "Enemy";
    (*enemy).health = 100;
    enemy->health = 100;
    (*enemy).xp = 12;
    enemy->xp = 12;
```

```
    (*enemy).talk("I will destroy you.");
    enemy->talk("I will destroy you.");

    return 0;
}
/* OUTPUT:
Frank says Hello
Enemy says I will destroy you.
Enemy says I will destroy you.
*/
```

13.5. Public and private

- `public:`
  - Accessible everywhere.
- `private:`
  - Accessible only by memebers or friends (we don't know yet) of the class.
- `protected:`
  - Used with inheritance (we don't know that yet.)

13.5.1. Declaration

- Once you declare an access modifier, everything from that point forward will have tha access modifier applied to it.

```
class Class_name {
    public:
    // declaration(s);
};
```

- By default everything in a class will be private if no access modifier is specified.

```
class Class_name {
    private:
    // declaration(s);
};
```

- Protected comes in when using inheritance, its basically the private access modifier for inheritance.

```
class Class_name {
    protected:
    // declaration(s);
};
```

- It is common to combine access modifiers because we want to hide only certain things.

```
class Player {
    private:
    std::string name;
    unsigned int health;
```

```
    int xp;
    public:
    void talk(std::string text_to_say);
    bool is_dead();
};
```

- If you try to access members declared private you will get a compiler error.

```
#include <iostream>
using namespace std;

class Player {
    private:
    std::string name;
    unsigned int health;
    int xp;
    public:
    void talk(std::string text_to_say);
    bool is_dead();
};


int main() {
    Player frank;
    frank.name = "Frank"; // compiler error.
    frank.health = 100; // compiler error.
    frank.talk("Ready to battle.");
    return 0;
}
```

13.6. Implementing member methods

- Very similar to how we implemented functions.
- Member methods have access to the member attributes.
  - So you don't need to pass them as arguments.
- Can be implemented outside the class declarationg.
  - Need to use `class_name::method_name`.
- Can separate specification from implementation.
  - .h file for the class declaration.
  - .cpp file for the class implementation.

13.6.1. Inside the class declaration

```
class Account {
    private:
    double balance;
    public:
    void set_balance(double bal) {
```

```
        balance = bal;
    }
    double get_balance() {
        return balance;
    }
};
```

13.6.2. Outside the class declaration

Just have the method prototypes inside the class.

```
class Account {
    private:
    double balance;
    public:
    void set_balance(double bal);
    double get_balance();
};

void Account::set_balance(double bal) {
    balance = bal;
}

void Account::get_balance() {
    return balance;
}
```

13.6.3. Separate specification from implementation

An include-guard is included so that if you include the file more than once you won't have a class redefinition error.

```
#ifndef _ACCOUNT_H_ // if this is not defined
#define _ACCOUNT_H_ // define it. Otherwise skip to #endif

#endif
```

Some compilers allow you to include-guard your files with a simple preprocessor directive called `#pragma`.

```
#pragma once
class Account {};

```

Specify the class in a file "Account.h":

```
#ifndef _ACCOUNT_H_
#define _ACCOUNT_H_

class Account {
    private:
    double balance;
    public:
    void set_balance(double bal);
    double get_balance();
```

```
};

#endif
```

Implement the class in “Account.cpp”

```
#include "Account.h"
void Account::set_balance(double bal) {
    balance = bal;
}
double Account::get_balance() {
    return balance;
}
```

13.6.4. Example

```
#include <iostream>
using namespace std;

class Account {
private:
    string name;
    double balance;
public:
    void set_balance(double bal) {balance = bal;}
    double get_balance() {return balance;}

    void set_name(string n);
    string get_name();
    bool deposit(double amount);
    bool withdraw(double amount);
};

void Account::set_name(string n) {
    name = n;
}
string Account::get_name() {
    return name;
}
bool Account::deposit(double amount) {
    balance += amount;
    return true;
}
bool Account::withdraw(double amount) {
    if (balance-amount >= 0) {
        balance -= amount;
        return true;
    }
    else {
        return false;
    }
}

```

```cpp

int main() {
    Account frank_account;
    frank_account.set_name("Frank's account");
    frank_account.set_balance(1000.00);
    if (frank_account.deposit(200.00)) {
        cout << "Deposit OK" << endl;
    }
    else {
        cout << "Deposit Not allowed." << endl;
    }

    if (frank_account.withdraw(500.00)) {
        cout << "Withdraw OK" << endl;
    }
    else {
        cout << "Not sufficient funds." << endl;
    }

    if (frank_account.withdraw(1500.00)) {
        cout << "Withdraw OK" << endl;
    }
    else {
        cout << "Not sufficient funds" << endl;
    }
    return 0;
}
/* OUTPUT:
Deposit OK
Withdraw OK
Not sufficient funds
*/
```

13.7. Constructors and destructors

- Constructors are special member methods that are invoked during object creation, they are commonly used for initialization, constructors have the same name as the class and can be overloaded.
- Destructors are also special member methods, they bear the same name as the class proceeded with a tilde `~`, destructors are invoked automatically when an object is destroyed, they have no return type and no parameters, only 1 destructor is alowed per class and these cannot be overloaded. They are useful to release memory, close files, and other resources. You can call the destructor using `delete`, however if you don't call it, once the object goes out of scope it will automatically be deleted by C++.
- Example of constructors and destructors:

```cpp
class Player {
    private:
        std::string name;
        int health;
        int xp;
    public:
        // Constructors.
        Player();
```

```
        Player(std::string name);
        Player(std::string name, int health, int xp);
        // Destructors.
        ~Player();
};

class Account {
    private:
        std::string name;
        double balance;
    public:
        // Constructors.
        Account();
        Account(std::string name, double balance);
        Account(std::string name);
        Account(double balance);
        // Destructors.
        ~Account();
};
```

13.7.1. Creating objects

- Objects can be called with any type of initialization and depending on the types of the parameters C++ will determine which one to use.

```
#include <iostream>
using namespace std;
class Player {
    private:
        std::string name;
        int health;
        int xp;
    public:
    // Constructors
    Player() {
        cout << "No args constructor called." << endl;
    }
    Player(std::string name) {
        cout << "String arg constructor called." << endl;
    }
    Player(std::string name, int health, int xp) {
        cout << "Three args constructor called" << endl;
    }
    // Destructor.
    ~Player() {
        cout << "Destructor called for " << name << endl;
    }
    void set_name(std::string name) {
        this->name = name;
    }
};

class Account {
    private:
```

```
        std::string name;
        double balance;
    public:
        // Constructors.
        Account();
        Account(std::string name, double balance);
        Account(std::string name);
        Account(double balance);
};

int main() {
    {
        Player slayer;
        slayer.set_name("Slayer");
    }
    /* OUTPUT:
    No args constructor called.
    Destructor called for Slayer
    */
    {
        Player frank;
        frank.set_name("Frank");
        Player hero("Hero");
        hero.set_name("Hero");
        Player villain("Villain", 100, 12);
        villain.set_name("Villain");
    }
    /* OUTPUT: Notice the objects are destroyed in reverse order because they are in a stack.
    No args constructor called.
    String arg constructor called.
    Three args constructor called
    Destructor called for Villain
    Destructor called for Hero
    Destructor called for Frank
    */
    {
        Player *enemy = new Player; // No args constructor.
        enemy->set_name("Enemy");
        Player *level_boss = new Player("Level Boss", 1000, 300); // 3 args constructor called.
        level_boss->set_name("Level Boss");
        delete enemy; // destructor called.
        delete level_boss; // destructor called.
    }
    /* OUTPUT:
    No args constructor called.
    Three args constructor called
    Destructor called for Enemy
    Destructor called for Level Boss
    */
    return 0;
}
```

13.8. The default constructor

- A default constructor does not expect any arguments, it is also called a no-args constructor. If you write no constructor at all for the class then C++ will generate a default constructor that does nothing. This default constructor is the one that is called when you create or instantiate a new object with no arguments.
- We are always free to provide our no args constructor and in fact it is best practice to do so.
- When there is at least one constructor in the class, C++ will not generate a dummy constructor, if we still need the no args constructor we must define it ourselves.

13.8.1. Example

```
#include <iostream>
using namespace std;

class Player {
    // C++ will generate a dummy constructor behind the scenes since there is none defined.
    private:
        std::string name;
        int health;
        int xp;
    public:
    void set_name(std::string name_val) {
        name = name_val;
    }
    std::string get_name() {
        return name;
    };
};

class Player_ {
    // C++ will not generate a dummy constructor, we have provided at least one.
    private:
        std::string name;
        int health;
        int xp;
    public:
    Player_() { // this is the default no args initializer.
        name = "None";
        health = 100;
        xp = 3;
    }
    void set_name(std::string name_val) {
        name = name_val;
    }
    std::string get_name() {
        return name;
    };
};

int main() {
```

```
    Player frank;
    frank.set_name("Frank");
    cout << frank.get_name() << endl;
    Player_ frank_with_constructor;
    cout << frank_with_constructor.get_name() << endl;
    return 0;
}
/* OUTPUT:
Frank
None
*/
```

13.9. Overloading constructor

- Classes can have as many constructors as necessary, each must have a unique signature. The compiler has to be able to determine which to call based on the initialization information provided when creating the objects, if there is any ambiguity, the compiler won't guess, it will instead generate a compiler error. Default constructors are no longer provided for you classes once you've declared at least one.

13.9.1. Example

```
#include <iostream>
using namespace std;

class Player {
    private:
    std::string name;
    int health, xp;
    public:
    Player();
    Player(std::string name_val);
    Player(std::string name_val, int health_val, int xp_val);
};

Player::Player() {
    name = "None";
    health = 0;
    xp = 0;
}

Player::Player(std::string name_val) {
    name = name_val;
    health = 0;
    xp = 0;
}

Player::Player(std::string name_val, int health_val, int xp_val) {
    name = name_val;
    health = health_val;
    xp = xp_val;
}

```

```
int main() {
    Player empty; // None, 0, 0
    Player hero {"Hero"}; // Hero, 0, 0
    Player villain {"Villain"}; // Villain, 0, 0
    Player frank {"Frank", 100, 4}; // Frank, 100, 4
    Player *enemy = new Player("Enemy", 1000, 0); // Enemy, 1000, 0
    delete enemy;
    return 0;
}
```

13.10. Constructor Initialization lists

- So far, all data memeber values have been set in the constructor body. What we really want to do is have the member data values initialized to our values before the constructor body executes, this is more efficient.
- Constructor initialization lists are:
  - More efficient.
  - Initialization list immediately follows the parameter list.
  - Initializes the data members as the objects created.
  - Order of initialization is the order of declaration in the class.
- Take the following example that does NOT use constructor initialization lists:

```
Player::Player() {
    name = "None";
    health = 0;
    xp = 0;
}
```

  - While this works it takes more time and is inefficient. A better way is to provide a constructor initialization list.
  - By the time we get to the statement `name = "None";` the string object has already been constructed and initialized to an empty string, in other words `name = "None";` just assigns to an already initialized object and doesn't initialize the object.
  - So in order to initialize the members as they are created as to have a true initialization we must use a constructor initialization list.
- Take the following example that does use constructor initialization lists:

```
Player::Player() : name{"None"}, health{0}, xp{0} {
}
```

  - The above actually initializes before doing anything, this initializes everything before the body of the constructor is ever excecuted.
  - The data memebers will not be initialized exactly in the order provided in the constructor initialization list, they will be initialized in the order in which they are provided in the class declaration.

13.10.1. Changing previous initialization to use constructor initialization lists

```cpp
#include <iostream>
using namespace std;

class Player {
    private:
    std::string name;
    int health, xp;
    public:
    Player();
    Player(std::string name_val);
    Player(std::string name_val, int health_val, int xp_val);
};

Player::Player() : name{"None"}, health{0}, xp{0} {
}

Player::Player(std::string name_val) : name{name_val}, health{0}, xp{0} {
}

Player::Player(std::string name_val, int health_val, int xp_val) : name{name_val},
 ↪  health{health_val}, xp{xp_val}  {
}

int main() {
    Player empty; // None, 0, 0
    Player hero {"Hero"}; // Hero, 0, 0
    Player villain {"Villain"}; // Villain, 0, 0
    Player frank {"Frank", 100, 4}; // Frank, 100, 4
    Player *enemy = new Player("Enemy", 1000, 0); // Enemy, 1000, 0
    delete enemy;
    return 0;
}
```

13.11. Delegating constructors

- Often the code for constructors is very similar, duplicated code can lead to errors. C++ allows delegating constructors which is code for one constructor can call another in the initialization list, this in turn avoids duplicating code.
- This only works it constructors are called using constructor initialization lists, you cannot call constructors explicitly from the body of another.
- When you call other constructors, the body of the called constructors will excecute and then excecution will continue from the constructor.
- So remember, if you delegate the constructors, both bodies of the constructors will be excecuted, the called constructor's body will be excecuted first and then the rest of the constructor body is excecuted.

13.11.1. Example

```cpp
#include <iostream>
using namespace std;
```

```
3
4  class Player {
5      private:
6      std::string name;
7      int health, xp;
8      public:
9      Player();
10     Player(std::string name_val);
11     Player(std::string name_val, int health_val, int xp_val);
12 };
13
14 Player::Player() : Player {"None", 0, 0 } {
15     cout << "No args constructor" << endl;
16 }
17
18 Player::Player(std::string name_val) : Player{name_val, 0, 0} {
19     cout << "One arg constructor" << endl;
20 }
21
22 Player::Player(std::string name_val, int health_val, int xp_val) : name{name_val},
   ↪  health{health_val}, xp{xp_val}  {
23     cout << "Three args constructor" << endl;
24 }
25
26 int main() {
27     Player empty;
28     Player hero {"Hero"};
29     Player frank {"Villain", 100, 55};
30     return 0;
31 }
32 /* OUTPUT:
33 Three args constructor
34 No args constructor
35 Three args constructor
36 One arg constructor
37 Three args constructor
38 */
```

13.12. Constructor parameters and default values

- We can use default values to simplify our code and reduce the number of overloaded constructors.
- Same rules apply as we learned with non-member functions.
- When doing this it is important to provide unambiguous constructors, for example if we declare one constructor with all default values and then declare a no arg constructor we would get a compiler error because C++ would not know which one to call.

13.12.1. Example

```
1 #include <iostream>
2 using namespace std;
3
```

```
class Player {
    private:
    std::string name;
    int health, xp;
    public:
    Player(std::string name_val = "None", int health_val = 0, int xp_val = 0);
};

Player::Player(std::string name_val, int health_val, int xp_val) : name{name_val},
↪  health{health_val}, xp{xp_val}  {
    cout << "Three args constructor" << endl;
}

int main() {
    Player empty;
    Player frank {"Frank"};
    Player hero {"Hero", 100};
    Player villain {"Villain", 100, 55};
    return 0;
}
/* OUTPUT:
Three args constructor
Three args constructor
Three args constructor
Three args constructor
*/
```

13.13. Copy constructor

- When objects are copied C++ must create a new object from an existing object.
- When is a copy of an object made?
    - Passing object by value as parameter.
    - Returning an object from a function by value.
    - Constructing one object based on another of the same class.
- C++ must have a way of accomplish this copying, so C++ uses a copy constructor. C++ will provide a compiler defined copy constructor if you don't.

13.13.1. 1st usecase: Pass object by-value

Example:

```
Player hero {"Hero", 100, 20};
void display_player(Player p) {
    // p is a copy of hero in this example.
    // use p
    // Destructor for p will be called.
}
display_player(hero);
```

13.13.2. 2nd usecase: Return object by value

Example:

```
Player enemy;
Player create_super_enemy() {
    Player an_enemy {"Super Enemy", 1000, 1000};
    return an_enemy; // A copy of an_enemy is returned, the copy is made by the copy
 ↪  constructor.
}
enemy = create_super_enemy();
```

13.13.3. Construct one object based on another

Example:

```
Player hero{"Hero", 100, 100};
Player another_hero{hero}; // A copy of hero is made.
```

13.13.4. Copy constructors

- If you don't provide your own way of copying objects by value then the compiler provides a default way of copying objects.
- The copy constructors copy the values of ach data memeber to the new object, this is called 'default-memberwise-copy' this basically means that for each data member C++ will copy them to the new object.
- In many cases this is fine, except for one thing, pointers.
- Beware if you have a pointer data member, because the pointer will be copied but data that the pointer is pointing to will not. This is explained in the next video.

13.13.5. Best practice is:

- Provide a copy constructor when your class has raw pointer members.
- Provide the copy constructor with a const reference parameter.
- Use STL classes as they already provide copy constructors.
- Avoid using raw pointer data members if possible.

13.13.6. Declaring the copy constructor

- Copy constructors are declared like this:

```
Type::Type(const type &source);
Player::Player(const Player &source);
Account::Account(const Account &source);
```

- Actually implementing the copy constructor is like this:

```
Player::Player(const Player &source) : name{source.name}, health{source.health},
 ↪  xp{source.xp} {
}
Account::Account(const Account &source) : name{source.name},
 ↪  balance{source.balance} {
}
```

13.13.7. Example

```
#include <iostream>
using namespace std;

class Player {
    private:
    std::string name;
    int health;
    int xp;
    public:
    std::string get_name() {return name;}
    int get_health() {return health;}
    int get_xp() {return xp;}
    Player(std::string name_val = "None", int health_val = 0, int xp_val = 0);
    // Copy constructor:
    Player(const Player &source);
    // Destructor:
    ~Player() { cout << "Destrocutor called for: " << name << endl; }
};

Player::Player(std::string name_val, int health_val, int xp_val) : name{name_val},
 ↪  health{health_val}, xp{xp_val} {
    cout << "Theree-args constructor for " + name << endl;
}

Player::Player(const Player &source) : name{source.name}, health{source.health},
 ↪  xp{source.xp} {
    cout << "Copy constructor - made a copy of: " << source.name << endl;
}

void display_player(Player p) {
    cout << "Name: " << p.get_name() << endl;
    cout << "Health: " << p.get_health() << endl;
    cout << "xp: " << p.get_xp() << endl;
}

int main() {
    Player empty;
    display_player(empty);
    Player copy_of_empty {empty};
    display_player(copy_of_empty);
    Player frank {"Frank"};
    Player hero {"Hero", 100};
    Player villain {"Villain", 100, 55};
```

```
    return 0;
}
/* OUTPUT:
Theree-args constructor for None
Copy constructor - made a copy of: None
Name: None
Health: 0
xp: 0
Destrocutor called for: None
Copy constructor - made a copy of: None
Copy constructor - made a copy of: None
Name: None
Health: 0
xp: 0
Destrocutor called for: None
Theree-args constructor for Frank
Theree-args constructor for Hero
Theree-args constructor for Villain
Destrocutor called for: Villain
Destrocutor called for: Hero
Destrocutor called for: Frank
Destrocutor called for: None
Destrocutor called for: None
*/
```

13.14. Shallow Copying with the Copy Constructor

- Consider a class that contains a pointer as a data member.
- Constructor allocates storage dynamically and initializes the pointer, the destructor releases the memory allocated by the constructor.
- The default copy constructor conducts a memberwise copy of the data members, thus each data member is copied from the source object and the pointer's value is copied, not the values it points to, thus we have two objects that point to the same values and the change in the pointer's values can be done using both objects. This can lead to all sorts of problems for example if one object's destructor is called and releases the memory the other object will still point to the released storage.

13.14.1. Example

```
#include <iostream>
using namespace std;

class Shallow {
    public:
        int *data;
    Shallow(int d); // Constructor.
    Shallow(const Shallow &Source); // Copy constructor.
    ~Shallow(); // Destructor.
};

Shallow::Shallow(int d) {
    data = new int; // allocate storage.
```

```
    *data = d;
}

Shallow::~Shallow() {
    delete data; // free storage.
    cout << "Destructor freeing data" << endl;
}

Shallow::Shallow(const Shallow &source) : data(source.data) {
    cout << "Copy constructor - Shallow" << endl;
}

void display_shallow(Shallow obj) { // obj is a copy of obj1.
    cout << *obj.data << endl;
    // obj goes out of scope and is thus destroyed. The pointer is freed.
}

int main() {
    Shallow obj1 {100};
    display_shallow(obj1);
    cout << *obj1.data << endl; // Referencing a pointer that has already been freed.

    return 0;
}
/* OUTPUT:
Copy constructor - Shallow
100
Destructor freeing data
15869168 // trash data
*/
```

13.15. Deep Copying with the Copy constructor

- Create a copy of the pointed-to data. Each copy will have a pointer to unique storage in the heap. Deep copy when you have a raw pointer as a class member.
- You can also delegate constructors from copy constructors.

13.15.1. Example

```
#include <iostream>
using namespace std;

class Deep {
    public:
    int *data;
    Deep(int d);
    Deep(const Deep &source);
    ~Deep();
};

Deep::Deep(int d) {
    data = new int; // allocate storage.
```

```
    *data = d;
}

Deep::~Deep() {
    delete data; // free storage.
    cout << "Destructor freeing data" << endl;
}

// Deep::Deep(const Deep &source) {
//     data = new int;
//     *data = *source.data;
//     cout << "Copy constructor -deep" << endl;
// }

Deep::Deep(const Deep &source) : Deep{*source.data} /* Delegating to the first
 ↪  constructor */ {
    cout << "Copy constructor -deep" << endl;
}

void display_deep(Deep s) {
    cout << *s.data << endl;
}

int main() {
    Deep obj1 {100};
    display_deep(obj1);
    *obj1.data = 100;
    return 0;
}
/* OUTPUT:
Copy constructor -deep
100
Destructor freeing data
Destructor freeing data
*/
```

13.16. Move constructor

- Sometimes when we excecute the compiler creates unnamed temporary values:

```
int total {0};
total = 100 + 200;
```

- In the above, 100 + 200 is not addressable, it is a right-hand side value which then gets assigned to the left-hand side which is the variable 'total' and the temporary objects.
- Naturally, the problem here is that this temporary values can be very big and with objects, shallow copies and deep pointer copies the overhead can get really big and this is why we need the move constructors come in.
- Sometimes copy constructors are called many times automaticallt due to the copy semantics of C++.
- Copy constructors doing deep copying can have a significant performance (bottleneck).

- C++11 introduced move semantics and the move constructor, the move constructor moves an object rather than copying it.
- Optional but recomended when you have raw pointers.
- Copy elision : C++ may optimize copying away completely (RVO- Return Value Optimization).
- If you don't provide a move construtor then what will be used instead is the copy constructor, for efficiency you should always provide a move constructor.

13.16.1. r-value references

- Used in moving semantics and perfect forwarding.
- Move semantics is all about r-value references.
- Used by move constructor and move assignment operator to efficiently move an objet rather than copy it.
- R-value reference operator (&&).
- Example of l-value reference parameters:

```
int x {10};
int &l_ref = x; // l-value reference.
l_ref = 10; // change x to 10.

int &&r_ref = 200; // r-value ref.
r_ref = 300; // change r_ref to 300.

int &&x_ref = x; // compiler error.
```

- Another example:

```
int x {100}; // x is an l-value.
void func(int &num); // Define function that expects an l-value.
func(x); // x is an l-value so OK.
func(200); // 200 is a r-value.
```

- Example of r-value reference parameters:

```
int x {100}; // x is an l-value.
void func(int &&num); // A funcion that expects an r-value reference.
func(200); // calls func with an r-value.
func(x); // error: x is an l-value.
```

- We could, in any case, overload the functions.

```
int x {100};

void func(int &num); // function that expects a l-value.
void func(int &&num); // function that expects a r-value.

func(x); // calls: func(int &num);
func(200); // calls: func(int &&num);
```

13.16.2. Move constructors

- Move constructors basically is the memberwise copy of objects but the pointer that is copied in one of the objects is nulled out in the other, so just one pointer is being used, you just moved a pointer not all the deep copy things the pointers where pointing to.
- Another way of thinking about it is that the move constructor just steals the data and then nulls out the source pointer.

```cpp
#include <iostream>
#include <vector>
using namespace std;

class Move {
    public:
    int *data;
    Move(int d); // Constructor.
    Move(const Move &source); // Copy constructor.
    ~Move(); // Destructor.
    Move(Move &&source) noexcept; // Move constructor.
};

Move::Move(int d) {
    data = new int;
    *data = d;
    cout << "Constructor called: " << *data << endl;
}

Move::Move(const Move &source) : Move(*source.data) {
    cout << "Copy constructor called -deep copy: " << *data << endl;
}

Move::Move(Move &&source) noexcept : data{source.data} {
    source.data = nullptr;
    cout << "Move constructor - moving resource: " << *data << endl;
}

Move::~Move() {
    cout << ((data == nullptr)? "Destructor called for nullptr.": "Destructo called.")
 ↪  << endl;
    delete data;
}

int main() {
    vector<Move> vec;
    // Move{10} is un-named, C++ will use the copy constructors to make copies,
    // but since we have provided a move constructor C++ wont use the copy cosntructor.
    vec.push_back(Move{10});
    vec.push_back(Move{20});
    return 0;
}
/* OUTPUT:
Constructor called: 10
Move constructor - moving resource: 10
Destructor called for nullptr.
```

```
Constructor called: 20
Move constructor - moving resource: 20
Move constructor - moving resource: 10
Destructor called for nullptr.
Destructor called for nullptr.
Destructo called.
Destructo called.
*/
```

13.17. The 'this' pointer

- 'this' is a reserved word that referes to the current object.
- Contains the address of the object, so it's a pointer to the object.
- Can only beused in class scope.
- All members acces is done via the 'this' pointer.
- Can be used bu the programmer to:
  - Access data members and methods.
  - To determine if two objects are the same (later on).
  - Can be dereferences (*this) to yield the current object.
- This is also used to have same name parameters and class data members.

```cpp
void account::set_balance(double balance) {
    balance = balance; // which balance? The parameter or the data memeber?
}

void account::set_balance(double balance) {
    this->balance = balance; // Now it is unambigous.
}
```

- To determine object identity (this will be useful to overload operators.):

```cpp
int Account::compare_balance(const Account &other) {
    if (this == &other) {
        std::cout << "The same objects" << std::endl;
    }
}
frank_account.compare(frank_account);
```

13.18. Using const with classes

- const classes.
  - Pass arguments to class member methods as `const`.
  - We can also create const objects.
  - What happens if we call member functions on const objects?
  - const-correctness.

- Creating a const object:
  - villain is a const object so it's attributes cannot change.

```
const Player villain {"Villain", 100, 55};
```

  - What happens when we call member methods on a const object?

```
const Player villain {"Villain", 100, 55};
void display_player_name(const Player &p) {
    cout << p.get_name() << endl;
}
display_player_name(villain); // Error. Even if we are not changing the
↪  variables and data members the compiler does not allow it.
```

  - The compiler assumes that any method called can potentially modify the object, so in the class declaration we must declare methods as const as well, that way we tell the compiler that we are sure that that method will not change the object. Once you have declared a method to be const if you change any data member in the method body the compiler will produce an error.

```
class Player {
    private:
    ...
    public:
    ...
    std::string get_name() const; // This method will not modify the data
↪  members.
};
const Player villain {"Villain", 100, 55};
villain.set_name("Nice guy"); // Not allowed.
std::cout << villain.get_name() << std::endl; // OK
```

13.19. Static Class Members

- Class data members can be declared as static.
  - A single data member that belongs to the class, not the objects.
  - Useful to store class-wide information.
  - Example: a variable that counts the instances of the class, it cannot be a member variable it needs to be a static member.
- Class functions can be declared as static.
  - Independent of any objects.
  - Can be called using the class name.
- Example of counting the number of players instanciated in the program:

```
class Player {
    private:
        static int num_players;
    public:
        static int get_num_players() {
            ...
        }
};
```

- A static variable is only intialized ONCE. This happens tipically in the cpp file not the header. You can only instantiate the static variables outside the class, not inside.

```cpp
#include "Player.h"
int Player::num_players = 0; // initialization (only once)
```

- Static methods only have access to static data. It does not access to non-static data members.

```cpp
int Player::get_num_players() {
    return num_players;
}
```

- Lets write an example of how to increment and decrement a static variable that counts the instances of an object.

```cpp
Player::Player(std::string name_val, int health_val, int xp_val) : name{name_val},
 ↪  health{health_val}, xp{xp_val} {
    ++num_players;
}
Player::~Player() {
    --num_players;
}
```

    - Be careful to only have the increment in one constructor, since if you delegate constructors you might be incrementing/decrementing the static variable by more than one.

- Now by calling the method "display_active_players" we can check how many instances we have of the Player class.

```cpp
void display_active_players() {
    cout << "Active players:" << Player::get_num_players() << endl;
}
```

13.20. Example

In "Player.h"

```cpp
#ifndef PLAYER_H
#define PLAYER_H
#include <string>
class Player {
    private:
        static int num_players;
        std::string name;
        int health;
        int xp;
    public:

    std::string get_name() {return name;}
    int get_health() {return health;}
    int get_xp() {return xp;}

    Player(std::string name_val = "None", int health_val = 0, int xp_val = 0);
```

```
    // Copy constructors
    Player(const Player &source) {}
    // Destructor
    ~Player();

    static int get_num_players(); // does not have access to name, health, or xp
}
```

In “Player.cpp”

```
#include "Player.h"
int Player::num_players {0};

Player::Player(std::string name_val, int health_val, int xp_val)
    : name{name_val}, health{health_val}, xp{xp_val} {
    ++num_players;
}
Player::Player(const Player &source) {
}
Player::~Player() {
    --num_players;
}
int Player::get_num_players() {
    return num_players;
}
```

In “main.cpp”

```
#include <iostream>
#include "Player.h"
using namespace std;

void display_active_players() {
    cout << "Active players: " << Player::get_num_players() << endl;
}

int main() {
    display_active_players();
    Player hero{"Hero"};
    display_active_players();
    {
        Player frank{"Frank"};
        display_active_players();
    }
    display_active_players();
    return 0;
}
/* OUTPUT:
0
1
2
1
*/
```

13.21. Structs vs Classes

- `struct` is supported in C++ and it comes from C.
- In addition to define a `class` we can declare a `struct`.
- Essentially the same as a `class` except that all the members are public by default.

13.21.1. When to use a class or struct?

- `struct`
  - Use `struct` for passive objects with public access.
- `class`
  - Use class for active objects with passive access.
  - Implementing getters/setters as needed.
  - Implementing member methods as needed.

13.21.2. Example

- `class`

```
struct Person {
    std::string name; // public
    std::string get_name(); // public
}
Person p;
p.name = "Frank"; // OK
std::cout << p.get_name(); // OK
```

- `struct`

```
class Person {
    std::string name; // private
    std::string get_name(); // private
};
Person p;
p.name = "Frank"; // compiler error, data is private.
std::cout << p.get_name(); // compiler error, data is private.
```

13.22. Friends of a class

- Friends are a very big controversial in C++, the whole argument is wether friends aid encasulation or enhance it. When a class extends friendship to a fucntion of a class then the other class (the one being granted friendship) can now access the private class members.
- Friend:
  - A function of class that has access to private class members.
  - And, that function of or class is NOT a member of the class it is accessing.
- Function:
  - Can be regular non-member functions.

    - Can be member methods of another class.
- Class:
    - Another class can have access to private class members.

13.22.1. Friends of a class

- Friendship must be granted not taken:
    - Declared explicitly in the class that is granting friendship.
    - Declared in the function prototype with the keyword friend.
- Friendship is not symmetric. (not reciprocal)
    - Must be explicitly granted.
        - If A is a friend of B.
        - B is NOT a friend of A.
- Friendship is not transitive:
    - Must be explicitly granted.
        - If A is a friend of B AND
        - B is a friend of C
        - Then A is NOT a friend of C.
- Friendship is not inherited.
- Example of non-member friend functions:

```
class Player {
    friend void display_player(Player &p);
    std::string name;
    int health;
    int xp;
    public:
    ...
};
// Now lets declare a non-member function that will change the private members of
 ↪  Player.
void display_player(Player &p) {
    std::cout << p.name << std::endl;
    std::cout << p.health << std::endl;
    std::cout << p.xp << std::endl;
}
```

- Example of other class member function friends:

```
class other_class {
    ...
    public:
    void display_player(Player &p) {
        std::cout << p.name << std::endl;
        std::cout << p.health << std::endl;
        std::cout << p.xp << std::endl;
    }
};
```

- Declaring an entire class a friend.

```
class Player {
    friend class other_class;
    std::string name;
    int health;
    int xp;
};
```

- Friendship allows you to not write setters and getters just for using the attributes in one other class, it allows you to grant other classes permision to change those variables without having to write setters and getters.
- Friendship doesn't always only serve to save time with writing setters/getters, it works for all sorts of things and must only be used when they make sense.
- Only the minimal friendship should be granted.

Capítulo 14

Operator overloading

14.1. Section overview

- Operator overloading allows the user to use the operator with objects.
- Overloading operators as member functions.
- Overloading operators as global functions.
- Overloading stream insertion (<<) and extraction operators (>>).

14.2. What is operator overloading?

- Using traditional operators such as `+, =, *, etc.` with user-defined types.
- Allows user defined types to behave similar to built-in-types.
- Can make code more readable and writeable.
- Not done automatically (except for the assignment operator). Operator overloading operations must be explicitly defined.

14.2.1. Example

- Suppose we have a Number class that models any number, so we can do anything with numbers such as complex numbers, integers, decimals, etc.
- For this kind of thing we would need to make this class call functions or methods to multiply - for example - and this does not feel natural.

```
1 Number result = multiply(add(a, b), divide(c, d));
2 Number result = (a.add(b)).multiply(c.divide(d));
```

- For C++ it does not make sense to do operations with two objects, thus you need to tell the C++ compiler what you mean when you are doing an operation with two objects, you can do this with methods - like exemplified above - or with overloading operators.
- Only overload operators when it makes sense, otherwise the code is not going to be readable and calling a function/method is better.
- Lets overload the `+, *` operators.

```
Number result = (a + b) * (c /d);
```

- Overloading operators is just for easier programming, behind the scenes we are still calling functions and methods.

14.2.2. What operators can be overloaded?

- The majority of C++'s operators can be overloaded.
- The following operators cannot be overloaded:

`::`	Scope resolution operator
`:?`	Conditional operator
`.*`	member operator
`.`	dot operator
`sizeof`	sizeof operator

14.2.3. Some basic rules

- Precedence and associativity cannot be changed. You cannot make operators so that one operation takes place before another when the operators do not behave like that normally.
- 'arity' cannot be changed (i.e. can't make the division operator unary), this means that if the operator expected to do something with two arguments then when you overload it, it expects two arguments. Example: $a + b$ expects argument a and argument b, you cannot overload it so that it takes in a third or takes in just one, it remains at 2.
- Can't overload operators for primitive type (`int`, `double`, etc).
- Can't create new operators.
- `[]`, `()`, `->`, and the assignment operator (`=`) must be declared as member methods.
- Other operators can be declared as member methods or global functions.

14.2.4. Examples

```
int a = b + c;
a < b;
std::cout << a;
```

```
std::string s1 = s2 + s3;
s1 < s2;
std::cout << s1;
```

```
double a = b + c;
a < b;
std::cout << a;
```

```
std::string my_string = s2 +
↪  s3;
s1 < s2;
s1 == s2;
std::cout << s1;
```

14.2.5. The class we will be using through out this section

Class definition in Mystring.h:

```
#ifndef MYSTRING_H
    #define MYSTRING_H

class Mystring {
private:
    char *str;
public:
    Mystring();
    Mystring(const char *s);
    Mystring(const Mystring &source);
    ~Mystring();

    void display() const;
    int get_length() const;
    const char *get_str() const;
};
#endif
```

Class implementation in Mystring.cpp:

```
// no args constructor.
Mystring::Mystring() : str(nullptr) {
    // Make an empty string
    str = new char[1];
    *str = '\0';
}

// copy constructor.
Mystring::Mystring(const char *s) : str{nullptr} {
    if (s == nullptr) {
        str = new char[1];
        *str = '\0';
    }
    else {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
}

// destructor.
Mystring::~Mystring() {
    delete[] str;
}

// print the string.
void Mystring::display() const {
    std::cout << str << ": " << get_length() << std::endl;
}

// string length.
int Mystring::get_length() const { return strlen(str); }
```

```
32
33 // string getter.
34 const char *Mystring::get_str() const { return str; }
```

14.3. Overloading the assignment operator

- We will overload the assignment operator to mean deep copy from a string object into another string object.
- C++ provides a default assignment operator used for assigning one object to another.

```
1 Mystring s1 {"frank"};
2 Mystring s2 = s1; // NOT assignment it is the same as Mystring s2 {s1}; this is
  ↪  done my constructors to initialize.
3 s2 = s1; // assignment, once initialized the assignment can be called.
```

- Default is memberwise assignment (shallow copy):
  - If we have raw pointer data member we must use deep copy.

14.3.1. Overloading the copy assignment operator (deep copy)

- The assignment must be overloaded as a member function. So we provide a member prototype:

```
Type &Type::operator=(const Type &rhs);
```

  - Replace "Type" with the name of the class in question.

```
1 Mystring &Mystring::operator=(const Mystring &rhs);
2 s2 = s1; // we write this.
3 s2.operator=(s1); // operator= method is called.
```

  - Behind the scenes the method ".operator=" is called.

Overloading the copy assignment operator (deep copy) step by step.

```
1 Mystring &Mystring::operator=(const Mystring &rhs) {
2     if (this == &rhs) {
3         return *this;
4     }
5     delete[] str;
6     str = new char[strlen(rhs.str) + 1];
7     strcpy(str, rhs.str);
8     return *this;
9 }
```

- First, check for self assignment, if the right hand side (rhs) is exactly the same as the left hand side (lhs) then we terminate the function by returning `*this` since we already made the copy and there is nothing else to do. We check the address of both pointers, if they are the same, then we assigning an object to itself.

```
if (this == &rhs) { // p1 = p1;
    return *this; // return current object.
}
```

- Deallocate storage for `this->str` since we are overwriting it. Since we need to deallocate this data because we will allocate the string somewhere else. This is to avoid memory leaks.

```
delete[] str;
```

- Allocate storage for the deep copy.

```
str = new char[std::strlen(rhs.str) + 1];
```

- Perform the copy. Use `strcpy()` function to copy the string one character at a time.

```
std::strcpy(str, rhs.str);
```

- Return the current by reference to allow chain assignment. Remember chain assignment is when we have several assignments in the same line (example: `s1 = s2 = s3 = s4;`)

```
return *this; // s1 = s2 = s3 = s4;
```

14.3.2. Example

Now lets add the overloaded assignment operator to the Mystring class. This is in Mystring.h.

```
#ifndef MYSTRING_H
    #define MYSTRING_H
class Mystring {
private:
    char *str;
public:
    Mystring();
    Mystring(const char *s);
    Mystring(const Mystring &source);
    ~Mystring();

    // copy assignment (overloaded).
    Mystring &operator=(const Mystring &rhs);

    void display() const;
    int get_length() const;
    const char *get_str() const;
};
#endif
```

On Mystring.cpp I add this implementation.

```
Mystring &Mystring::operator=(const Mystring &rhs) {
    std::cout << "Copy assignment" << std::endl;
    if (this == &rhs) {
        return *this;
    }
```

```
    delete[] this->str;
    str = new char[strlen(rhs) + 1];
    strcpy(this->str, rhs.str);
    return *this;
}
```

On the main.cpp:

```
#include <iostream>
#include <cstring>
#include "Mystring.h"
int main() {
    Mystring a {"hello"};
    Mystring b;
    b = a;
    b = "This is a test.";
    return 0;
}
/* OUTPUT:
Copy assignment
Copy assignment
*/
```

14.4. Overloading the move assignment operator

- Move assignment operator is also denoted with the '='.
- You can choose to overload the move assignment operator.
  - C++ will use the copy assignment operator if necessary.

```
Mystring s1;
s1 = Mystring {"Frank"}; // move assignment, this is an unnamed object that is
↪  constructed and has no name (this is an r value), by moving it to s1 it is
↪  granted a name.
```

- The assignment deep copy operator in the last section works with left value references, so values. The move assignment works with right value references. This means temporary unnamed objects that need assignment (move).
- You don't need to provide a move assignment operator, if you don't C++ will use the copy constructor by default. It is always recommended to provide a move assignment operator and a move constructor.
- The main difference between the move assignment operator and the copy assignment operator (seen in the last section) is that in one we are deep copying every character - in this case - of the string, while in this one we are just stealing the pointer from the right hand side and assigning it to the left.

14.4.1. Overloading the move assignment operator

- `Type &Type::operator=(Type &&rhs);`

```
// in the Mystring class.
Mystring &Mystring::operator=(Mystring &&rhs);
```

```
s1 = Mystring{"Joe"}; // move operator= called.
s1 = "Frank"; // move operator= called.
```

14.4.2. Overloading the move assignment operator - steps

```
Mystring &Mystring::operator=(Mystring &&rhs) {
    if (this == &rhs) { // self assignment.
        return *this;   // return current object.
    }
    delete[] str; // deallocate current storage.
    str = rhs.str; // steal the pointer
    rhs.str = nullptr; // null out the rhs object.
    return *this;
}
```

- Check for self assignment:

```
if (this == &rhs) {
    return *this;
}
```

- Deallocate storage `for this->str` since we are overwriting it and we don't want a memory leak.

```
delete[] str;
```

- Steal the pointer from the rhs object and assignment it to this-¿str:

```
str = rhs.str;
```

- Null out the rhs pointer:

```
rhs.str = nullptr;
```

- Return the current object by reference to allow chain assignment.

```
return *this;
```

14.4.3. Example

Comparing the move assignment operator and the copy assignment operator. On the Mystring.h:

```
#include <iostream>
#include <vector>
#include <string.h>

#ifndef MYSTRING_H
    #define MYSTRING_H

class Mystring {
private:
    char *str;
public:
    Mystring(); // no args constructor.
```

```
    Mystring(const char *s); // overloaded constructor.
    Mystring(const Mystring &source); // copy constructor
    ~Mystring(); // destructor
    Mystring(Mystring &&source); // move constructor

    Mystring &operator=(const Mystring &rhs); // copy assignment.
    Mystring &operator=(Mystring &&rhs); // Move assignment.

    void display() const;
    int get_length() const;
    const char *get_str() const;
};

// no args constructor.
Mystring::Mystring() : str(nullptr) {
    // Make an empty string
    str = new char[1];
    *str = '\0';
}

// copy constructor.
Mystring::Mystring(const char *s) : str{nullptr} {
    if (s == nullptr) {
        str = new char[1];
        *str = '\0';
    }
    else {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
}

// destructor.
Mystring::~Mystring() {
    delete[] str;
}

// move constructor
Mystring::Mystring(Mystring &&source) : str{source.str} {
    source.str = nullptr;
    std::cout << "Move contructor used" << std::endl;
}

// move assignment.
Mystring &Mystring::operator=(Mystring &&rhs) {
    std::cout << "Using move assignment" << std::endl;
    if (this == &rhs) {
        return *this;
    }
    delete[] str;
    str = rhs.str;
    rhs.str = nullptr;
    return *this;
}
```

```
67
68
69  // print the string.
70  void Mystring::display() const {
71      std::cout << str << ": " << get_length() << std::endl;
72  }
73
74  // string length.
75  int Mystring::get_length() const { return strlen(str); }
76
77  // string getter.
78  const char *Mystring::get_str() const { return str; }
79
80  // copy assignment operator overloading.
81  Mystring &Mystring::operator=(const Mystring &rhs) {
82      if (this == &rhs) {
83          return *this;
84      }
85      delete[] str;
86      str = new char[strlen(rhs.str) + 1];
87      strcpy(str, rhs.str);
88      return *this;
89  }
90  #endif
```

On the main.cpp:

```
1   #include <iostream>
2   #include <vector>
3   #include "Mystring.h"
4
5   using namespace std;
6
7   int main() {
8       Mystring a{"Hello"}; // overloaded constructor
9       a.display();
10      a = Mystring{"Hola"}; // overloaded constructor
11      a.display();
12      a = "Bonjour";
13      a.display();
14      return 0;
15  }
16
17  /* OUTPUT:
18  Hello: 5
19  Using move assignment
20  Hola: 4
21  Using move assignment
22  Bonjour: 7
23  */
```

14.5. Overloading operators as member functions

- Don't be confused by 'member functions' this just means methods.
- You can overload operators using methods or functions in a program, if you use method the operator will be overloaded for that object. In this case this is member function overloading

14.5.1. Unary operands

- Unary operators as member methods. (`++`, `--`, `-`, `!`)

```
ReturnType Type::operatorOp();
```

```
Number Number::operator-() const;
Number Number::operator++(); // pre-increment
Number Number::operator++(int); // post-increment
Number Number::operator!() const;

Number n1 {100};
Number n2 = -n1; // n1.operator-()
n2 = ++n1; // n1.operator++()
n2 = n1++; // n1.operator++(int)
```

- Example with the Mystring class.
  - Lets use the `operator-` to make lower case a upper case string.

```
Mystring larry1 {"LARRY"};
Mystring larry2;
larry1.display(); // LARRY
larry2 = -larry1; // larry1.operator-()
larry1.display(); // LARRY
larry2.display(); // larry
```

- Code to make a string minus mean lower case:

```
Mystring Mystring::operator-() const {
    char *buff = new char[strlen(str) + 1];
    strcpy(buff, str);
    for (size_t i = 0; i < strlen(buff); i++) {
        buff[i] = tolower(buff[i]);
    }
    Mystring temp {buff};
    delete[] buff;
    return temp;
}
```

14.5.2. Binary operators as member methods

- Binary operators are `+`, `-`, `==`, `!=`, `<`, `>`

```
ReturnType Type::operatorOp(const Type &rhs);
```

```
Number Number::operator+(const Number &rhs) const;
Number Number::operator-(const Number &rhs) const;
bool Number::operator==(const Number &rhs) const;
bool Number::operator<(const Number &rhs) const;

Number n1 {100}, n2 {200};
Number n2 = n1 + n2; // n1.operator+(n2)
n3 = n1 - n2; // n1.operator-(n2)
if (n1 == n2) ... // n1.operator==(n2)
```

- Example of implementation of equality operator in the Mystring class.

```
bool Mystring::operator==(const Mystring &rhs) const {
    if (std::strcmp(str, rhs.str) == 0) {
        return true;
    }
    else {
        return false;
    }
}
```

- Example of string concatenation using the + operator overloading:

```
Mystring larry {"Larry"};
Mystring moe {"Moe"};
Mystring result = larry + stooges; // larry.operator+(stooges);
result = moe + "is also a stooge"; // compiler error, moe.operator+("is also a
↪  stooge"); the right side needs to be the same type as the class you are usig.
result = "Moe" + stooges; // "Moe".operator+(stooges); ERROR.
```

- Mystring operator+ (concatenation):

```
Mystring Mystring::operator+(const Mystring &rhs) const {
    size_t buff_size =
          std::strlen(str)
        + std::strlen(rhs.str) + 1;
    char *buff = new char[buff_size];
    std::strcpy(buff, str);
    std::strcat(buff, rhs.str);
    Mystring temp {buff};
    delete[] buff;
    return temp;
}
```

14.5.3. The complete code would look like this

```
#include <iostream>
#include <vector>
#include <cstring>

using namespace std;

```

```
class Mystring {
private:
    char *str;
public:
    Mystring();
    Mystring(const char *s);
    Mystring(const Mystring &source);
    ~Mystring();

    // copy assignment (overloaded).
    Mystring &operator=(const Mystring &rhs);
    // move assignment (overloaded).
    Mystring & operator=(Mystring &&rhs);
    // operator minus to make lower case.
    Mystring operator-() const;
    // operator plus to make concatenation.
    Mystring operator+(const Mystring &rhs) const;
    // equality operator ==
    bool operator==(const Mystring &rhs) const;

    void display() const;
    int get_length() const;
    const char *get_str() const;
};
// no args constructor.
Mystring::Mystring() : str(nullptr) {
    // Make an empty string
    str = new char[1];
    *str = '\0';
}

// overloaded constructor.
Mystring::Mystring(const char *s) : str{nullptr} {
    if (s == nullptr) {
        str = new char[1];
        *str = '\0';
    }
    else {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
}

// copy constructor
Mystring::Mystring(const Mystring &source)
    : str{nullptr} {
    str = new char[strlen(source.str)+ 1];
    strcpy(str, source.str);
    std::cout << "Copy constructor used" << std::endl;
}



// destructor.
```

```cpp
Mystring::~Mystring() {
    delete[] str;
}

// print the string.
void Mystring::display() const {
    std::cout << str << ": " << get_length() << std::endl;
}

// string length.
int Mystring::get_length() const { return std::strlen(str); }

// string getter.
const char *Mystring::get_str() const { return str; }

// copy assignment operator overloading.
Mystring &Mystring::operator=(const Mystring &rhs) {
    // std::cout << "Copy assignment" << std::endl;
    if (this == &rhs) {
        return *this;
    }
    delete[] this->str;
    str = new char[strlen(rhs.str) + 1];
    strcpy(this->str, rhs.str);
    return *this;
}

// move assignment operator.
Mystring &Mystring::operator=(Mystring &&rhs) {
    if (this == &rhs) { // self assignment.
        return *this;   // return current object.
    }
    delete[] str; // deallocate current storage.
    str = rhs.str; // steal the pointer
    rhs.str = nullptr; // null out the rhs object.
    return *this;
}

// equality operator.
bool Mystring::operator==(const Mystring &rhs) const {
    return (std::strcmp(this->str, rhs.str) == 0);
}

// make lower case with -
Mystring Mystring::operator-() const {
    char *buff = new char[std::strlen(str) + 1];
    std::strcpy(buff, str);
    for (size_t i {0}; i < std::strlen(buff); i++ ) {
        buff[i] = std::tolower(buff[i]);
    }
    Mystring temp {buff};
    delete[] buff;
    return temp;
}
```

```
115
116 // concatenate with +
117 Mystring Mystring::operator+(const Mystring &rhs) const {
118     char *buff = new char[std::strlen(str) + std::strlen(rhs.str) + 1];
119     std::strcpy(buff, str);
120     std::strcat(buff, rhs.str);
121     Mystring temp {buff};
122     delete[] buff;
123     return temp;
124 }
125
126
127 int main() {
128     cout << std::boolalpha << endl;
129     Mystring larry{"Larry"};
130     Mystring moe{"Moe"};
131
132     Mystring stooge = larry;
133     larry.display();
134     moe.display();
135
136     std::cout << (larry == moe) << std::endl;
137     std::cout << (larry == stooge) << std::endl;
138
139     larry.display();
140     Mystring larry2 = -larry;
141     larry2.display();
142
143     Mystring stooges = larry + "Moe";
144
145     Mystring two_stooges = moe + " " + "Larry";
146
147     return 0;
148 }
149
150 /* OUTPUT:
151 Copy constructor used
152 Larry: 5
153 Moe: 3
154 false
155 true
156 Larry: 5
157 larry: 5
158 */
```

14.6. Overloading operators as Global functions

- Overloading operators can be done by member functions (by methods that describe interactions with multiple instances of a class) or by global functions.
- Some operators - for example the unary operators - need to be overloaded using global functions. The reason for this is basically because you do not have two operands.
- These are not member functions, therefore we no longer have a `this` pointer refering to the object in

the left hand side. These operators don't really need access to private attributes in the class, and thus they are overloaded using non member functions. If they must do anything with private attributes it is usually just the matter of declaring them to be friends of the class.

- In the case of a unary operator, only a single parameter is specified in the parameter list.
- Example:

```
Mystring operator-(const Mystring &obj) {
    char *buff = new char[std::strlen(obj.str) + 1];
    std::strcpy(buff, obj.str);
    for (size_t i {0}; i < std::strlen(buff); i++) {
        buff[i] = std::tolower(buff[i]);
    }
    Mystring temp {buff};
    delete[] buff;
    return temp;
}
```

14.6.1. Binary operators as global functions

- +, - ==, <, >, etc:

```
ReturnType operatorOp(const Type &lhs, const Type &rhs);
```

```
Number operator+(const Number &lhs, const Number &rhs);
Number operator-(const Number &lhs, const Number &rhs);
Number operator==(const Number &lhs, const Number &rhs);
Number operator<(const Number &lhs, const Number &rhs);

Number n1 {100}, n2 {200};
Number n3 = n1 + n2; // operator+(n1, n2)
n3 = n1 - n2; // operator-(n1, n2)
if (n1 == n2) { ... } // operator==(n1, n2)
```

- This following example assumes that the function being overloaded is a friend of the Mystring class since we are accessing private attributes, otherwise we would need getters and setters. Overloading the operator==

```
bool operator==(const Mystring &lhs, const Mystring &rhs) {
    if (std::strcmp(lhs.str, rhs.str) == 0) {
        return true;
    }
    else {
        return false;
    }
}
```

- The following example is overloading the operator+ for concatentation as a non-member function:

```
Mystring larry {"Larry"};
Mystring moe {"Moe"};
Mystring stooges {" is one of the three stooges"};
```

```
4
5 Mystring result = larry + stooges; // operator+(larry, stooges);
6
7 result = moe + " is also a stooge"; // operator+(moe, "is also a stooge")
8
9 result = "Moe" + stooge; // ok with non member functions.
```

- The order of the arguments with non member functions is irrelevant, with member functions we needed the left to be an object in order to perform the concatenation.

```
1 Mystring operator+(const Mystring &lhs, const Mystring &rhs) {
2     size_t buff_size = std::strlen(lhs.str) + std::strlen(rhs.str) + 1;
3     char *buff = new char[buff_size];
4     std::strcpy(buff, lhs.str);
5     std::strcat(buff, rhs.str);
6     Mystring temp {buff};
7     delete[] buff;
8     return temp;
9 }
```

14.6.2. Declaring friends of a class

- Friends need to be declared inside the class, it doesn't matter where you declare them, you can declare them inside the private area, public area, or the protected area.
- Example:

```
1 class MyClass {
2     friend bool func1() {};
3     friend bool func2() {};
4 };
```

14.6.3. Example

```
1  #include <iostream>
2  #include <vector>
3  #include <cstring>
4
5  using namespace std;
6
7  class Mystring {
8      friend bool operator==(const Mystring &lhs, const Mystring &rhs);
9      friend Mystring operator-(const Mystring &obj);
10     friend Mystring operator+(const Mystring &lhs, const Mystring &rhs);
11 private:
12     char *str;
13 public:
14     Mystring();
15     Mystring(const char *s);
16     Mystring(const Mystring &source);
17     ~Mystring();
18
19     // copy assignment (overloaded).
```

```
    Mystring &operator=(const Mystring &rhs);
    // move assignment (overloaded).
    Mystring & operator=(Mystring &&rhs);

    void display() const;
    int get_length() const;
    const char *get_str() const;
};
// no args constructor.
Mystring::Mystring() : str(nullptr) {
    // Make an empty string
    str = new char[1];
    *str = '\0';
}
// overloaded constructor.
Mystring::Mystring(const char *s) : str{nullptr} {
    if (s == nullptr) {
        str = new char[1];
        *str = '\0';
    }
    else {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
}
// copy constructor
Mystring::Mystring(const Mystring &source)
    : str{nullptr} {
    str = new char[strlen(source.str)+ 1];
    strcpy(str, source.str);
    std::cout << "Copy constructor used" << std::endl;
}
// destructor.
Mystring::~Mystring() {
    delete[] str;
}

// print the string.
void Mystring::display() const {
    std::cout << str << ": " << get_length() << std::endl;
}

// string length.
int Mystring::get_length() const { return std::strlen(str); }

// string getter.
const char *Mystring::get_str() const { return str; }
// copy assignment operator overloading.
Mystring &Mystring::operator=(const Mystring &rhs) {
    // std::cout << "Copy assignment" << std::endl;
    if (this == &rhs) {
        return *this;
    }
    delete[] this->str;
```

```
    str = new char[strlen(rhs.str) + 1];
    strcpy(this->str, rhs.str);
    return *this;
}
// move assignment operator.
Mystring &Mystring::operator=(Mystring &&rhs) {
    if (this == &rhs) { // self assignment.
        return *this;   // return current object.
    }
    delete[] str; // deallocate current storage.
    str = rhs.str; // steal the pointer
    rhs.str = nullptr; // null out the rhs object.
    return *this;
}
// Equality non-member function
bool operator==(const Mystring &lhs, const Mystring &rhs) {
    return (std::strcmp(lhs.str, rhs.str) == 0);
}
// Make lowercase non-member function
Mystring operator-(const Mystring &obj) {
    char *buff = new char[std::strlen(obj.str) + 1];
    std::strcpy(buff, obj.str);
    for (size_t i {0}; i < std::strlen(buff); i++) {
        buff[i] = std::tolower(buff[i]);
    }
    Mystring temp {buff};
    delete[] buff;
    return temp;
}
// Concatenation non-member function
Mystring operator+(const Mystring &lhs, const Mystring &rhs) {
    char *buff = new char[std::strlen(lhs.str) + std::strlen(rhs.str) + 1];
    std::strcpy(buff, lhs.str);
    std::strcat(buff, rhs.str);
    Mystring temp {buff};
    delete[] buff;
    return buff;
}

int main() {
    cout << std::boolalpha << endl;
    Mystring larry{"Larry"};
    larry.display();

    larry = -larry;
    larry.display();

    Mystring moe{"Moe"};

    Mystring stooge = larry;

    std::cout << (larry == moe) << std::endl;
    std::cout << (larry == stooge) << std::endl;

```

```
    Mystring stooges = "Larry" + moe;
    stooges.display();

    Mystring two_stooges = moe + " " + "Larry";
    two_stooges.display();

    return 0;
}

/* OUTPUT:
Larry: 5
larry: 5
Copy constructor used
false
true
*/
```

14.7. Overloading the stream insertion and extraction operators

- Overloading these operators mean we can now print and insert strings to ouw object very much like the `std::cout`.
- Examples:

```
// <<
Mystring larry {"Larry"};
cout << larry << endl; // Larry
Player hero {"Hero", 100, 33};
cout << hero << endl; // {name: hero, health: 100, xp:33}

// >>
Mystring larry;
cin >> larry;
Player hero;
cin >> hero;
```

- Stream insertion and extraction operators are usually overloaded using non member functions, this makes sense because:
  - It doesn't make sense to implement as member methods:
    - The left operand must be a user-defined class of the type we are overloading.

      ```
      Mystring larry;
      larry << cout; // ???
      ```

    - Not the way we normally use these operators.

      ```
      Player hero;
      hero >> cin; // ???
      ```

    - You can overload them as member functions, however you can overload them as member functions and use the `hero << cout;` code, but it will be weird and not many will understand.
- Example `<<`:

```cpp
std::ostream &operator<<(std::ostream &os, const Mystring &obj) {
    os << obj.str; // if friend function.
    // os << obj.get_str(); // if not a friend function.
    return os;
}
```

- Return a reference to the `ostream` so we can keep inserting.
- Don't return `ostream` by value! It needs to be a reference, otherwise you are copying the entire stream.

- Example `>>`:

```cpp
std::istream & operator>>(std::istream &is, Mystring &obj) {
    char *buff = new char[1000];
    is >> buff;
    obj = Mystring{buff}; // If you have a copy or move assignment.
    delete[] buff;
    return is;
}
```

- Return a reference to the `istream` so we can keep inserting.
- Update the object passed in.

Capítulo 15

Inheritance

15.1. Overview

- What is inheritance and why is it useful?
- Terminology and notation.
- Inheritance vs. composition.
- Deriving classes form existing classes.
  - 3 types of inheritance.
- Protected members and class access.
- Constructors and destructors.
  - Passing arguments to base class constructors.
  - Order of constructor and destructor calls.
- Redefining base class methods.
- Class hierarchies.
- Multiple inheritance.

15.2. What is inheritance?

- What is it and why is it used?
  - Provides a method for creating new classes from exiting classes.
  - The new class contains the data and behaviurs of the existing class.
  - Allow for reuse of existing classes.
  - Allows us to fucus on the common attributes among a set of classes.
  - Allws new classes to modify behaviors of existing classes to make it unique:
    - Without actually modifying the original class.
  - The object that is being inherited from must bear some relation to the inherited object. It does not make sense to inherit the object employee to the object planet.
- Related classes:
  - Player: Enemy, Level boss, Hero, Super player, etcetera.

- Account: Savings account, checking account, trust account, etcetera.
- Shape: line, oval, circle, square, etcetera.
- Person: employee, student, faculty, staff, administrator, etcetera.

- Sometimes the relationship between classes are clear and some may not. But their relationship must make sense.
- An example:
  - Account is a parent class: balance, deposit, withdraw.
  - Savings account inherits from Account: but needs to consider the interest rate.
  - Checking account inherits from Account: but needs to have a minimum balance and a per check fee.
  - Trust account inherits from Account: but it needs an interest rate.
- Without inheritance code tends to duplicate, see the following implementation without inheritance:

```
class Account {
    int balance, deposit, withdraw;
};
class Savings_Account {
    int balance, deposit, withdraw, interest rate;
};
class Checking_Account {
    int balance, deposit, withdraw, minimum_balance, per_check_fee;
};
class Trust_Account {
    int balance, deposit, withdraw, interest_rate;
};
```

- With inheritance we can implemented like this:

```
class Account {
    int balance, deposit, withdraw;
};
class Savings_Account : public Account {
    int interest_rate;
};
class Checking_Account : public Account {
    int minimum_balance, per_check_fee;
};
class Trust_Account : public Account {
    int interest_rate;
};
```

15.3. Terminology and notation

- Inheritance:
  - Process of creating new classes from existing classes.
  - It is a reuse mechanism.
- Single inheritance:

- A new class is created from another 'single' class.

- Multiple inheritance:
  - A new class is created from two or more other classes.

15.3.1. Terminology

- The base class is the parent class, or the super class:
  - The class being extended or inherited from.
- The derived class or child class, sub class:
  - The class being created from the base class.
  - Will inherit attributes and operations from base classes.
- In UML we model class inheritance like this:

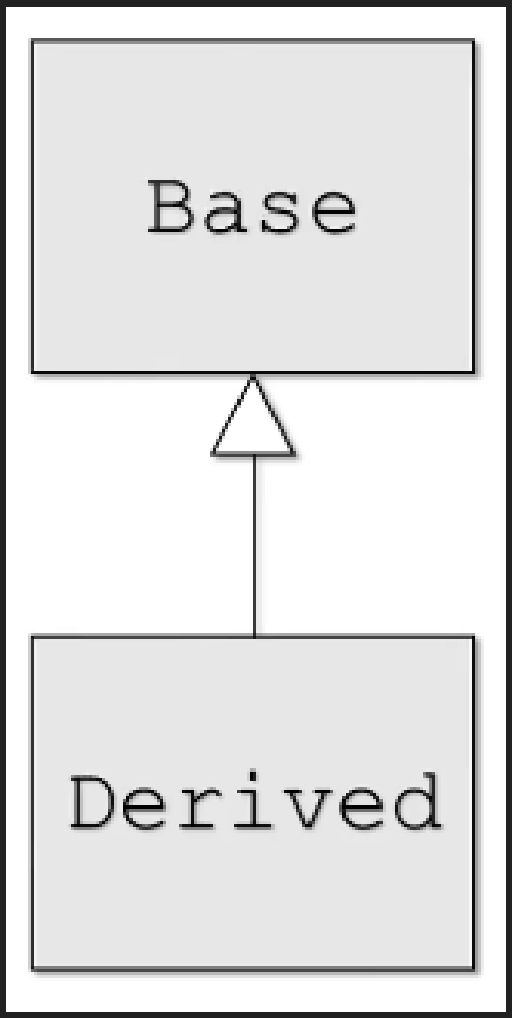


Figura 15.1: UML inheritance diagram

- Is-A" relationship:
  - Public inheritance.

- Derived classes are sub-types of their base classes.
- Can use a derived class object wherever we use a base class object.

- Generalization:
  - Combining similar classes into a single, more general class based on common attributes.
- Specialization:
  - Creating new classes from existing classes proving more speialized attributes or operations.
- Inheritance or class hierarchies:
  - Organization of our inheritance relationships.

15.3.2. Inhertiance

- Class hierarchy denote the order in which the attributes will be shared. The further up we go in the hierarchy we become more general and the further down we become more specialized:
  - A
  - B is derived from A
  - C is derived from A
  - D is derived from C
  - E is derived from D

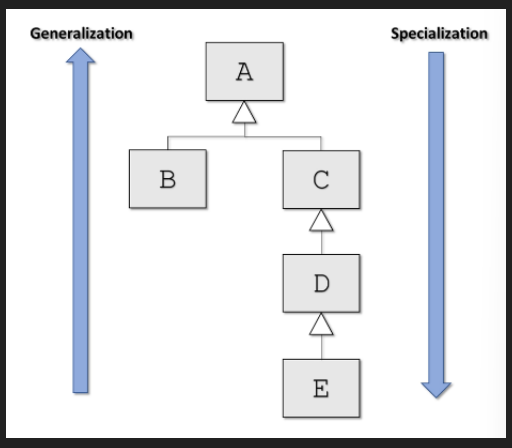


Figura 15.2: Inheritance hierarchy

- Another example:
  - Person
  - Employee is derived from Person
  - Student is derived from Person
  - Faculty is derived from Employee
  - Staff is derived from Employee
  - Administrator is derived from Employee

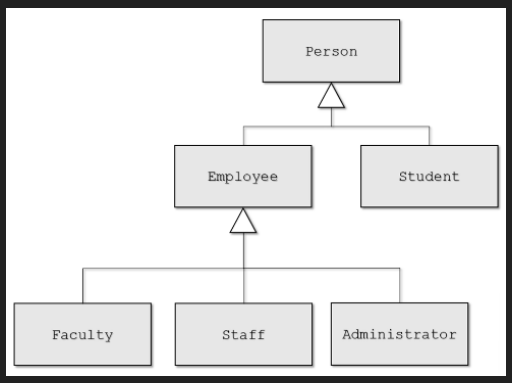


Figura 15.3:

15.4. Inheritance vs composition

- Until now we have seen public inheritance.
- Public inheritance:
  - Create an Is-A" relationship.
    - Employee 'is-a' Person.
    - Checking Account 'is-a' Account.
    - Circle 'is-a' Shape.
- Composition:
  - Creates a 'has-a' relationship.
    - Person 'has-a' Account.
    - Player 'has-a' Special Attack.
    - Circle 'has-a' Location.

15.4.1. Example

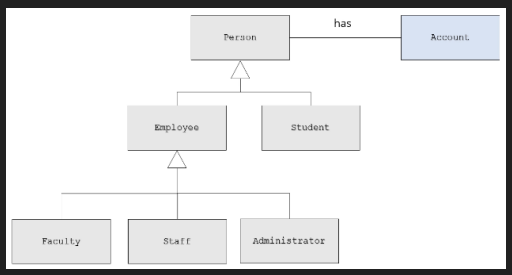


Figura 15.4: Example of composition

- Account does not have an 'is-a' relationship with Person, it has a 'has-a' relation.
- We create the association at Person since this means all subclasses that inherit Person will also have the Account relationship.

- Class members have an 'has-a' relationship with the class being declared.

```
class Person {
private:
   std::string name;    // has-a name
   Account account; // has-a account
};
```

Figura 15.5: 'has-a' relation between data members and class

15.5. Deriving classes from existing classes

- The syntax for the base class:

```
class Base {
    // Base class members ...
};
class Derived : <access-specifier> Base {
    // Derived class members ...
};
```

  - The ⟨access-specier⟩ can be: public, private, or protected.

- Types of inheritance:
  - Public:
    - Most common.
    - Establishes an 'is-a' relationship between derived and base classes.
  - Private and protected:
    - Establishes derived class has a base class" relationship.
    - Is implemented in terms of" relationship.
    - Different from composition.
    - Private and protected inheritance are beyond the scope of this course.

- Example:

```
class Account {
    // Account class members ...
};
class Savings_Account : public Account {
    // Savings_Account class members ...
};
```

  - Savings_Account 'is-a' Account.

```
Account account {};
Account *p_account = new Account();
account.deposit(1000.0);
p_account->withdraw(200.0);
delete p_account;
```

```
Savings_Account sav_account {};
Savings_Account *p_sav_account = new Savings_Account();
sav_account.deposit(1000.0);
p_sav_account->withdraw(200.0);
delete p_sav_account;
```

15.6. Protected emers and class access

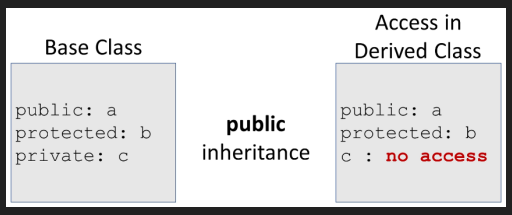
- The protected class member modifier:

```
class Base {
    protected:
        // protected class members ...
};
```

- Give the following properties:
  - Protected class members are accessible from the base class itself.
  - Protected class members are accessible from classes derived from base.
  - Protected class members are not accessible by objects of base or derived.
- C++ handles 3 types of inheritance: public, protected and private.

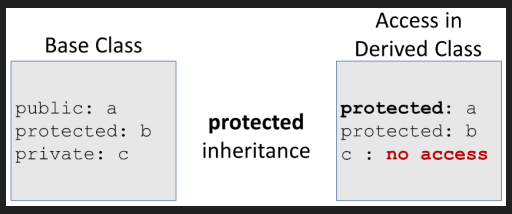
15.6.1. Public inheritance

- When inheriting public class members the base class will inherit the class members are public also, and the inherited class will also have access to the base class members.
- It does not matter how deep the class hierarchy goes the attributes will be accesable by all derived classes.



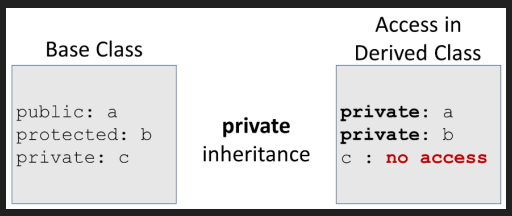
15.6.2. Protected inheritance

- When inherting protected class members the derived class will inherit them as protected also, and will have access to them.
- Protected is not 'is-a' inhertiance.
- When inheriting using protected inheritance public attributes are inhertied as protected. Protected attributes are inherited as protected and private attributes are inaccessible.



15.6.3. Private inheritance

- When inheriting private class members the derived class will inherit them but will not be able to modify them, the derived class will have no access. Any attempt to modify or use them in any way will result in a compiler error.
- Private inheritance is not 'is-a' inheritance.
- When inherting using private, all public and protected attributes are inherited as private, the base class private attributes are inaccessible to the derived class.



15.6.4. Friends of base classes

- Remember that if a class is a friend of a base class then they will have access to public, private and protected information in this class.

15.6.5. Example

```
#include <iostream>
using namespace std;

class Base {
    public:
        int a {0};
        void display() {cout << a << ", " << b << c << endl;} // public method.
    protected:
        int b {0};
    private:
        int c {0};
};

class Derived : public Base {
    public:
    // a will be public, accessible.
    // b will be protected, accessible.
    // c inherited but will not be accessible.
    void access_base_members() {
        a = 100; // OK
        b = 200; // OK
```

```
        // c = 300; // not accessible, compiler error.
    }
};

int main() {
    Base base;
    base.a = 100; // a is public, OK.
    // base.b = 200; // b is protected, compiler error.
    // base.c = 300; // c is private, compiler error.
    Derived derived;
    derived.a = 100; // OK
    // derived.b = 200; // Error, protected.
    // derived.c = 300; // Error, private.
    return (0);
}
```

15.7. Constructors and destructors

- Constructors:
  - A derived class inherits from its base class.
  - The base part of the derived class MUST be initialized BEFORE the derived class is initialized.
  - When a derived object is created:
    - Base class constructor executes then.
    - Derived class constructor executes.
- Destructors:
  - Class destructors are invoked in the reverse order as constructors.
  - The derived part of the derived class must be destroyed before the base class destructor is invoked.
  - When a derived objet is destroyed:
    - Derived class destructor executes then
    - Base class destructor executes
    - Each destructor should free resources allocated in it's own constructors
- A derived class does not inherit the following:
  - Base class constructors
  - Base class destructor
  - Base class overloaded assignment operators.
  - Base class friends.
- However, the derived class constructors, destructors, and overloaded assignment operators can invoke the base class versions.
- C++11 allows explicit inheritance of base 'non-special' constructors with:
  - `using Base::Base;` anywhere in the derived class declaration.
  - Lots of rules involed, often better to define constructors yourself. For example no move constructor is copied, there are a lot of rules.
- Examples:

```cpp
#include <iostream>
using namespace std;

class Base {
    public:
    Base()  {cout << "Base constructor" << endl;}
    ~Base() {cout << "\tBase destructor" << endl;}
};

class Derived : public Base {
    public:
    Derived()  {cout << "Derived constructor" << endl;}
    ~Derived() {cout << "\tDerived destructor" << endl;}
};

int main() {
    {
        Base b;
    }
    {
        Derived d;
    }
}
/* Output:
Base constructor
        Base destructor
Base constructor
Derived constructor
        Derived destructor
        Base destructor
*/
```

15.8. Passing arguments to the base class constructors

- When you have several constructors being used in the base class, sometimes it is not clear which to use when instantiating the base class from the derived class.
- The base part of the derived class must be initialized first.
- How can we control exactly which base class constructor is used during initialization?
- We can invoke the whichever base class constructor we wish to use in the initialization list of the derived class.
- Example:

```cpp
class Base {
    Base();
    Base(int);
};
Derived::Derived(int x) : Base(x) /*optional initializers for derived*/ {
    // derived initializer code.
}
```

- If we dont explicitly invoke the base class constructor then the base class constructor without args will be invoked automatically because the base class must be initialized before the derived class.
- Example:

```cpp
#include <iostream>
using namespace std;

class Base {
    int value;
public:
    Base() : value{0} { cout << "Base no args constructor" << endl; }
    Base(int x) : value{x} {cout << "Int base constructor" << endl; }
};

class Derived : public Base {
    int doubled_value;
public:
    Derived() : Base{}, doubled_value{0} { cout << "Derived no args constructor"
    << endl; }
    Derived(int x) : Base{x}, doubled_value{x*2} { cout << "int derived
    constructor" << endl; }
};

int main() {
    {
        Base b1;
        Base b2(3);
    }
    {
        Derived d1;
        Derived d2(4);
    }
}
/* Output:
Base no args constructor
Int base constructor
Base no args constructor
Derived no args constructor
Int base constructor
*/
```

	Output
`Base base;`	`Base` no-args constructor
`Base base{100};`	int `Base` constructor
`Derived derived;`	`Base` no-args constructor `Derived` no-args constructor
`Derived derived{100};`	int `Base` constructor int `Derived` constructor

15.9. Copy/move constructors and operator= with derived classes

- Copy/move onstructors are not inherited from the base class.
- You may not need to provide your own:

    - Since the compiler can provide default versions and that might work just fine.
- We can explicitly invke the base class versions from the derived class.
- We can invoke the base copoy constructor explicitly:
    - Derived object 'other' will be sliced.

```
Derived::Derived(const Derived &other)
    : Base(other) {
        // code
    }
```

- Copy constructors:
    - The base class:

```
class Base {
    int value;
public:
    // same constructors as previos example.
    Base(const Base &other)
        : value{other.value} {
        cout << "base copy constructor" << endl;
    }
};
```

    - The derived class:

```
class Derived : public Base {
    int double_value;
public:
    // same constructors as previos example
    Derived(const Derived &other)
        : Base(other), doubled_value{other.double_value}
        {
        cout << "derived copy constructor" << endl;
    }
}
```

- Operator=:
    - Base class:

```
class Base {
    int value;
public:
    // same constructors as previos example.
    Base &operator=(const Base &rhs) {
        if (this != &rhs) {
            value = rhs.value; // assign
        }
        return *this;
    }
};
```

    - Derived class:

```
class Derived : public Base {
    int double_value;
public:
    // same constructors as previos example
    Derived &operator=(const Derived &rhs) {
        if (this != &rhs) {
            Base::operator=(rhs); // assign base part.
            doubled_value = rhs.doubled_value; // assign derived part.
        }
        return *this;
    }
}
```

- Copy/move constructors and overloaded operator=:
  - Often you do not need to provide your own.
  - If you DO NOT define them in the derived class:
    - Then the compiler will create them automatically and call the base class's version.
  - If you DO provide derived versions:
    - Then you must invoke the base versions explicitly yourself.
  - Be careful with raw pointers:
    - Especially if base and derived each have raw pointers.
    - Provide them with deep copy semantics.

15.10. Redefining base class methods

- One of the most useful features of inheritance is that base class methods are available to the derived class.
- Derived class can directly invoke base class methods.
- Derived class can override or redefine base class methods.
- Very powerful in the conext of polymorphism (next section).
- To redefine a method in the derived class you just need to provide a method by the same name in the derived class.
- Example:

```
class Account {
public:
    void deposit(double amount) { balance += amount; }
};

class Savings_Account {
public:
    void deposit(double amount) { // redefines base class methods.
        amount += some_interest;
        Account::deposit(amount); // invoke call base class method.
    }
};
```

- Notice that this allows us to not repeat code, since we can make the modification to the amount and then call the base class method.

15.10.1. Static binding of method calls

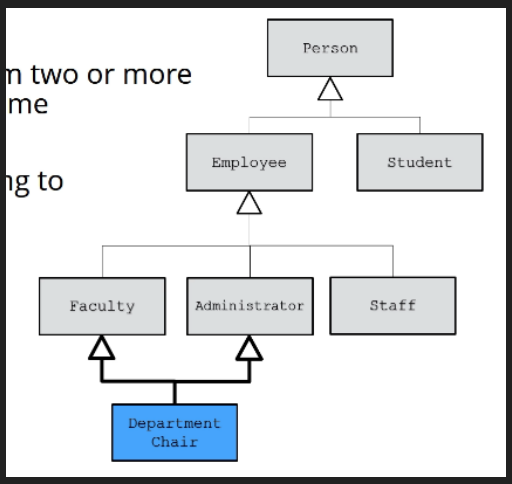
- Binding of which method to use is done at compile time:
  - Default binding for C++ is static.
  - Derived class objects will use Derived::deposit.
  - But, we can explicitly invoke Base::deposit from Derived::deposit.
  - OK, but limited – much more powerful approach is dynamic binding which we will see in the next section.
- Examples of static binding of method calls:

```
Base b;
b.deposit(1000.0); // Base::deposit

Derived d;
d.deposit(1000.0); // Derived::deposit

Base *ptr = new Derived();
ptr->deposit(1000.0); // Base:deposit, because it is apointer to a base class.
```

- By default C++ uses static binding. Which means that it determines the method to use at compile time.

15.11. Multiple inheritance

- A derived class inherits from two or more base classes at the same time.
- The base classes may belong to unrelated class hierarchies.
- A department chair:
  - Is-a faculty and
  - Is-a administrator



- C++ syntax:

```
class Department_Chair : public Faculty, public Administrator {
    ...
};
```

- This feature is beyond the scope of this course.
- It has some compelling use-cases.
- Easily misused.
- Can be very complex.
- We can also use the other types of inheritance, not just public, you can use protected, private etcetera.

15.12. Section challenge

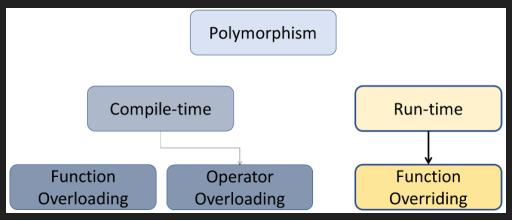
Capítulo 16

Polymorphism

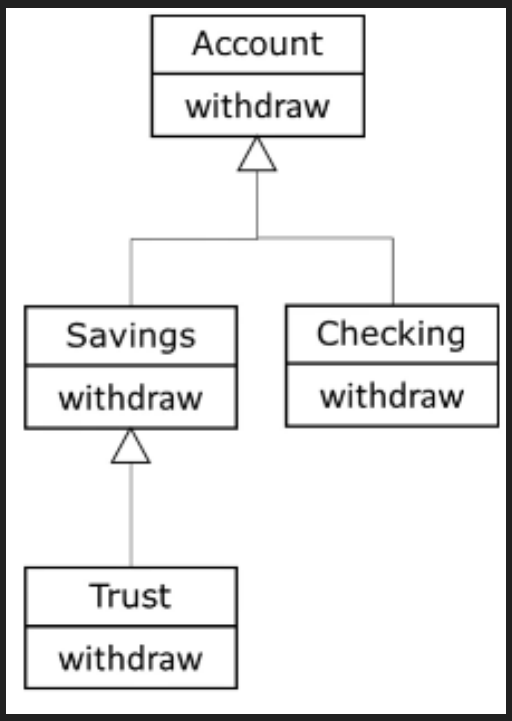
16.1. What is polymorphism?

- It is a fundamental part of object oriented programming.
- Polymorphism(2 types):
  - Compile-time/early binding/static binding
  - Run-time/late binding/dnamic binding
- Runtime polymorphism:
  - Being able to assign different meanings to the same functions at run time.
- Allows us to program more abstractly:
  - Think general vs. especific.
  - Let C++ figure out which function to call at run time.
- Not the default in C++, run-time polymorphism is achived via:
  - Inheritance,
  - Base class pointers and references,
  - Virtual functions.

16.1.1. Types of polymorphism



- The best way to understand polymorphism is by using an example: let's think of a non-polymorphic example first.




```
Account a;
a.withdraw(1000); // Account::withdraw

Savings b;
b.withdraw(1000); // Savings::withdraw

Checking c;
c.withdraw(1000); // Checking::withdraw

Trust d;
d.withdraw(1000); // Trust::withdraw

Account *p = new Trust();
p->withdraw(1000); // Account::withdraw()
                   // Should be Trust::withdraw()
```

- See the problem? it is that the last exapmle gives a problem since we call the wrong function. So the static binding here is giving us problems.
- See another example:

```
void display(const Account &acc) {
    acc.display();
    // will always use Account::display
}
Account a;
display_account(a);

Savings b;
display_account(b);

```

```
Checking c;
display_account(c);

Trust d;
display_account(d);
```

    - Now the problem is that the function parameter expects an account and will call the account's displayed method. This is the problem with static binding.

- Now let's see the same example with dynamic polymorphism.

    - Now withdraw is a virtual method in account.

```
Account a;
a.withdraw(1000); // Account::withdraw

Savings b;
b.withdraw(1000); // Savings::withdraw

Checking c;
c.withdraw(1000); // Checking::withdraw

Trust d;
d.withdraw(1000); // Trust::withdraw

Account *p = new Trust();
p->withdraw(1000); // Trust::withdraw()
```

- Virtual methods allow the compiler to put the instructions necessary to check what kind of data the a pointer is pointing to and delegate the bindign to compilte time. Thus no more bugs.

- The same thing occurs when the above example is called using virtual methods.

```
void display(const Account &acc) {
    acc.display();
    // will use the display method
    // depending on what object is
    // being pointed to at run time.
}
Account a;
display_account(a);

Savings b;
display_account(b);

Checking c;
display_account(c);

Trust d;
display_account(d);
```

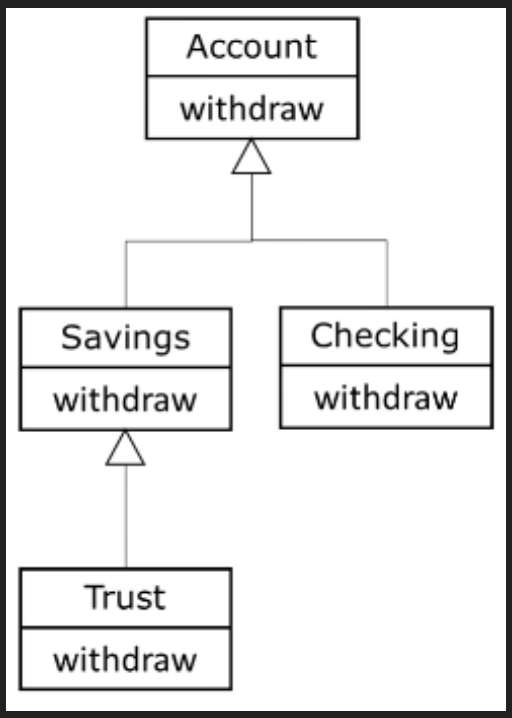
- This allows us to not duplicate code.

- With dynamic polymorphism we can define a function with a base class parameter and any class that derives from the base will get called the appropriate method name.

16.2. Polymorphism

- For dynamic polymorphism we must have:
  - Inheritance
  - Base class pointer or base class references
  - virtual functions
- Supose the following example has been implemented using dynamic polymorphism:

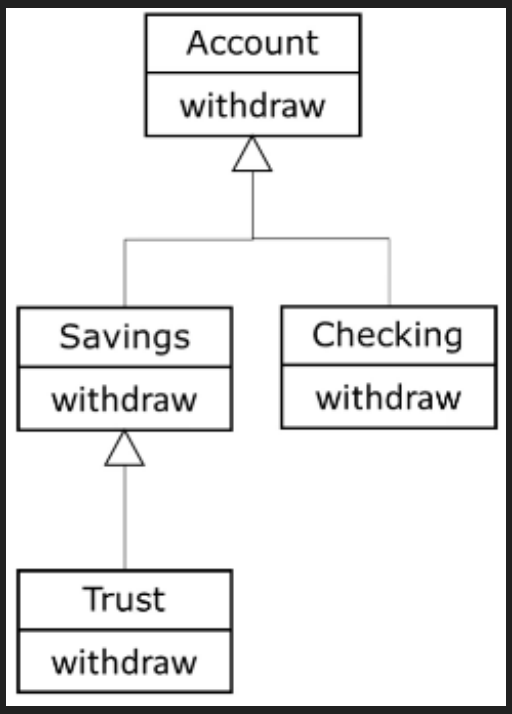
```
Account *p1 = new Account();
Account *p2 = new Savings();
Account *p3 = new Checking();
Account *p4 = new Trust();

p1->withdraw(1000); // Account::withdraw
p2->withdraw(1000); // Savings::withdraw
p3->withdraw(1000); // Checking::withdraw
p4->withdraw(1000); // Trust::withdraw

// delete pointers
```



- Now we suppose we want to create an array that holds pointers to accounts, we need dynamic polymorphism:

```
Account *p1 = new Account();
Account *p2 = new Savings();
Account *p3 = new Checking();
Account *p4 = new Trust();

```

```
Account *array[] = {p1, p2, p3, p4};

for (auto i = 0; i < 4; ++i) {
    array[i]->withdraw(1000);
}
```



- This allows us to program more abstractly.
- We simply withdraw 1000 from each of the accounts regardless of their specific type.

16.3. Virtual functions

- Redfined functions are bound statically.
- Overridden functions are bound statically.
- Virtual functions are overriden.
- Allows us to treat all objects generally as objects of the base class.

16.3.1. Declaring virtual functions

- Declare the function you want to override as virtual in the BASE class.
- Virtual functions are virtual all the way down the hierarchy from this point.
- Dynamic polymorphism only via account class poitner or reference.

```
class Account {
    public:
        virtual void withdraw(double amount);
};
```

- Once we declare a function as virtual it is virtual in all the classes that inherit the base class from that point forward.

16.3.2. Example

- Here is an example of a derived class that has a virtual method.
- The method was declared virtual in the base so the virtual keyword in the derived classes are not necessary, but placing the keyword there is best practice.

```
class Checking : public Account {
public:
    virtual void withdraw(double amount);
};
```